



A survey on autonomous navigation for mobile robots: From traditional techniques to deep learning and large language models

Abderrahim Waga¹ · Said Benhlila¹ · Ali Bekri¹ · Jawad Abdouni² · Fatima Zahrae Saber³

Received: 20 May 2025 / Accepted: 31 July 2025
© The Author(s) 2025

Abstract

Autonomous navigation is a cornerstone of modern robotic systems. This review provides a comprehensive analysis of the landscape of obstacle avoidance and path planning techniques for mobile robots. We categorize and evaluate a range of approaches, beginning with traditional graph-based methods such as A* and Dijkstra, and geometric techniques like Voronoi diagrams and cell decomposition. The review extends to modern metaheuristic algorithms, including genetic algorithms (GA), particle swarm optimization (PSO), and ant colony optimization (ACO). Furthermore, we explore hybrid models that integrate traditional methods with machine learning, such as reinforcement learning (RL) and neural networks (NN). These hybrid approaches aim to address specific challenges, including escaping local minima and enabling real-time decision-making in uncertain environments. A significant focus is placed on the emerging role of Large Language Models (LLMs), analyzing their application in translating natural language commands into navigational actions and improving human-robot interaction. This work critically analyzes the trade-offs of each paradigm—including computational efficiency, scalability, and adaptability across these diverse methods. Finally, this review outlines emerging trends and open challenges, highlighting potential research directions in collaborative robotics, multi-agent systems, and the broader field of mobile robot navigation.

Keywords Autonomous mobile robot navigation · Path planning algorithms · Deep reinforcement learning · Metaheuristic optimization · Large language models in robotics

Said Benhlila, Ali Bekri, Jawad Abdouni and Fatima Zahrae Saber contributed equally to this work

✉ Abderrahim Waga
a.waga@edu.umi.ac.ma
Said Benhlila
s.benhlila@umi.ac.ma
Ali Bekri
a.bekri@umi.ac.ma
Jawad Abdouni
j.abdouni@enim.ma
Fatima Zahrae Saber
fatimazahrae.saber@uit.ac.ma

¹ Computer science, Moulay ismail university, Zitoun, Meknes, Meknes, Zitoun, Meknes 50000, Morocco

² National school of applied sciences of kenitra, Ibn tofail university, Kenitra, Kenitra 14000, Morocco

³ Computer science, Ibn tofail university, Kenitra, Kenitra 14000, Morocco

1 Introduction

Mobile robots have emerged as a common object of study in various applications, from medicine to the military to education, and many other fields (Katona et al. 2024). Mobile robots can reason and act independently and do not require direction from a human operator. Essentially, mobile robots have numerous sensors (cameras, lidar, and ultrasonic sensors) that are intended to provide the mobile robot with as much information as possible by sampling the environment. By processing this sensor data, the robot executes real-time path adjustments and collision avoidance maneuvers to navigate safely through its environment. A navigation system has several subsystems that must incorporate perception, localization, path planning, and obstacle avoidance.

The perception subsystem, in particular, is a critical and rapidly advancing field that forms the foundation for intelligent navigation. Modern robotic perception moves

beyond simple obstacle detection and involves complex visual understanding tasks. These include identifying salient objects by effectively fusing data from multiple sensor modalities like RGB and thermal cameras (Tang et al. 2025), a crucial capability for robust performance in challenging lighting and weather conditions. Furthermore, for a robot to interact intelligently with its environment, it must be able to perform fine-grained recognition—distinguishing between highly similar object sub-categories (e.g., different types of tools) even from limited data (Tang et al. 2022). This often requires sophisticated meta-learning and regularization strategies to enable models to learn quickly and generalize from just a few examples (Tang et al. 2020). The perception challenge extends to dynamic environments, where a robot must also be capable of fine-grained action recognition to understand the behavior of humans or other agents from multiple viewpoints (Tang et al. 2023).

The basic element of the robot is perception, as this subsystem collects all valuable information regarding the environment using sensors located on the mobile robot. We now address the path-planning component (Mohanty et al. 2022). The aforementioned aspect of the navigation system determines the trajectory between the current position of the robot and the final destination. Several methods have been used to address this issue. Figure 1 illustrates the methods used to address this issue.

Several techniques have been investigated in this area. This review explores the studies and improvements made in this field. Path planning is divided into five categories:

- Graph-based search, which systematically uses structured graph-traversal algorithms to explore possible paths. They aim to find the shortest or most efficient path in a defined environment by treating the problem as a graph, where nodes represent points in space, and edges represent paths between them. Common examples include variations of the A* and Dijkstra algorithms.

- Sampling-based: These techniques are based on random sampling points in the environment to build a feasible path incrementally. Sampling-based methods work particularly well in high-dimensional spaces and complex environments in which exhaustive searching throughout the entire space is not feasible. These methods provide a balance between exploration and exploitation to connect these points while avoiding obstacles.
- Gradient-based methods are a category of path planning algorithms that use gradient data, such as from potential fields, to navigate toward a destination while avoiding obstacles. These techniques are characteristically local in their approach, guiding the system by following the direction of steepest descent to minimize a potential function.
- Bio-inspired algorithms: These algorithms have roots in biological phenomena and reproduce natural behaviors in nature, such as swarm intelligence, genetic evolution, and ant colony behavior; all are examples of using bio-inspired algorithms. Bio-inspired algorithms are typically applied to complex optimization problems when either traditional algorithms fail or are slow to find solutions.
- Learning-based: These approaches utilize the progress in machine learning with approaches such as deep learning, reinforcement learning, and large language models, whereby systems can use data and experience to learn optimal paths. LLMs can analyze large amounts of text and associated environmental data to help generate decisions and plans and are particularly useful in situations of dynamics and uncertainty when pre-computed paths may not be ideal.

This review examines various research works in the field of autonomous navigation for mobile robots to address key questions such as:

Question 1: What are the strengths and weaknesses of each method?

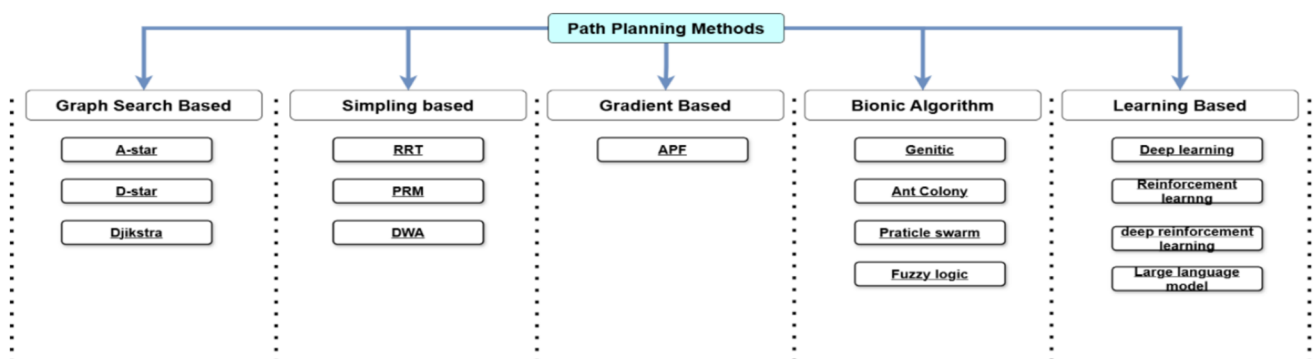


Fig. 1 Overview of path planning methods

- Question 2: Which method requires significant computational resources, and which does not?
- Question 3: What are the most commonly used sensors in each navigation category?
- Question 4: Can these navigation techniques be applied in critical areas such as fully autonomous driving? If yes, to what extent?
- Question 5: What challenges do robots face in achieving autonomous navigation? To answer these questions, we followed a specific methodology for collecting the articles analyzed in this review. We selected recent studies that have addressed obstacle avoidance and path planning. Articles were sourced from well-known scientific databases such as Scopus, Web of Science, and Google Scholar.

Figure 2 shows the total number of publications in the three scientific databases that address the methods presented in Fig. 1. Regarding the keywords used to retrieve these statistics, we combined "mobile robot" with the respective method, such as "mobile robot & A*" or "mobile robot & deep reinforcement learning." The exact date for calculating these values is December 21, 2024.

Figure 2 presents the number of publications indexed in Scopus, Web of Science, and Google Scholar for various path-planning and navigation techniques for mobile robots. The data illustrate the popularity and research focus of each

method, ranging from classical algorithms such as Dijkstra and A* to advanced techniques such as reinforcement learning and imitation learning. From these values, we can observe that classical techniques have maintained a consistent number of publications, reflecting their reliability and foundational role in robotics. Optimization algorithms such as GA and PSO also show high publication counts, particularly in Google Scholar, suggesting their widespread applicability in solving complex path-planning problems. Conversely, newer techniques, such as imitation learning and LLM, have relatively fewer publications, signifying that these methods are still in their early research phase. This reflects a promising area for future exploration, as these techniques leverage advancements in AI to outperform traditional methods. Several reviews addressed problems similar to ours, such as those referenced in Singh et al. (2023); Zhang et al. (2018); Deshpande et al. (2023); Rafai et al. (2022); Liu et al. (2023); Abdouni et al. (2025). While several comprehensive reviews have addressed the multifaceted domain of mobile robot navigation, the rapid evolution of artificial intelligence continually creates new opportunities and research gaps. To contextualize the specific contributions of this survey, the following table provides a direct comparison against other notable reviews. This analysis uses key metrics to highlight the distinctive scope, depth, and timeliness of our work, particularly its focus on bridging traditional methods with the most recent breakthroughs in learning-based navigation and Large Language Models (LLMs). These comparative metrics are summarized in the Table 1.

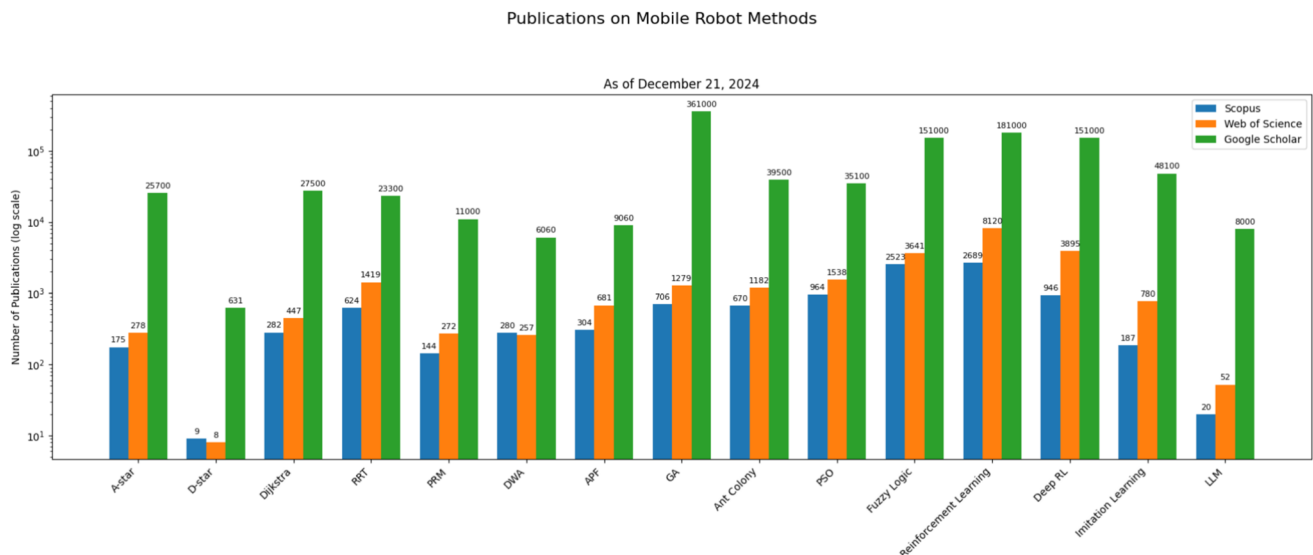


Fig. 2 Comparative analysis of scientific publications on mobile robot methods across scopus, web of science, and google scholar (as of December 21, 2024)

Table 1 A Comparative Analysis of Survey Papers in Path Planning

Study	Covers DRL?	Covers LLMs?	Category of Study	Reference Span	References Cited
Our survey	Yes	Yes	Graph, Sampling, Gradient, Bio-inspired, Learning	1958–2025	183
Singh et al. (2023)	Yes	No	Focused review on Deep Reinforcement Learning	2002–2023	93
Zhang et al. (2018)	No	No	General review of global and local path planning	1959–2018	74
Deshpande et al. (2023)	Yes	No	AI-focused review (Reactive and Classical approaches)	1965–2023	77
Rafai et al. (2022)	Limited	No	General review (Classical and Heuristic approaches)	1959–2022	120
Liu et al. (2023)	No	No	Comprehensive review (Classical, Bio-inspired, and AI)	1959–2023	152
Abdouni et al. (2025)	Yes	No	Challenge-focused review (Graph, Sampling, AI)	1956–2025	92

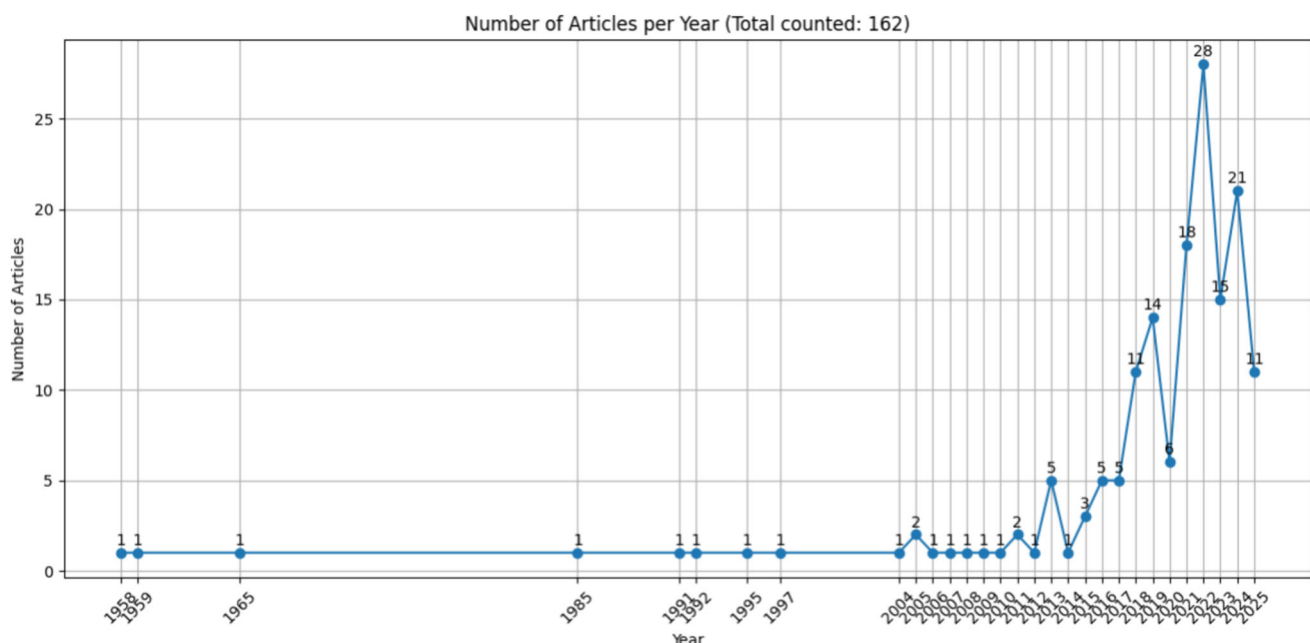
Figure 3 shows the studies we reviewed in this paper, organized by their publication date. Most of these studies were published over the past ten years, and more than 160 works were analyzed.

The rest of this paper is structured as follows. In Section 2, we examine graph-based methods for optimal path planning. Section 3 is devoted to sampling-based approaches, such as rapidly exploring random trees (RRT). Section 4 discusses gradient-based techniques for computing optimal trajectories, including artificial potential fields. Section 5 explores bio-inspired algorithms, such as genetic algorithms and fuzzy logic systems. Section 6 focuses on artificial intelligence-based methods, including deep reinforcement learning and

large language models. Section 7 provides a comparative discussion of the approaches covered in this review. Finally, Section 8 concludes the paper and outlines future research perspectives.

2 Graph search based

This section will present the traditional methods used in autonomous mobile robot navigation. The Subsection 2.1 will focus on the Dijkstra algorithm and other studies based on similar approaches. The Subsection 2.2 will discuss the A* algorithm and the Subsection 2.3 will be dedicated to the D* technique.

**Fig. 3** Distribution of reviewed works by publication year (1958–2025)

2.1 Dijkstra algorithm

Dijkstra's algorithm, introduced by Dutch computer scientist Edsger Wybe Dijkstra in 1956 and published in 1959 (Dijkstra 1959, 2025), is one of the most widely used algorithms for finding the shortest path between nodes in a graph. Originally designed to solve the shortest path problem in directed graphs, it has since found extensive applications in mobile robotics, geographic information systems, computer science, and transportation systems. The algorithm works by expanding outward from a starting point, progressively evaluating the shortest path to other graph nodes. It does so by keeping track of two sets of vertices: one set contains vertices whose shortest path from the starting node has been determined, and the other contains those yet to be processed. The algorithm begins by assigning the starting node a distance of zero and all other nodes a distance of infinity. It then iteratively selects the node with the smallest current distance, evaluates its neighboring nodes, and updates their distances if a shorter path is found through the current node. The flowchart of the Dijkstra algorithm is shown in Fig. 4.

The algorithm begins by initializing two sets: S , containing the starting node, and U , for unvisited nodes. Each iteration checks whether the shortest paths for all vertices have been determined. If so, the algorithm terminates. If not, it selects the vertex K with the smallest distance from the starting node, which is not yet in S . Then, the algorithm updates the distances of neighboring vertices not included in S by considering the newly selected vertex K . Finally, the vertex K is added to set S . These steps are repeated until all vertices' shortest paths are identified. Several modifications to Dijkstra's algorithm have been introduced in various research studies to optimise Dijkstra's algorithm, as referenced. A study specifically (Bhattacharyya and Karmakar 2023) integrated intelligent energy management options with Dijkstra's algorithm. They reduced energy consumption by taking into account road conditions and traffic parameters. Another version of Dijkstra's algorithm (Garg and Devi 2023) included an adaptive penalty function, and optimised pathfinding by shortening computation times and doubling performance in a real time application such as networking and route planning. A study (Wang et al. 2022) integrated statistical characteristics of the localization errors with Dijkstra's algorithm to minimise the cumulative errors where GPS was not available. Jingcun et al. (2007) presented a heuristic to optimise Dijkstra's algorithm where a cost function was used to focus exploration on nodes, thus improving efficiency of the algorithm in systems where nodes were very large. Alshammrei et al. (2022) propose an optimized variant of Dijkstra for mobile robots (MRs), which generates collision-free paths in an environment that is modeled as a directed graph. In this study, the authors used ultrasonic sensors to

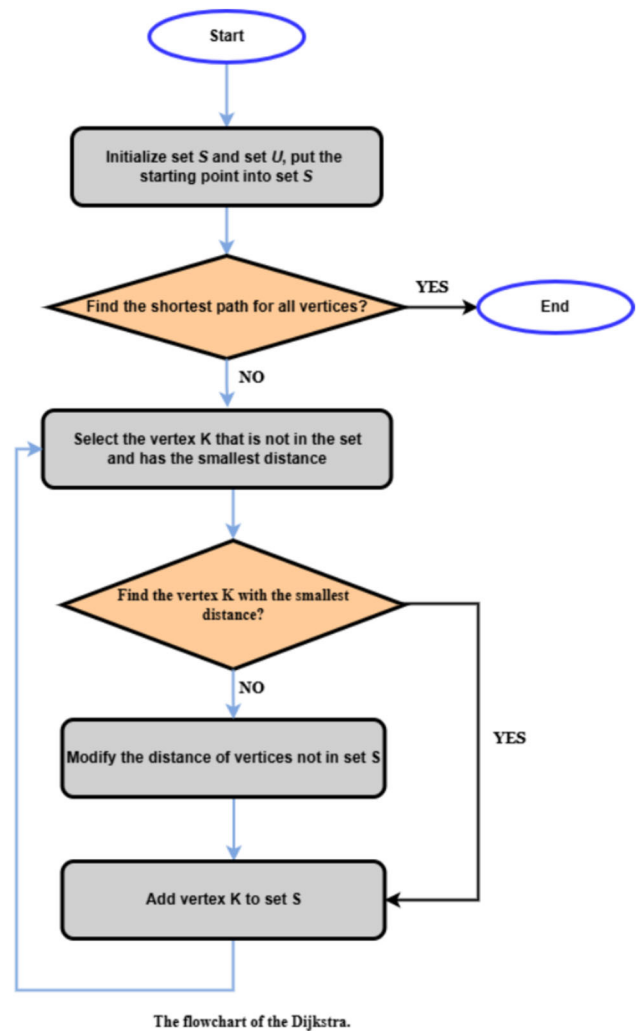


Fig. 4 The flowchart of the Dijkstra

provide real-time obstacle detection, which is useful for dynamically adjusting the MRs planned path when an obstacle is encountered. Reports similar to those in Alshammrei et al. (2022) can be found in Katona et al. (2024), which contains many additional obstacle avoidance algorithms, which include foundational algorithms like Dijkstra's and various varieties of newer algorithms which use genetics and neural networks. These reports outline uses of autonomous navigation. Li (2021) reviews studies about a relaxed Dijkstra algorithm, which accelerates real-time path planning in large environments by generating computed paths in far less time than traditional methods. Additionally, Çelik and Köseoğlu (2023) explores a modified Dijkstra algorithm implemented in a robot operating system (ROS) environment. It compares its performance to standard Dijkstra and other algorithms, ultimately finding it more effective for autonomous mobile robots. The Table 2 illustrates additional research efforts utilizing the Dijkstra method for path planning.

Table 2 Other studies using Dijkstra techniques

Study Details	
Study: (Alyasin et al. 2019)	
Year	2019
Basic Algorithm	Dijkstra
Flags	Optimization: No, Fusion: No, Simulation: No, Real: Yes
Sensors and Material	Robot: A smart mobile robot was built for the experiment, featuring a differential drive with two DC motors and a passive wheel. Sensors: The robot uses two infrared (IR) sensors to follow a black line on a white surface. Materials: The experiment was conducted on a predefined road network created on a physical map, with nodes and weighted paths drawn out.
Test Tools	Software: A Raspberry Pi 3 microcontroller was used as the robot's control unit. The path-finding algorithm was written in the Python programming language. AI Models: The study implements Dijkstra's algorithm to solve the shortest path problem on a known graph.
Performance	The experimental results demonstrated that the system has a high accuracy in selecting the shortest path for the robot to move along. - By applying Dijkstra's algorithm, the mobile robot successfully navigated the network to reach its destination via the optimal path, achieving the goal with minimum cost (weight) and in the shortest time.
Study: (Balado et al. 2019)	
Year	2019
Basic Algorithm	Dijkstra
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: Yes
Sensors and Material	Robot: The study focuses on pathfinding for human users, specifically pedestrians and wheelchair users, by modeling their different motor skills. Sensors: The data is acquired using Mobile Laser Scanning (MLS) systems, a type of LiDAR technology.
Test Tools	Software: Matlab. AI Models: SVM, Dijkstra pathfinding algorithm. Metrics: Accuracy, Processing time, Time to calculate a route.
Performance	The study presents a methodology that successfully generates detailed navigation graphs directly from point cloud data for pedestrian pathfinding. The system automatically reduces point density, corrects data occlusions on sidewalks, and distinguishes between static and dynamic objects to define the navigable space. By creating separate graphs for standard pedestrians and wheelchair users, it generates safe, realistic, and accessible routes that correctly navigate around obstacles, use crosswalks, and account for physical barriers like curbs and stairs.
Study: (Fink et al. 2019)	
Year	2019
Basic Algorithm	Dijkstra
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Robot: The study focuses on planetary rovers, with the Mars Curiosity rover being a primary example. The research highlights the issue of rover wheel damage from hazardous terrain. Sensors: The methodology is designed to use data from orbital instruments like the Mars Orbiter Laser Altimeter (MOLA) and the high-resolution imaging Science Experiment (HiRISE) camera. It can also incorporate in-situ data from rover-based cameras for on-the-ground assessment.
Test Tools	Software: The system presented is the Globally Rover Traverse-Optimizing Planner (GRTOP). Metrics: Lowest number of deployment steps, shortest traverse based on 3D Euclidean distance.
Performance	- The GRTOP system automates and speeds up the process of rover path planning, which is currently a laborious manual task. It can quickly generate globally optimal traverses that satisfy numerous constraints at once. - A key feature is its ability to generate a set of alternates, Pareto-optimal traverses, providing mission planners with a range of optimized choices for different trade-off scenarios.
Study: (Guo et al. 2019)	
Year	2019
Basic Algorithm	Dijkstra
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Robot: The study did not use a robot but instead conducted experiments with 12 different models of real-world gasoline-powered vehicles.

Table 2 continued

Study Details	
Test Tools	Software: MATLAB was used for the simulation and data analysis, while ArcGIS was used to create and manage the road network maps. Datasets: An extensive dataset was generated from approximately 120 hours of real-world driving experiments, resulting in over 380,000 processed data points. Data was collected across various urban road types (expressway, main, secondary) and during different traffic periods (peak, off-peak). Metrics: Shortest time, shortest distance, least fuel consumption.
Performance	The proposed improved Dijkstra algorithm was significantly more efficient, reducing the search area by over 67% and improving the pathfinding operation time by 25% compared to the traditional version.
Study: (Huang 2018)	
Year	2018
Basic Algorithm	Dijkstra
Flags	Optimization: Yes, Fusion: No, Simulation: No, Real: No
Sensors and Material	Robot: The Pioneer robot is mentioned for future work, but the tests were simulations. Sensors: No specific sensors are mentioned in the paper. Test Hardware: An Intel Core i3 processor with 2 GB of RAM was used for the tests.
Test Tools	Software: Simulations were conducted using Visual Studio 2010, Matlab2010b, and C++. Metrics: Performance was judged on running time and path length.
Performance	Speed: The RDJ algorithm is at least 10 times faster than both the DJ and A* algorithms. Accuracy: The path length error is low on smaller maps, with a maximum of 1.27%.
Study: (Yu et al. 2018)	
Year	2018
Basic Algorithm	Dijkstra
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: Yes
Sensors and Material	Robot: A 12-meter driverless electric bus was used as the test platform. Sensors: The bus was equipped with a multi-sensor suite for perception and localization, including lidars, millimeter-wave (MMW) radars, a color camera, and an integrated navigation system with RTK-GPS and an IMU.
Test Tools	Software: ArcGIS. Metrics: The system's performance was evaluated based on control precision, trajectory tracking error, speed response, stability, and its ability to handle various driving scenarios.
Performance	- Simulations and real-world field tests successfully validated the practicality and reliability of the entire path planning and navigation control system. - The hierarchical system demonstrated its ability to handle complex driving situations, successfully performing maneuvers such as turning, merging into traffic, and overtaking other vehicles.

2.2 A* algorithm

The A* algorithm (Foead et al. 2021; A Stars 2025) is an enhancement of Dijkstra's algorithm, which is designed to find the shortest path in a graph efficiently. Like Dijkstra's method, it iteratively updates the weighted values of neighboring nodes. At each step, the algorithm selects the child node with the smallest total cost to continue exploring until all relevant nodes have been traversed. However, A* is distinguished by introducing a heuristic function that intelligently guides the search towards the target. The core of A* lies in its evaluation function, $f(n) = g(n) + h(n)$, where - $g(n)$ represents the actual cost from the starting node to the current node n . - $h(n)$ is the heuristic that estimates the cost from n to the target node, often using Euclidean distance. When the path length $g(n)$ remains constant, the search primarily relies on the heuristic $h(n)$. As the algorithm moves closer to the target, $h(n)$ decreases, resulting in a lower $f(n)$ value.

This ensures that A* always advances toward the goal, prioritizing the most promising paths. The flowchart of the A* algorithm is shown in Fig. 5.

A* outperformed Dijkstra's algorithm by incorporating real and estimated costs, enabling faster convergence. This makes it particularly effective for path planning tasks, such as guiding a mobile robot to its destination, where search efficiency is critical. By focusing the exploration along the direction of the target, A* achieves a more optimal performance than the exhaustive search employed in Dijkstra's approach. This paper (Guruji et al. 2016) seeks to present a modification of the A* algorithm to reduce processing time with maintain reasonable navigation of the robot in a dynamic environment. In comparison to the original A* algorithm, this method is more efficient, as it only calculates the value of the heuristic function at the onset of each collision phase (not at each node) which increases efficiencies and reduces unnecessary work before making a decision.

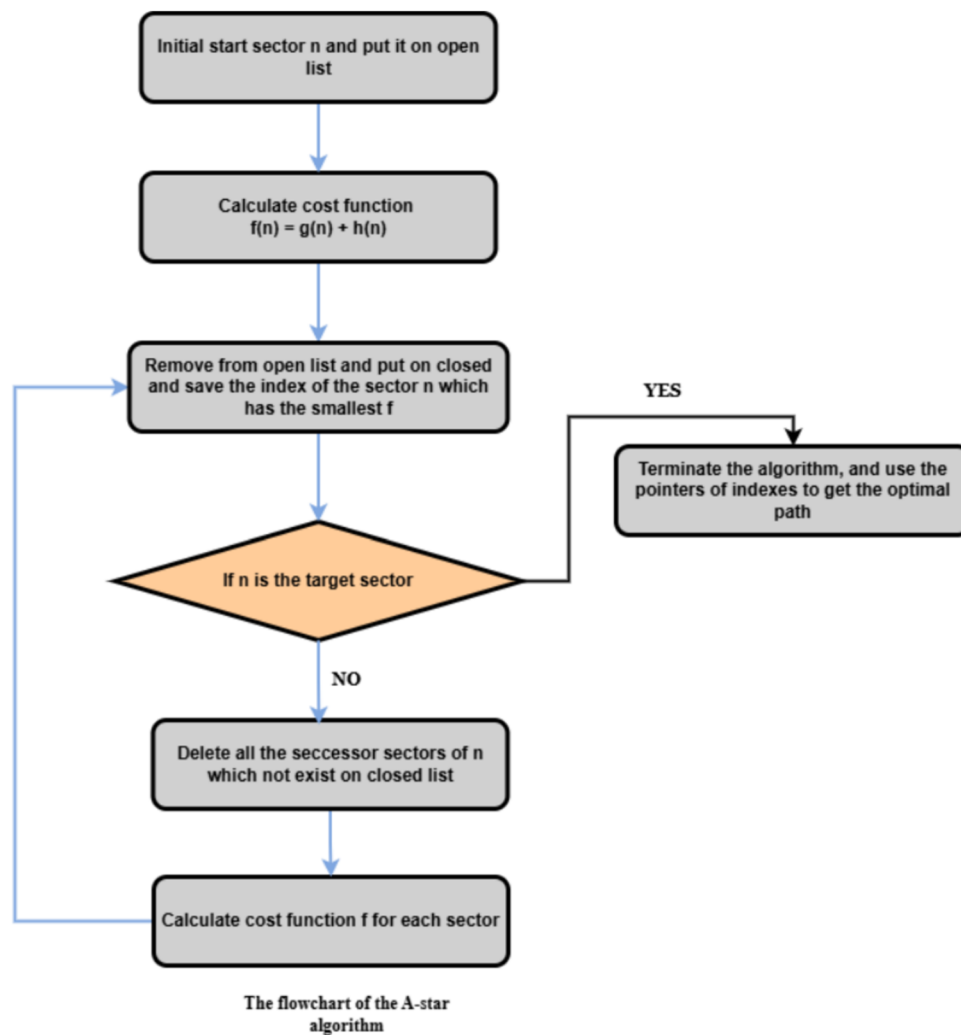


Fig. 5 The flowchart of the A* algorithm

This novel algorithm achieves processing times up to 95%, thus increasing its value for problems complaining on slower processors. There are limitations to the algorithm: in some instances, path length was slightly longer than those determined by the A*; particularly it matters where the start and end points are in relation to the obstacle and the proximity of the obstacles. In addition, the algorithm is based on a fixed work environment, which may not work in scenarios where there are numerous dynamic changes. Another paper (Erke et al. 2020) provides an innovative path planning algorithm for autonomous land vehicles (ALVs) and was an enhancement to the basic A algorithm by introducing some improvements. The authors established a universal evaluation standard, allowing the performance of different path planning algorithms to be measured and allowing optimal parameter choice. Secondly, the authors introduced a guideline-based heuristic function which can be used for more efficiency in searches and for desired obstacle avoid-

ance by supplying key points around obstacles taught in (how to) achieve avoidance of obstacles before reaching them. Additionally, the authors employed a variable-step approach to reduce computation for a more real-time application (as needed) as computing and resources were limited. Experimental results show that this improvement outperformed the traditional A methods by decreasing number of expansion points and computation time. However, it also had a couple of limitations. Highly dynamic and changeable work environments presents a challenge as more obstacles are introduced quickly. The other limitation requires the use of guidelines which may not be used in some facilities. The first of the studies (Yonetani et al. 2021; Franssen and Van Eekelen 2023; Xin et al. 2019) presents Neural A, the data-driven search method for path planning where it reworks the classic A algorithm to be differentiable such that Neural A can connect with a convolutional encoder to produce end-to-end trainable neural network planner. Neural A* can determine path planning

problems by encoding the problem as a guidance map and performing a differentiable A* search along the guidance map. In the second study, the authors provide a new heuristic to the A* algorithm that solves path planning for automated guided vehicles in a geometric graph that determines the lowest-cost path. The new heuristic includes turning costs; thus avoiding the traditional problem of neglecting turning costs or rigidity such as not being able to do a moving turn on a specific graph structure. The authors of the third study enhanced the A* algorithm for use in path planning of mobile robots while still dealing with a few inflection points and taking into consideration the smoothness of paths between nodes by replacing original nodes from sub optimal paths with completely optimized versions through the setting of a threshold limit on the open list as a priority for node expansion, applying the Floyd algorithm on the way to determine the shortest path length to determine the majority of the path length was not sub-optimally bloated with nodes and applying some soft-kink/smooth technique on top of the newly optimized way of navigating around sub optimal bloat. In this way they produced paths that were shorter, smoother and less costly from an efficiency standpoint through little to this drastic change from de-spair while still being reasonable as a path planning time. Path planning opinions on A* algorithm research have also been done by other researchers explorations of the A* for their purposes. Xin et al. (2019) applied a threshold value to facilitate prioritization of nodes, which could reduce search grid points and improve the smoothness of geometrical optimization. Fransen and Van Eekelen (2023) described a heuristic method where lacked turning costs in geometric graphs and on average was quicker to find a valid path with less iteration. Yonetani et al. (2021) presented Neural A*, an algorithm that is a first-order differentiable A* algorithm that first have the advantage of improving optimality over search, and predicting more realistic human trajectories. Erke et al. (2020) improved the A* algorithm in ALVs by using lessening perceptions (guidelines and key points) of collision-free space obstacles to improve performance in various environments. Guruji et al. (2016) Was able to properly recalibrate the heuristic function in A* algorithm to evaluate before only collision, and significantly reduced processing times, improving search efficiency by up to 95%. The Table 3 shows other studies that use the A* algorithm.

2.3 D* algorithm

In order to tackle the issue of environmental variation the D* algorithm (Optimal and Efficient 2025) was proposed by Anthony Stentz in 1994, as a variant of the A algorithm. D* allows robots to update their path based on information pro-

vided by the environment, meaning that they can alter their path when they run into obstacles or experience environmental changes. The algorithm minimizes the amount of space searched through efficient searching while simultaneously altering the cost of traversing edges based on newly gathered information. Since then we have seen refinements to D* with proposals such as Field D* and Theta*, which improved the speed and flexibility of path planning as performed by a mobile robot in a dynamic and uncertain environment. These variations on D* algorithm were concerned mainly with minimizing overhead, and improved path quality and time to react. The flowchart of the D* algorithm is shown in Fig. 6.

The D* algorithm for path planning in dynamic environments, where obstacles may move or change over time, is illustrated in the diagram as a flowchart. The process begins with making the starting state the last state in which the data structures are built. Then, the D* algorithm uses the information from the environment and computes the shortest path. The D* algorithm then evaluates if there is no change in the cost to traverse the two nodes g-last and g-current. If there is no change detected from g-last to g-current then the process will be terminated as we have confirmed that the path from g-last to g-current is optimal, and therefore no more finite value could be evaluated. Nevertheless, in the case where there are updates in the graph or obstacles introduced to the graph (which can include the movement of obstacles or obstacles appearing), the algorithm searches for nodes containing affected references and updates the respective cost. After identifying the affected references, the algorithm computes the shortest path again with the new environment. This will continue to iterate until there are no more updates detected; ultimately allowing the algorithm to effectively search for other finite values in the graph, which indicates it is changing to accommodate new environmental changes with an optimal path for the robot. Liu et al. (2023) proposed the PRM-D* approach, which extends the classical probabilistic roadmap method (PRM) with the D* algorithm to improve obstacle avoidance in dynamic environments. The combined use of the probabilistic roadmap method with a dynamic obstacle avoidance algorithm offers the possibility for more rapid planning and provides a good performance in real-world scenarios. In a similar research direction, the authors in Raheem and Hameed (2018) proposed a heuristic D* path planning algorithm joined with particle swarm optimization (PSO) to minimize the path length when moving obstacles were present, demonstrating the performance of their pipeline in simulation results. Both research directions indicated the value of joining state-of-the-art algorithms to improve path planning in dynamic environments in a safe and efficient manner. The Table 4 shows other studies that use the D* algorithm.

Table 3 Other Studies Using A* Techniques

Study Details	
Study: (Ziegler et al. 2008)	
Year	2008
Basic Algorithm	A*
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: Yes
Sensors and Material	Robot: An autonomous car named "ANNIEWAY". Sensors: A 360-degree high-definition laser range scanner.
Test Tools	AI Models: An A* search algorithm guided by a hybrid heuristic cost function. Metrics: Search time and the ability to perform complex maneuvers (parking, turns, long-distance navigation).
Performance	- The system was successfully used in the DARPA Urban Challenge 2007. - The Voronoi heuristic enables fast pathfinding even in complex, maze-like areas.
Study: (Zhenyang and Wei 2022)	
Year	2022
Basic Algorithm	A* + DWA
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: Yes
Sensors and Material	Robot: A general mobile robot simulated within the Robot Operating System (ROS). Test Hardware: A 64-bit Windows 10 machine with 32GB of memory.
Test Tools	Software: MATLAB 2021a and ROS. AI Models: A fusion algorithm that combines an improved A* for global planning with an adaptive Dynamic Window Approach (DWA) for local obstacle avoidance. Metrics: Path length, calculation time, and the number of iterations.
Performance	- The improved A* algorithm produces shorter and smoother paths compared to the traditional A*. - The adaptive DWA significantly reduces calculation time and avoids paths that are dangerously close to obstacles.
Study: (Zagradjanin et al. 2021)	
Year	2021
Basic Algorithm	A* + FL
Flags	Optimization: Yes, Fusion: Yes, Simulation: No, Real: No
Sensors and Material	Robot: A system of three homogeneous transport robots. Sensors: Robots are equipped with sensors having a range of one grid cell.
Test Tools	Software: Matlab simulation of the cloud architecture. Metrics: Total path length of all robots and the number of path corrections needed.
Performance	- The cloud architecture successfully offloads complex tasks from the individual robots. - Using the FIS to avoid crowds reduced the total path length by approximately 2%.
Study: (Martins et al. 2022)	
Year	2021
Basic Algorithm	A*
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Robot: A general mobile robot; the work is simulation-based. Test Hardware: A computer with an Intel Core i7-8850H CPU.
Test Tools	Software: The algorithm was implemented in Python 3.8.3, and the workspace map was created with Inkscape-0.92.1. Metrics: Algorithm process time, path smoothness, path length, and the number of points in the path.

Table 3 continued

Study Details	
Performance	Processing Time: After an initial run to generate the map file, IMOA* reduced the algorithm processing time by an average of 99.98% compared to the traditional A*, which had to regenerate the map on every run. Path Smoothness: The path's smoothness was improved by an average of 45%.
Study: (He et al. 2022)	
Year	2021
Basic Algorithm	A*
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Robot: Unmanned ships, also known as Maritime Autonomous Surface Ships (MASS). Sensors: Automatic Identification System (AIS) and Radar. Test Hardware: An Intel i5 processor with 16 GB of memory.
Test Tools	Software: MATLAB R2021a. AI Models: A novel Dynamic Anti-collision A* (DAA*) algorithm. Metrics: Calculation time, path length, and collision risk.
Performance	- Generates safer paths that respect ship domains and maritime rules, unlike traditional A*. - Demonstrates a 15% reduction in search time compared to other modified A* methods.
Study: (Ferguson and Stentz 2006)	
Year	2006
Basic Algorithm	A*
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: Yes
Sensors and Material	Robot: Deployed on various real-world systems, including Pioneer robots and Mars Exploration Rovers. Sensors: General onboard sensors for environmental mapping. Test Hardware: A 1.5 GHz Powerbook G4 laptop.
Test Tools	AI Models: Field D*: An extension of D* that uses linear interpolation to create paths with continuous headings, avoiding the rigid angles of standard grid planners. Metrics: Path cost (quality), planning and replanning time, and memory usage.
Performance	- Field D* produces smoother, more direct, and lower-cost paths than standard grid-based planners like D* Lite. - Paths from Field D* were on average 4% lower cost than those from D* Lite in experiments.

Table 4 Other studies using D* techniques

Study Details	
Study: (Kadry et al. 2020)	2020
Year	D*
Basic Algorithm	Optimization: Yes, Fusion: No, Simulation: No, Real: No
Flags	Robot: General "mobile robots". Sensors: The paper recommends using infrared rangefinders, including one on a 360-degree servo, and suggests a Kalman filter to improve scan data.
Sensors and Material	AI Models: A modification of the D* algorithm. Metrics: The algorithm is compared based on decision speed and path length.
Test Tools	- The modified algorithm makes decisions faster by eliminating the need for positioning sensors like GPS. - It is designed to correct inefficient movements that the basic D* algorithm might make when near obstacles like walls.
Performance	
Study: (Likhachev et al. 2005)	2005
Year	D*
Basic Algorithm	Optimization: Yes, Fusion: No, Simulation: Yes, Real: Yes
Flags	Robot: The algorithm was tested on two real-world platforms: an ATRV vehicle and a Segway Robotic Mobility Platform. It was also tested in a simulation of a 3-degree-of-freedom robotic arm. Sensors: The ATRV robot used two laser range finders and an inertial measurement unit (IMU).
Sensors and Material	AI Models: The paper introduces Anytime Dynamic A* (AD*), a novel algorithm that combines the features of two other algorithms: ARA* (Anytime Repairing A*) and D* Lite. Metrics: The performance was evaluated based on solution cost (path quality), number of state expansions (computational effort), and the ability to work within a fixed deliberation time.
Test Tools	- AD* successfully combines the benefits of anytime planning and incremental replanning, allowing it to efficiently find and improve solutions in complex, dynamic environments. - Compared to D* Lite, AD* produced solutions of much higher quality because it could improve its suboptimality bound over time.
Performance	
Study: (Alferov et al. 2024)	2024
Year	D*
Basic Algorithm	Optimization: Yes, Fusion: No, Simulation: No, Real: No
Flags	Robot: A general mobile robot (wheeled, tracked, etc.). Sensors: Requires a rangefinder or a stereo camera for obstacle detection.
Sensors and Material	AI Models: This system uses the modified D* algorithm for path planning and adds a reinforcement learning layer to correct movement errors.
Test Tools	Metrics: The goal is to minimize the robot's deviation from the calculated trajectory.
Performance	- The use of reinforcement learning helps minimize path deviations caused by external factors like friction or internal motor variations. - The system is versatile, applicable to different types of robots, and can work indoors without GPS. - It reduces the cost of the robot's equipment by being less demanding on computing systems and requiring fewer sensors for operation.

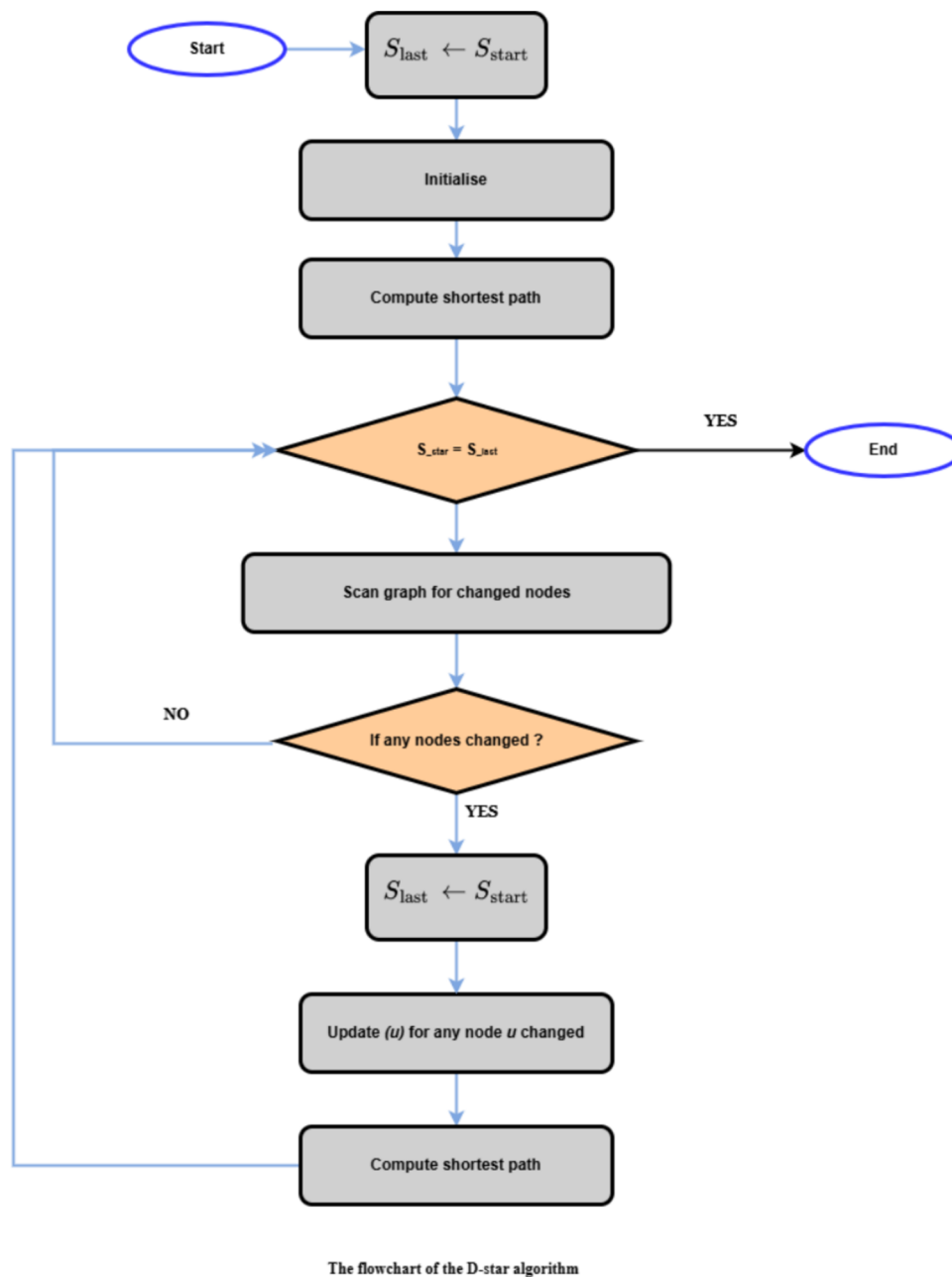


Fig. 6 The flowchart of the D* lite algorithm

3 Sampling based

After surveying the state-of-the-art approaches in Section 2, we now turn to this section on sampling-based planners, where random exploration strategies efficiently cover the configuration space.

3.1 The rapidly-exploring random tree (RRT)

The RRT algorithm (Steven 2025) creates a search tree with random sampling in configuration space. RRT has sev-

eral advantages that differentiate it from classical methods: Firstly, RRT can find a feasible path more readily than classical methods because it doesn't require explicit modelling of the environment. Secondly, RRT can allow for high-dimensional and complex situations, where classical algorithms would fail altogether. Thirdly, RRT finds a viable path, but not necessarily the best path. The RRT algorithm generates a tree-like search structure by randomly sampling points in the configuration space, starting from the initial condition of x_{init} to form the root node of the tree. RRT and its variations have been of great interest to the mobile

robot path planning community because these algorithms can be adapted to difficult and complex high-dimensional environments. Typical RRTs also produce paths with a low total cost and are not guaranteed to generate the lowest cost paths as with classical path planning algorithms. RRT also suffers from slow convergence and unstable behaviors whereas classical path planning algorithms slowly show uni-modal consequences. The flowchart of the RRT algorithm is shown in Fig. 7.

The flowchart demonstrates the path planning process using the rapidly-exploring random tree (RRT) process. The RRT process begins by identifying the state space X , the

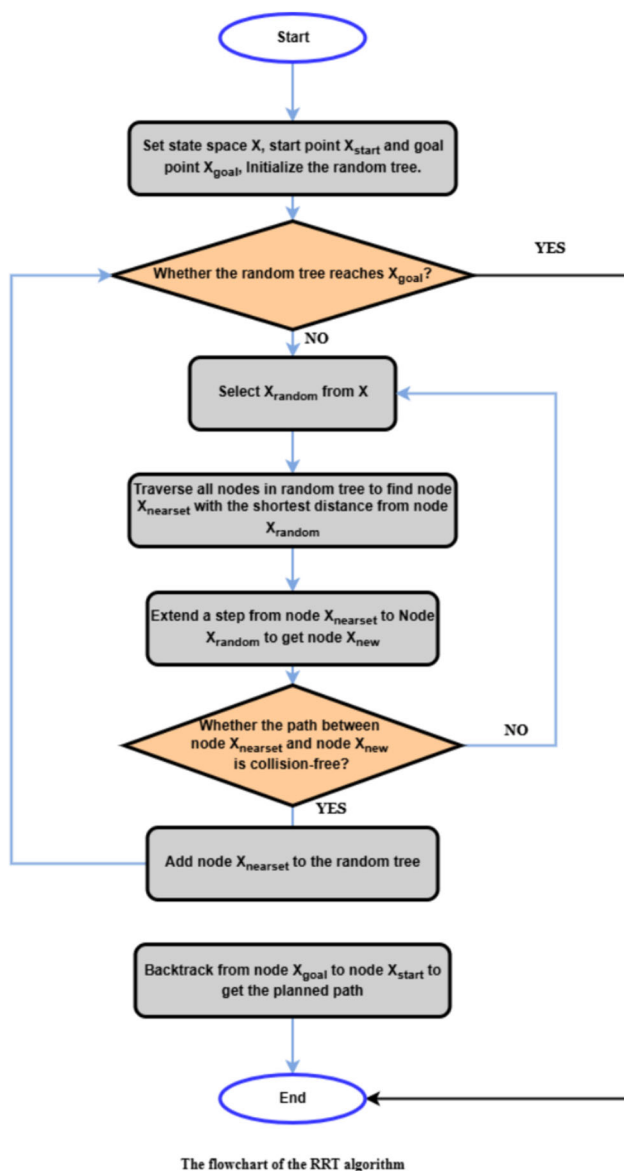


Fig. 7 The flowchart of the RRT algorithm

starting point X_{start} , and the goal point X_{goal} , as well as initializing the tree at X_{start} . It samples a random point X_{random} from X , iteratively identifies the nearest node in the tree, $X_{nearest}$, and takes a step from node $X_{nearest}$ towards point X_{random} , thus creating new node, X_{new} . Prior to adding any new nodes to the tree, the RRT process checks whether the path is collision-free from node $X_{nearest}$ to X_{new} , and, if collision free, the node is added to the tree. This process repeats until the random tree reaches the goal point X_{goal} . Finally, the algorithm backtracks from X_{goal} to X_{start} to construct the planned path. This approach effectively explores the state space to find a feasible path between the start and goal points while avoiding obstacles. Karaman and Frazzoli (2011) the RRT* algorithm was introduced to address this limitation by incorporating the cost of neighboring states after each successful expansion. It employs the Choose-Parent and Rewire strategies to adjust the connections between new states and their neighbors. The RRT* algorithm is asymptotically optimal, meaning it can eventually find the optimal solution as the number of samples increases indefinitely. However, this comes at the cost of significant computational time. In essence, RRT* sacrifices computational efficiency for improved performance. The main factor behind its inefficiency is the need to traverse the entire search tree each time a new state is added. As the number of states grows, the time required to perform this traversal becomes increasingly costly. Building upon this foundation, Nasir et al. (2013). Proposed the RRT*-smart algorithm, which leverages the idea of triangle inequality to optimize the path after finding a solution and introduces bias sampling toward high-quality nodes. As its name suggests, this algorithm selects sampling nodes more purposefully but also faces the same challenges as the informed-RRT*. To further address these challenges, researchers have proposed various enhancements. For instance, the RRT* algorithm introduces asymptotic optimality by rewiring the search tree to optimize paths, albeit at the cost of increased sample requirements (Karaman and Frazzoli 2011). Further advancements, such as RRT*-connect, utilize bidirectional search strategies to accelerate the pathfinding process (Fan et al. 2024). Moreover, recent studies have focused on improving sampling strategies to enhance the efficiency of RRT algorithms. Liu and Liu (2022) introduced a dynamic probability-based sampling method that mitigates local minima issues, resulting in smoother and shorter paths. Similarly, Wang and Zheng (2024) employed a generalized voronoi diagram to guide sampling, significantly improving convergence speed and path quality. Another variant, the informed RRT*, optimizes sampling by concentrating on promising regions of the state space, thereby reducing computational overhead. The Table 5 represents other studies that have utilized the RRT method.

Table 5 Other studies using RRT techniques

Study Details

Study: (Dong et al. 2025)

Year

2025

Basic Algorithm

RRT

Flags

Optimization: Yes, Fusion: No, Simulation: Yes, Real: No

Sensors and Material

Robot: A general "mobile robot"; the work is simulation-based.

Sensors: The algorithm assumes that environmental data has been collected from sensors such as LiDAR, cameras, and ultrasonic sensors.

Test Hardware: Simulations were performed on a computer with a Core i7-12800X CPU and 16 GB of RAM.

Test Tools

AI Models: The paper proposes the Faster High-Quality RRT* (FHQ-RRT*) algorithm. It is compared against RRT*, F-RRT*, and MQ-RRT*.

Metrics: The algorithms were evaluated on time cost, path cost, total nodes, and the cost/frequency of specific internal processes like node creation and rewiring.

Performance

Path Cost: FHQ-RRT* consistently achieved the lowest path cost across all four test maps when compared to the other algorithms.

Time Cost: FHQ-RRT* was significantly faster, reducing planning time by over 40% to 77% depending on the map's complexity.

Study: (Ran et al. 2024)

Year

2024

Basic Algorithm

RRT

Flags

Optimization: Yes, Fusion: No, Simulation: Yes, Real: No

Sensors and Material

Robot: A general "mobile robot" that is subject to kinematic constraints.

Test Hardware: The simulations were run on a PC with a 3.00 GHz Intel Core i5 and 8 GB of RAM.

Test Tools

AI Models: The paper proposes an Improved RRT* algorithm that is compared with Kinodynamic RRT* (Kino-RRT*) and RRT* with B-spline (RB).

Metrics: The algorithms were evaluated on time cost, number of path nodes, path length, and path smoothness (average curvature change rate).

Performance

Path Quality: The proposed algorithm produced the smoothest paths (lowest average curvature) compared to the other methods, making them more suitable for a real robot to follow.

Path Length: The node reconnection strategy proved effective, shortening the initial path length by 3% to 21% across the different test maps.

Study: (Wang et al. 2022)

Year

2022

Basic Algorithm

RRT

Flags

Optimization: Yes, Fusion: No, Simulation: Yes, Real: No

Sensors and Material

Robot: The focus is on articulated vehicles used in underground mining, such as large Load-Haul-Dump (LHD) vehicles and trucks. A Scooptram ST3.5 diesel LHD was used for simulation parameters.

Sensors: The system uses a suite of sensors including laser lidar, inertial measurement unit (IMU), cameras, and RFID.

Test Hardware: Simulations were run on a computer with an Intel Core i7-8550U CPU and 16 GB of memory.

Table 5 continued

Study Details	
Test Tools	Software: The algorithm was developed in Python 3.7. It uses a vectorized map to represent the narrow and complex underground drifts, which is more detailed and efficient than common rasterized maps.
AI Models: The paper proposes an Improved RRT* algorithm and compares it against the classic RRT and classic RRT* algorithms.	
Metrics: Average path length, search time, number of search nodes, number of path nodes, and the effective ratio of the steering angle.	
Performance	- The improved algorithm achieved a steering angle effectiveness of 100%, meaning the path fully respected the vehicle's physical limits, unlike the classic RRT (81.87%) and RRT* (92.71%).
- It significantly reduced the number of nodes in the final path (average of 16.2) compared to classic RRT (32) and classic RRT* (28.8), simplifying vehicle control.	
Study: (Abdouni et al. 2022)	
Year	2022
Basic Algorithm	RRT
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Robot: A general "Autonomous mobile robot"; the work is entirely simulation-based.
Test Hardware: A PC with an Intel Core i3-7100 CPU and 8GB of RAM.	
Test Tools	Software: The code was written in Python and tested on three different 1000x1000 pixel maps.
Metrics: The algorithms were compared based on their average run time, path length, and the number of nodes generated in the search tree.	
Performance	A-RRT (vs. RRT): The proposed sampling strategy significantly improved the RRT algorithm's performance.
- Convergence time was reduced by 32.92%.	
- Path length was shortened by 8.77%.	
- The number of nodes in the search tree was reduced by 86.51%.	

3.2 The probabilistic roadmap method (PRM)

The probabilistic roadmap method (PRM) algorithm was proposed in the study (Chen et al. 2021) to address specific challenges other methods face, such as latency issues and handling environments with high or low dimensions. PRM uses lines to define the connectivity of the robot's free spaces. In this context, we discuss visibility and voronoi graphs (Shahabi and Sharifzadeh 2018). The Fig. 8 shows a comparison between the two graphs: the visibility graph and the voronoi graph.

The visibility graph connects points (e.g., the start, goal, and polygon vertices) with edges where a straight line is unobstructed by obstacles. It is designed to find the shortest path through direct connections between visible points. In contrast, the voronoi graph partitions the space into regions

based on the closest distance to obstacles, creating a roadmap that maximizes clearance. The visibility graph prioritizes efficiency with direct paths, while the Voronoi graph focuses on safety by keeping paths equidistant from boundaries. The flowchart of the PRM algorithm is shown in Fig. 9.

The algorithm establishes a node set X and a path set F and generates uniform random sampling points n_i . Each sampling point n_i is evaluated to check whether it is in an obstruction or free space. Points within obstructions are discarded, while valid points are added to the node set X . Connections are attempted between the start point x_{init} , the goal point x_{goal} , and nodes in X , provided the paths between points do not intersect obstacles. Valid paths are added to the path set F . Next, an undirected graph $G(X, F)$ is established using the node and path sets. The graph is analyzed to determine whether it contains a valid path between x_{init}

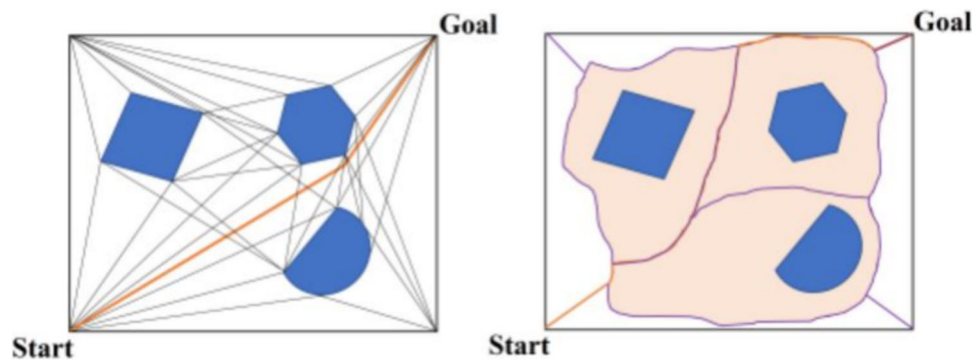


Fig. 8 Visibility graph(left) - Voronoi graph (right)

and x-goal. If no such path exists, the algorithm concludes with "No path." Otherwise, an optimal path is extracted from the graph, ensuring collision-free navigation from the start to the goal. Many studies have explored various strategies to enhance path planning methods for mobile robots. One such approach is the improved PRM (Ma et al. 2022), which utilizes cubic spline interpolation. This method introduces a novel bidirectional search PRM algorithm that incorporates cubic spline techniques, significantly improving both

the efficiency and smoothness of path planning compared to traditional PRM methods. The bidirectional PRM algorithm notably reduces the average search time compared to the standard PRM. Furthermore, as map complexity increases, the efficiency improvements of the bidirectional PRM become even more pronounced. Another study (Qiao et al. 2022) presents a novel optimized PRM algorithm that enhances path planning efficiency by increasing the number of free sampling points in narrow channels, reducing query time,

The flowchart of the PRM algorithm

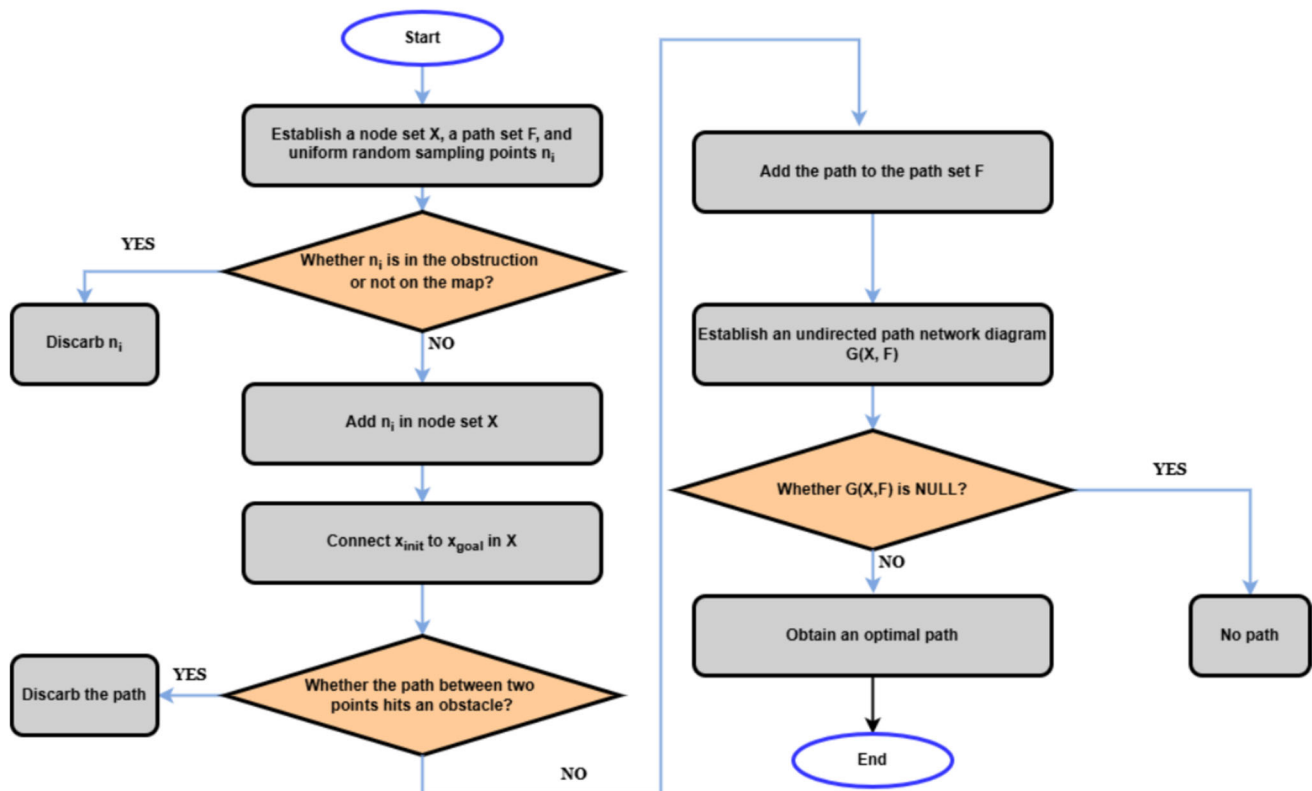


Fig. 9 The flowchart of the PRM algorithm

and improving the success rate of finding collision-free paths compared to traditional PRM methods. The study in Ravankar et al. (2020) proposes a new way of distributing nodes in potential fields. Setting specific coefficients for high and low potential regions allows for a successful reactive plan in dynamic environments, and also helps to eliminate the local minimum problem in one of the methods discussed in the artificial potential field. The study in Ye et al. (2022) proposes a novel path planning algorithm that allows planners to create meaningful sampling points that allow a planner to better navigate tight corridors. This method has shown to perform better in environments with complicated obstacles than the conventional methods. The study in Ouach and Eren (2022) gives a thorough review of path planning methodologies employed by mobile robots. Because of the ability of the probabilistic roadmap method (PRM) to allow for larger node densities and more optimal connection distances, the PRM has been shown in some cases to achieve negotiation on shorter paths and with more smoothly navigated paths than other forms of sampling based mobile robot path planning algorithm. The research presented has advanced this area of research by providing path smoothing methodologies as well as a detailed discussion on the implications of computational efficiency vs. the quality of the performed path. The Table 6 represents other studies that have utilized the PRM method.

3.3 The dynamic window approach (DWA)

The dynamic window approach (DWA), developed by Fox et al. (1997), utilizes a trajectory planning algorithm that utilizes curvature and speed to progress through a dynamic environment. The way that it functions, is that it simulates trajectories based on linear and angular velocities for a short period of time. Each trajectory is evaluated by a set of scoring rules based on three criteria of performance for obstacle avoidance, position relative to goal, and keeping an ongoing motion. DWA changes the obstacle avoidance problem into a speed optimization problem, meaning that the robot can navigate smoothly and safely, and provide a valid answer while maintaining the constraints of the robot in consideration. With an emphasis on efficiency, DWA performs well in an environment with complexity, reactivity and accuracy demands. The flowchart of the DWA algorithm is shown in Fig. 10.

The flowchart represents the dynamic window approach (DWA) procedural flow for real-time path planning for a mobile robot. The flow starts with input parameters, for example, the current state of the robot, the goal, and the speed sampling to create velocity and hence velocity options. From these velocities, the process creates possible trajectories for the robot to go through. Each trajectory is then evaluated based on the following criteria: obstacles avoidance, progress toward the goal, and smoothness of motion.

If there is an obstacle in the form of a "thunderstorm" that is in the way of the optimal trajectory, it recalculates the trajectory. Otherwise it simply takes the optimal trajectory and goes through the motion in a sequential process until it reaches the goal, allowing the robot to successfully navigate its environment safely. However, DWA has several limitations. It struggles with dynamic and complex environments, particularly large-scale or unpredictable obstacles. It does not prioritize energy efficiency, making it less suitable for long-term operations. Additionally, DWA faces challenges in multi-robot coordination, handling robot formations, and adapting to multi-objective scenarios, such as simultaneously optimizing safety and energy consumption. Many studies have explored various strategies for optimizing path planning and obstacle avoidance for mobile robots to address those challenges. The initial approaches, such as Yang and Teng (2022), focus on reducing cumulative and average turning angles, thereby improving path smoothness. The hybrid algorithm demonstrated effectiveness and laid the groundwork for integrating machine vision into navigation for complex environments. Building on these advancements, another study (Hou et al. 2024) introduces a novel approach that combines an enhanced sparrow search algorithm (MISSA) with the dynamic window approach (DWA). This integration achieved superior path optimization and stability in static and dynamic environments while identifying potential future directions, such as multi-objective and multi-robot planning. Also, the study (Cao and Mohamad Nor 2024) introduced a new approach for making judgments about robot formations that is capable of avoiding dynamic and aggressive obstacles in an unknown environment and it does so efficiently, safely and robustly. The method has improved the

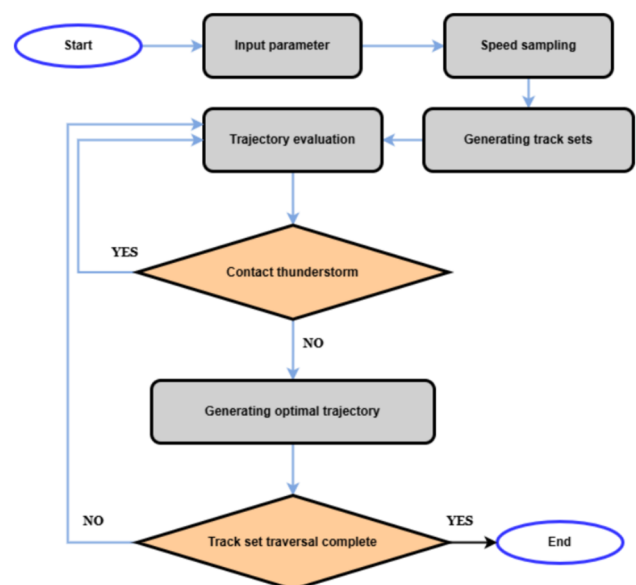


Fig. 10 The flowchart of the DWA algorithm

Table 6 Other studies using PRM techniques

Study Details

Study: (Chen et al. 2021)

Year

2021

Basic Algorithm

PRM

Flags

Optimization: Yes, Fusion: No, Simulation: Yes, Real: Yes

Sensors and Material

Robot: The research applies to high-degree-of-freedom (DOF) manipulators. Specifically, simulations were conducted for a 6-DOF manipulator and an 8-DOF modular manipulator.

Sensors: The methodology relies on a point-cloud model for collision detection, implying the use of 3D scanning sensors in a physical application.

Test Tools

Software: Simulations were developed in Matlab.

AI Models: The paper introduces an improved PRM method.

Datasets: Custom-simulated environments were used, including a 2D configuration space with narrow passages and a 2D workspace with obstacles for the 6-DOF manipulator.

Performance

The proposed method can construct a high-quality roadmap with significantly fewer nodes than traditional and Gaussian PRM methods. It achieves higher sampling density in narrow passages and reduces redundant samples in open areas. Consequently, it finds paths more efficiently with shorter search times and lower path costs.

Study: (Esposito and Wright 2019)

Year

2019

Basic Algorithm

PRM

Flags

Optimization: Yes, Fusion: No, Simulation: Yes, Real: No

Sensors and Material

—

Test Tools

Software: The experiments utilized problems from the Open Motion Planning Library (OMPL).

AI Models: The paper introduces a novel post-processing algorithm for Probabilistic Roadmaps (PRMs) inspired by matrix completion. The core idea is to decompose the roadmap's adjacency matrix into a sum of low-rank and sparse matrices to estimate the status of all possible edges.

Metrics: Accuracy, Average path length.

Performance

The proposed algorithm demonstrates high efficiency and utility. After checking just 5% of the candidate edges in a roadmap, the algorithm can predict the status of all possible edges with 99% accuracy, without performing any additional collision checks. The primary practical benefit is that the resulting denser roadmap allows for finding significantly shorter average path lengths. This performance improvement comes at the cost of a slight increase in spatial complexity and query times.

Study: (Ravankar et al. 2020)

Year

2020

Basic Algorithm

PRM + APF

Flags

Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: Yes

Sensors and Material

Robot: The study is centered on mobile robots. The real-world experiments were conducted using a Pioneer 3-DX differential drive robot.

Sensors: The experimental robot was equipped with a 2D SICK LMS111-LRF (Laser Range Sensor) and an Asus Xtion Pro Live 3D vision sensor.

Test Tools

Software: The implementation was done using MATLAB and the Robot Operating System (ROS) on an Ubuntu Linux operating system.

Table 6 continued

Study Details	
AI Models: The core of this work is the HPPRM (Hybrid Potential based Probabilistic Roadmap) algorithm.	
Metrics: Success Rate, Mean Path Length, Mean Execution Time.	
Performance	The proposed HPPRM algorithm achieved a success rate of over 95% in both local and global planning tests. It significantly outperforms the original PRM in narrow passages, generating safer and shorter paths due to its potential-field-based node sampling. The virtual force method for local planning effectively prevents the robot from getting stuck in local minima. In dynamic environments, HPPRM also showed a higher success rate and better performance than TAPF, PRM, and A*.
Study: (Qin et al. 2022)	
Year	2019
Basic Algorithm	PRM
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Robot: The study focuses on a general "reconfigurable robot" with "leg motions". The validation is done through a simulation of a robot in 3D space.
Test Tools	AI Models: The paper proposes a new algorithm named MM-PRM (Multimodal-Probabilistic Roadmap). This model's performance is compared against the traditional PRM (Probabilistic Roadmap) algorithm.
Datasets: A simulated 3D environment was used for testing. This environment includes a cube with four surfaces and two rectangular barriers that create a narrow channel.	
Performance	The simulation results demonstrated that the proposed MM-PRM algorithm has a shorter runtime compared to the conventional PRM algorithm across different modes.

smoothness of motion trajectories and decreased travel or run time or distance travelled but there are still outstanding problems particularly in relation the cost to resolve formation with multiple robots and more sophisticated dynamic and large scale obstacles. Lastly, the paper (Henkel et al. 2016) described the EDWA, which looked at energy-efficient and included a cost function based on energy and an energy prediction model based on energy using linear regression. The EDWA achieved a reduction of 9.79% in energy consumption over the dynamic window approach, thus supporting its more sustainable use in navigation. The Table 7 represents other studies that have utilized the DWA method.

4 Gradient based

After exploring sampling-based methods, we now turn to gradient-based approaches. The fundamental principle of these techniques is to define a scalar cost function, denoted as $U(\mathbf{q})$, over the robot's configuration space, where $\mathbf{q}$ represents the robot's position. This function creates a virtual "landscape" where "valleys" lead to the target and "hills" represent obstacles.

The robot's motion is then directly guided by following the direction of steepest descent, which is the negative gradient of this function. The control force F_{control} applied to the robot can be generally expressed as:

$$F_{\text{control}} = -\nabla U(\mathbf{q})$$

where $\nabla U(\mathbf{q})$ is the gradient of the cost function at the configuration $\mathbf{q}$. The main challenge in these methods lies in defining a function $U(\mathbf{q})$ that is both effective and free of undesirable local minima.

One of the most common and intuitive implementations of this approach is the Artificial Potential Field (APF) method, which explicitly defines the composition of the cost function $U(\mathbf{q})$.

4.1 Artificial potential field (APF)

The APF method, proposed by Khatib (1985), specifies the general cost function $U(\mathbf{q})$ by decomposing it into the sum of an attractive potential and a repulsive potential:

$$U(\mathbf{q}) = U_{\text{att}}(\mathbf{q}) + U_{\text{rep}}(\mathbf{q})$$

Table 7 Other studies using DWA techniques

Study Details

Study: (Zhenyang and Wei 2022)

Year

2022

Basic Algorithm

A* + DWA

Flags

Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: Yes

Sensors and Material

Material: A generic mobile robot.

Environment: A 22m x 22m grid map in a ROS simulation.

Test Tools

Framework: Robot Operating System (ROS) with Rviz for visualization, and MATLAB.

Algorithm: A fusion of an Improved A* algorithm (with safety distance checks and path smoothing) and an Adaptive DWA.

Metrics: Path length, computation time, and number of iterations.

Performance

- The Improved A* algorithm produced shorter and smoother paths than the preliminary version.

- The Adaptive DWA was more computationally efficient (35.4s vs. 55.8s) and safer than the traditional DWA.

- The final fused algorithm was successfully verified in ROS simulations for dynamic obstacle avoidance.

Study: (Hong et al. 2015)

Year

2016

Basic Algorithm

FL + DWA

Flags

Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: Yes

Sensors and Material

Material: A mobile robot.

Environment: A complex and changeable environment.

Test Tools

Algorithm: A Modified Dynamic Window Approach (DWA) that uses Fuzzy Logic to adjust the weights of its objective function in real time.

Metrics: Safety and smoothness of the robot's movement.

Performance

- The proposed method makes the robot move more safely and smoothly compared to the original DWA.

- It successfully uses fuzzy logic to adapt the DWA algorithm's parameters to different situations in real time, overcoming the limitation of the original DWA's constant weights.

Study: (Wu et al. 2022)

Year

2022

Basic Algorithm

A* + DWA

Flags

Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: No

Sensors and Material

Material: Forklift AGV (FAGV).

Sensors: Laser scanner, binocular cameras, and motion sensors.

Environment: Simulated storage warehouse environment.

Test Tools

Framework : Unspecified simulation software.

Algorithm: A hybrid of an Improved A* algorithm for global planning and an Improved Dynamic Window Approach (DWA) for local obstacle avoidance.

Metrics: Path smoothness (number of turns) and successful dynamic obstacle avoidance.

Table 7 continued

Study Details

Performance	- The Improved A* algorithm reduced the number of paths turning points by 62.5% compared to the traditional A* method.
- The Improved DWA resulted in safer local paths that stayed closer to the global route and reduced the FAGV's speed near obstacles.	
Study: (Li et al. 2022)	
Year	2022
Basic Algorithm	A* + DWA
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: Yes
Sensors and Material	Material: WPV4 all-terrain robot.
Sensors: Laser Radar (SICK LMS111), GPS, and IMU.	
Environment: A real-world dense planting red jujube orchard.	
Test Tools	Framework: Robot Operating System (ROS), using GMapping for SLAM and AMCL for localization.
Algorithm: A hybrid algorithm joining an Improved A* and an Improved DWA.	
Metrics: Navigation accuracy (average deviation in cm) and navigation stability (standard deviation).	
Performance	- The system achieved high navigation accuracy, with an average deviation of 2.68 cm in straight-line tests.
- In navigating specific L-shaped and U-shaped paths, the improved algorithm reduced the average positioning error by 0.91 cm and 0.54 cm, respectively, compared to a traditional fusion algorithm.	

These two components are defined as follows:

– Attractive Potential (U_{att}): This function generates a "force" that pulls the robot toward its goal destination q_{goal} . It is often modeled as a parabolic field whose minimum is at the target:

$$U_{att}(q) = \frac{1}{2} \zeta \rho(q, q_{goal})^2$$

Here, $\rho(q, q_{goal})$ is the Euclidean distance between the robot and the target, and ζ is a positive gain coefficient.

– Repulsive Potential (U_{rep}): This function generates "forces" that push the robot away from obstacles. The potential is significant near an obstacle and fades to zero at a certain safety distance ρ_0 . For a given obstacle, it can be modeled as:

$$U_{rep}(q) = \begin{cases} \frac{1}{2} \eta \left(\frac{1}{\rho(q, q_{obs})} - \frac{1}{\rho_0} \right)^2 & \text{if } \rho(q, q_{obs}) \leq \rho_0 \\ 0 & \text{if } \rho(q, q_{obs}) > \rho_0 \end{cases}$$

Here, $\rho(q, q_{obs})$ is the distance to the nearest obstacle, η is a gain coefficient, and ρ_0 is the distance of influence of

the obstacle. The total repulsive potential is the sum of the potentials from all obstacles.

By computing the gradient of $U(q)$, the attractive and repulsive forces are obtained, and their sum dictates the robot's movement.

The APF-based process is illustrated in the Fig. 11 as follows:

The problem of local minima in the potential field (PF) method is encountered when the sum of the attractive forces (forces pulling the agent toward the intended goal location) and repulsive forces (forces that help the agent avoid obstacles) create areas which have zero gradient but are not actually at the goal. The local minima is problematic because the agent can become 'stuck' and cannot reach its destination. Local minima can happen due to overlapping forces, obstacles poorly positioned with respect to the agent or due to the simplicity of the function used to model the fields. Many researchers have explored these limitations and provided solutions. For instance, Luo et al. (2022) proposed the improved artificial potential field (IAPF) algorithm, which successfully addressed local minimum problems in mobile robot path planning by integrating the A* method. IAPF

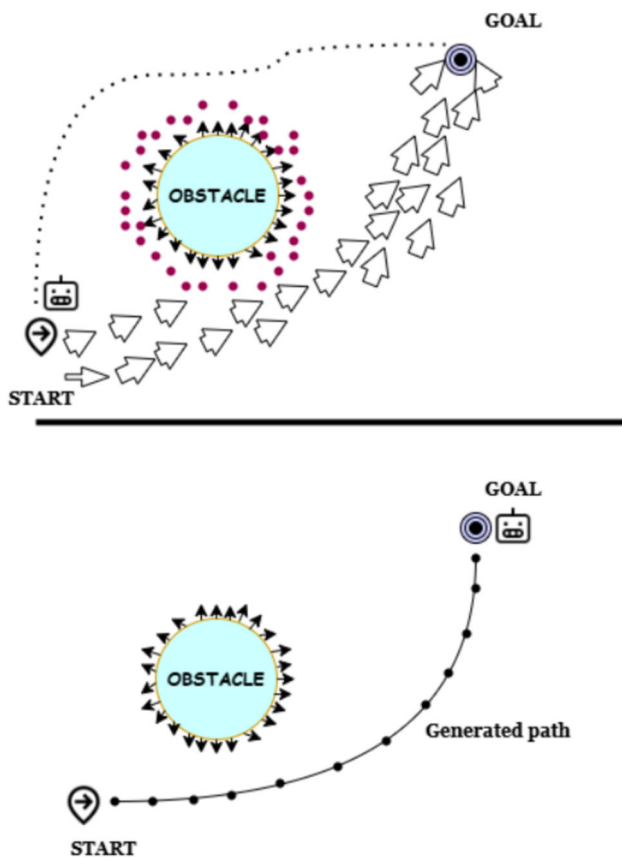


Fig. 11 APF-based navigation for a mobile robot

demonstrated its utility and efficacy through simulation experiments. A different approach was also implemented (Lazarowska 2019) using the discrete artificial potential field (DAPF) algorithm. DAPF proposed a new path planning alternative for autonomous mobile robots. The method can navigate in static and dynamic environments in near-real-time, and uses the path optimization algorithm (POA) to improve on smoothness and lengths of the original paths proposed. DAPF has demonstrated its feasibility and efficacy in both simulation and real-world implementations. The research (Arambula Cosío and Padilla Castañeda 2004) also introduced a novel autonomous robot navigation algorithm using moving auxiliary attraction points and a genetic algorithm used to modify artificial potential fields. This is a mechanism to get better global and local obstacle avoidance without changing normal building environments.

Other studies have also contributed innovative solutions to this issue. For example:

- Sfeir et al. (2011) introduces a new repelling potential to reduce oscillations near obstacles.
- Melchiorre et al. (2023) enhances artificial potential fields with local attractors, improving robot navigation.

The Table 8 represents other studies that have utilized the APF method.

5 Bio-inspired algorithm

While potential fields offer an elegant solution for obstacle avoidance, they can struggle in complex scenarios; we therefore examine bio-inspired algorithms next, which leverage evolutionary mechanisms to optimize trajectories.

5.1 Genetic algorithm

Genetic algorithms (GAs) are broadly classified techniques for optimization that are inspired by natural genetics and the ideas of natural selection. They were originally introduced by Bremermann in 1958 (Bremermann 1958) and formally proposed by Holland and co-authors in 1975 (Holland 1992). The GA paradigm is based upon Darwin's theory of evolution. The basic idea of the GA process is to simulate biological processes of selection, crossover, and mutation to solve complex problems. The basic GAs typically operate on a population of candidate solutions represented as a chromosome in binary string format. GAs rely on the "survival of the fittest" process that governs nature in which individuals with higher fitness values have a greater chance of survival and passing on their genetic material to the next generation. The bio-inspired operators of selection, crossover and mutation in a GA create the potential for the population to evolve through generations towards potentially optimal solutions. GAs dynamically explore and modify the chromosomal structures within the population to maintain diversity and avoid premature convergence. This combination of randomization and structured information makes GAs powerful search algorithms capable of identifying feasible and efficient trajectories for various optimization and machine learning challenges. The process terminates once the population converges or predefined optimization criteria are met, yielding a well-adapted solution to the problem (Chentoufi et al. 2017; Fatmi et al. 2017). The GA process is illustrated in the Fig. 12 as follows:

Several researchers have attempted to improve genetic algorithms by reducing the number of iterations required for convergence. For example, the study (Wang 2023) discusses enhancements to genetic algorithm path planning by incorporating K-nearest neighbor for efficiency. It reviews various improvements, such as hill-climbing and adjacency crossover, to optimize convergence speed. Another study (Liu et al. 2014) explores genetic algorithms with fixed and variable-length chromosomes for path optimization, emphasizing the importance of customized genetic operators for diverse applications. Similarly, Hu and Yang (2022) highlights the limitations of classical approaches with fixed-

Table 8 Other studies using APF techniques

Study Details	
Study: (Zheyi and Bing 2021)	
Year	2021
Basic Algorithm	APF + FL
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: No
Sensors and Material	Material: An Automated Guided Vehicle (AGV). Environment: A complex environment simulated in Matlab.
Test Tools	Framework: Matlab simulation. Algorithm: An Improved Artificial Potential Field algorithm that adds an angle function and integrates fuzzy control. Metrics: The accuracy and efficiency of the path planning task.
Performance	- The proposed algorithm can complete path planning tasks accurately and efficiently. - It demonstrates advantages over the traditional artificial potential field method, especially in complex environments.
Study: (Research on Automatic 2025)	
Year	2021
Basic Algorithm	APF
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Material: An autonomous driving vehicle. Environment: Highway driving scenarios.
Test Tools	Framework: Not specified. Algorithm: A hybrid method using an Improved Artificial Potential Field (APF) for path planning and Model Predictive Control (MPC) for path tracking.
Performance	- The proposed method improves upon the traditional Artificial Potential Field by considering the speed and direction of both the autonomous vehicle and dynamic obstacles. - It successfully combines path planning and path tracking to create a complete control method for autonomous overtaking.
Study: (Wang et al. 2021)	
Year	2021
Basic Algorithm	APF + NN
Flags	Optimization: No, Fusion: Yes, Simulation: Yes, Real: Yes
Sensors and Material	Material: A mobile robot. Environment: A dynamic environment, tested in both simulation and a physical experiment.
Test Tools	Framework: An unspecified simulation environment and a physical robot setup. Algorithm: A hybrid method using an extensible neural network that is trained with data generated by a fuzzy artificial potential field algorithm. Metrics: The method is evaluated based on its path optimization ability.
Performance	- The neural network achieved excellent path optimization ability due to the high-quality training data generated by the fuzzy artificial potential field method. - The algorithm successfully planned paths for a mobile robot in dynamic environments.
Study: (Shin and Kim 2021)	
Year	2021
Basic Algorithm	APF + A*
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: No
Sensors and Material	Material: Unmanned Vehicle (UV), specifically a Crazyflie 2.0 quadcopter for experiments. Sensors: An Ultra-Wide Band (UWB) positioning system with eight anchors. Environment: A simulated indoor area and a real indoor gymnasium.
Test Tools	Framework: A custom simulation and a real-world test with a quadcopter. Algorithm: A hybrid method combining the Artificial Potential Field (APF) with a novel Positioning Risk (PR) field (PR-APF). Metrics: The primary metric was the success rate of reaching the goal over 100 trials.
Performance	- The proposed PR-APF method achieved a success rate of over 90%, while the conventional APF method failed on more than 50% of the same routes. - The algorithm successfully guided the vehicle to avoid areas with poor positioning accuracy, resulting in safer navigation.

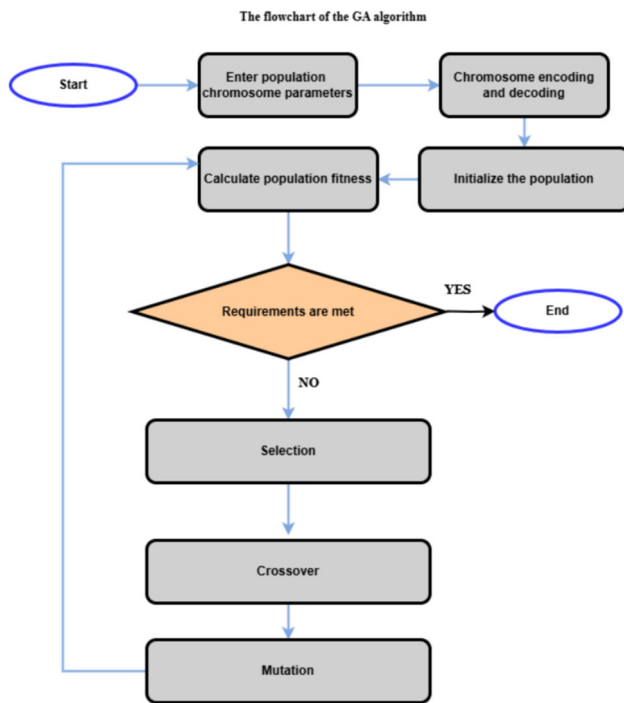


Fig. 12 Process of GA

length chromosomes and stresses integrating domain knowledge into genetic algorithms to improve efficiency in complex environments. In Liu et al. (2010), the authors present a novel and efficient method for path planning in dynamic environments. This approach enhances real-time calculation speed and precision through real-number coding and adaptive functions. Meanwhile, Ab Wahab et al. (2024) demonstrates that the proposed improved genetic algorithm, which uses a linear rank-based CBPRM approach, significantly boosts population initialization efficiency and path optimization performance, particularly in complex environments. Finally, the study (Lamini et al. 2018) proposes a method that significantly reduces the number of turns in optimal paths across various environments, further enhancing the effectiveness of genetic algorithms for robot navigation. The Table 9 represents other studies that have utilized the GA method.

5.2 Fuzzy logic

Fuzzy logic, introduced by Lotfi A. Zadeh in 1965 (Zadeh 1965), revolutionized traditional binary decision-making systems by enabling reasoning with uncertainty and partial truths. Unlike classical logic, which operates on absolute "true" or "false" values, FL works within the spectrum of possibilities between 0 and 1. This nuanced approach allows FL to address real-world problems where sharp boundaries

are impractical. The foundation of FL lies in fuzzy sets, defined as $A = \{x, \mu_A(x) \mid x \in X, \mu_A(x) : X \rightarrow [0, 1]\}$, where $\mu_A(x)$ is the membership function that quantifies the degree of membership of x within the set. Classical set operations such as intersection, union, and complement extend naturally to fuzzy sets while retaining fundamental logical properties. FL's strength lies in its ability to model expert intuition and mimic human reasoning processes. It is widely applied in robotics, guiding autonomous systems through complex and unstructured environments. In path planning, FL dynamically adjusts a robot's speed and trajectory to navigate around obstacles, ensuring efficient goal completion without the rigidity of traditional methods. The FL process is illustrated in the Fig. 13 as follows:

However, FL is not without its challenges. Its reliance on pre-defined fuzzy inference rules, often based on expert knowledge, limits its adaptability in rapidly changing environments. Additionally, as the complexity of input data increases, the size of fuzzy rule sets can grow significantly, impacting real-time performance. To address these limitations, several researchers have proposed solutions. The study (Sun et al. 2023) presents a novel integration of fuzzy control with the dynamic window technique. This approach enhances the algorithm's capability for safe and reliable dynamic obstacle avoidance by dynamically adjusting weight coefficients, resulting in improved path planning and smoother motion trajectories for mobile robots. Another study (Tuan et al. 2024) highlights using lidar sensors and fuzzy controllers for effective path planning and obstacle avoidance. Similarly, Faisal et al. (2013) proposes a wireless fuzzy

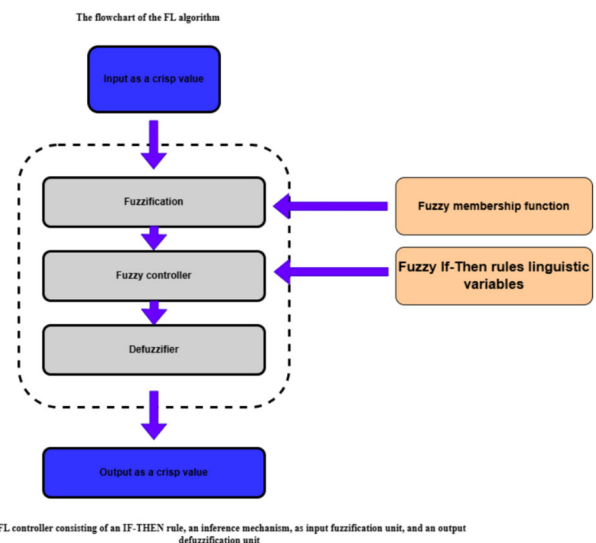


Fig. 13 The FL process

Table 9 Other studies using GA techniques

Study Details	
Study: (Mahmud et al. 2019)	
Year	2019
Basic Algorithm	GA
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Material: Agricultural mobile robot prototype. Sensors: Not specified (planning in a virtual environment). Environment: A virtual replica of a real-world greenhouse.
Test Tools	Framework: Unspecified virtual environment software. Algorithm: Non-dominated Sorting Genetic Algorithm using Reference Point Based (NSGA-III), compared against NSGA-II. Metrics: Solution quality (C-metric indicator) and execution time.
Performance	- The proposed NSGA-III offered superior solution quality compared to the NSGA-II baseline. - In complex scenarios, all solutions found by NSGA-II were dominated by solutions from NSGA-III.
Study: (Shivgan and Dong 2020)	
Year	2020
Basic Algorithm	GA
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Material: Unmanned Aerial Vehicle (UAV) / drone. Sensors: Not specified (path planning on known waypoints). Environment: Simulated area with a set of waypoints.
Test Tools	Framework: MATLAB. Algorithm: Genetic Algorithm (GA), compared against a Greedy Algorithm. Metrics: Total energy consumption (kJ), distance traveled (m), and computation time (s).
Performance	- The proposed GA consumes 2-5 times less energy than the Greedy algorithm. - The energy savings become more significant as the number of waypoints increases.
Study: (Tao et al. 2022)	
Year	2022
Basic Algorithm	GA
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Material: Wall surface inspection robot. Sensors: GPS, Inertial Measurement Unit (IMU). Environment: Simulated map of a bridge surface with GPS-denied zones.
Test Tools	Framework: MATLAB 2020b. Algorithm: Improved Genetic Algorithm, initialized by a Particle Swarm Optimization (PSO) algorithm. Metrics: Convergence speed, path quality (length and angle change).
Performance	- The algorithm successfully planned paths that avoided GPS-denied zones. - It outperformed traditional GA and PSO in convergence rate and path quality. - Achieved an average convergence of 35.9 iterations in a simple environment.
Study: (Zhang et al. 2025)	
Year	2025
Basic Algorithm	GA
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: Yes
Sensors and Material	Material: A wheeled mobile robot. Sensors: Laser and ultrasonic radar. Environment: MATLAB simulation and a corresponding real-world experimental setup.
Test Tools	Framework: MATLAB, ROS, and Gazebo. Algorithm: An Improved Genetic Algorithm (GA) with multiple enhancements in initialization, selection, crossover, and mutation. Metrics: Path length, path smoothness (turning points), and convergence speed (iterations).
Performance	- The proposed algorithm outperformed traditional GA and A*, reducing the final path length by up to 6.5%. - It generated significantly smoother paths with far fewer turning points (e.g., 3 turns vs. 14 for traditional GA).

logic control system for wheeled mobile robots in warehouse environments. As demonstrated, most researchers explore various methods to enhance fuzzy logic-based systems. For instance, the study (Ahmed et al. 2015) combines fuzzy logic with the potential field method, while others, such as Wondosen and Shiferaw (2024); Waga et al. (2021), focus exclusively on systems based solely on fuzzy logic. The Table 10 represents other studies that have utilized the FL method.

5.3 Particle swarm optimization

Particle swarm optimization (PSO) (Kennedy and Eberhart 1995) represents a dynamic approach to problem-solving inspired by the cooperative behaviors observed in nature, such as bird flocks navigating to optimal feeding grounds or fish schools avoiding predators while seeking resources. In this algorithm, every "particle" represents an individual candidate solution in a multi-dimensional search space represented by properties such as position, velocity, and fitness value. Particles navigate through the search space iteratively, where each particle is moved according to its history of successful positions and also the history of successful positions of the swarm as a whole. The particle uses both historical positions and velocity to find a balance between exploratory and exploitative processes. The balance between individual exploration and collective learning enables the algorithm to be adaptive and robust enough to explore complex optimization problems in a complex search space. At the heart of PSO, the particle uses inertia to maintain momentum while also taking into account the concept of adaptive learning. A particle's velocity is what determines how fast it will move and in what direction. In addition, the velocity incorporates elements of both its own history of successful adaptation as well as the history of adaptation of its neighbors. This is accomplished mathematically by incorporating stochastic components into the particle's behavior which allows for randomness in exploring the search space during the process while still not converging prematurely to local minima. The PSO process is illustrated in the Fig. 14 as follows:

The following flowchart represents the particle swarm optimization (PSO) algorithm that initializes parameters, such as inertia weight (w), cognitive coefficient (C_1), social coefficient (C_2), and initializes the particles positions, values, and velocities. The PSO algorithm measures the fitness function for each particle (fitness function evaluations iteratively increment), and subsequently identifies the particles Pbest and the Gbest solutions, and re-evaluates the particles' positions by resetting their velocity and particle locality until the stopping criteria has been met; yielding Gbest as the solu-

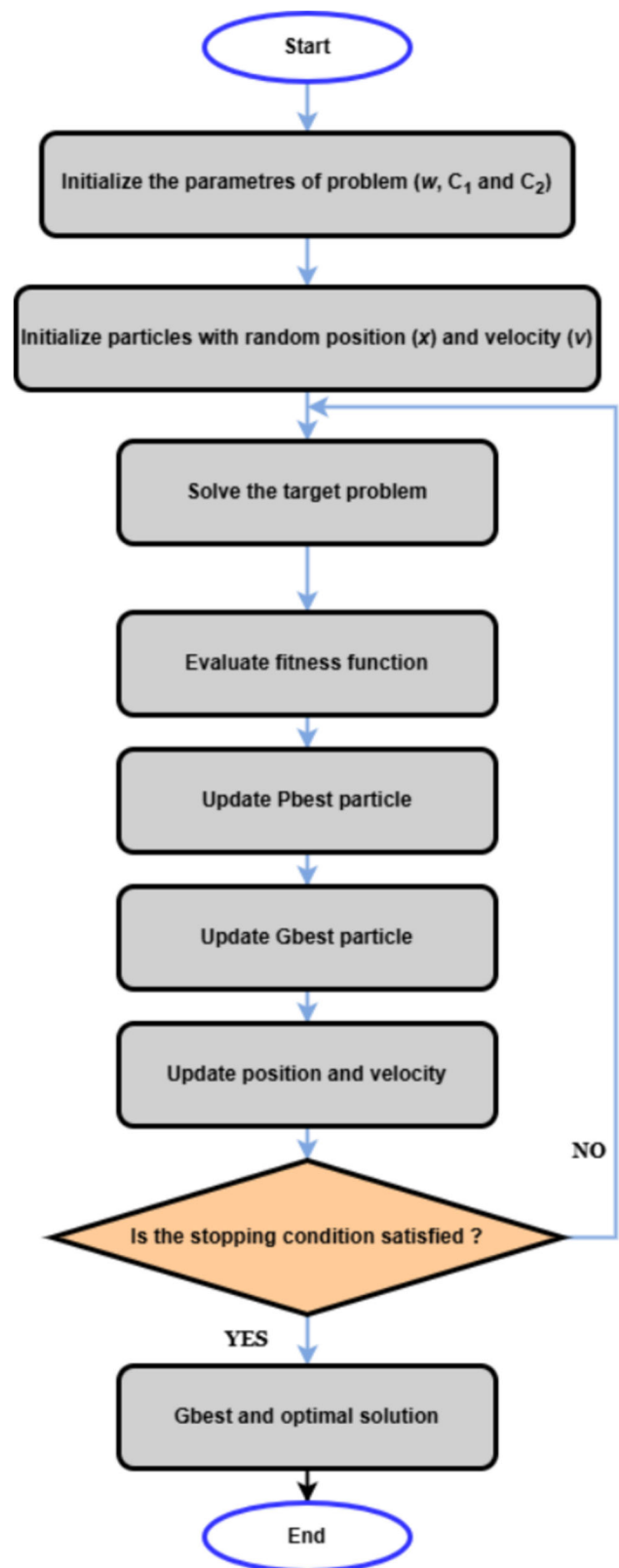


Fig. 14 The PSO process

Table 10 Other studies using FL techniques.

Study Details	
Study: (Feng et al. 2021)	
Year	2021
Basic Algorithm	APF + FL
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: No
Sensors and Material	Material: Autonomous vehicle. Environment: Simulated traffic scenes.
Test Tools	Algorithm: A hybrid method using an Improved Artificial Potential Field (APF) algorithm for path planning and Model Predictive Control (MPC) / Fuzzy-PID for path tracking. Metrics: Vehicle response curves, safety, comfort, and stability.
Performance	– The algorithm successfully generated a safe path while ensuring vehicle comfort and stability in simulated dynamic scenarios. – The method effectively combined path planning with a tracking controller to create a stable vehicle trajectory.
Study: (Samadi Gharajeh and Jond 2022)	
Year	2022
Basic Algorithm	FL + NN
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: No
Sensors and Material	Material: A wheeled mobile robot with simple hardware. Sensors: Three ultrasonic sensors (left, front, right) and a Global Positioning System (GPS) module. Environment: A V-REP simulation environment.
Test Tools	Framework: V-REP simulator integrated with MATLAB. Algorithm: An Adaptive Neuro-Fuzzy Inference System (ANFIS) combined with a utility function to determine the steering angle. Metrics: The algorithm's performance was evaluated by analyzing the discovered path and comparing the total navigation time against other methods.
Performance	– The proposed technique successfully navigated the robot through cluttered environments, finding near-optimal, collision-free paths. – The discovered path was approximately 30% shorter than the paths found by the other compared methods.
Study: (Liu and Li 2022)	
Year	2022
Basic Algorithm	MGWO
Flags	Optimization: No, Fusion: Yes, Simulation: Yes, Real: No
Sensors and Material	Material: A generic mobile robot. Sensors: 2D laser measurements and image data. Environment: Simulated grid maps with static and dynamic obstacles.
Test Tools	Framework: Unspecified simulation environment. Algorithm: A hybrid of Modified Gray Wolf Optimization (MGWO) for global planning and a Bayesian network for local situation assessment and obstacle avoidance. Metrics: Path length (shortest, average) and path length variance.
Performance	– The proposed MGWO algorithm found shorter paths than standard Gray Wolf Optimization (GWO) and Particle Swarm Optimization (PSO), reducing the average path length by up to 7.13%. – The hybrid method successfully demonstrated dynamic obstacle avoidance for both static and moving obstacles while following the globally planned path.
Study: (Sangeetha et al. 2021)	
Year	2021
Basic Algorithm	FL + ACO
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: No
Sensors and Material	Material: The study focuses on path planning for autonomous unmanned vehicles and mentions real-time navigation experiments with a Khepera III mobile robot. Sensors: The paper does not specify which sensors were used, as the method is tested in simulated occupancy grids where obstacle information is known. Environment: The experiments were conducted in MATLAB on simulated occupancy grids (e.g., 100×100) containing both static and dynamic obstacles.
Test Tools	Framework: Simulations were implemented in MATLAB 2019b. Algorithm: The paper proposes a Fuzzy Gain-Based Dynamic Ant Colony Optimization (FGDACO) algorithm. This method enhances the pheromone update mechanism of an ant colony system and uses fuzzy logic for collision avoidance. It is compared against the Cuckoo Optimization Algorithm (COA), a Fuzzy-Genetic Algorithm (Fuzzy-GA), and a Fuzzy Logic Ant Colony Optimization (FLACO). Metrics: The performance was evaluated based on the path length (in meters) and computation time (in seconds).
Performance	– The proposed FGDACO algorithm consistently found the shortest path in the least amount of time when compared to COA, Fuzzy-GA, and FLACO in all tested scenarios. – In one scenario, FGDACO outperformed the other algorithms in path length by 6% (vs. COA), 11% (vs. Fuzzy-GA), and 3% (vs. FLACO). – The algorithm demonstrated high stability and consistency, with a low standard deviation of only 2% on average across multiple independent runs.

Table 11 Other studies using PSO techniques

Study Details	
Study: (Chai et al. 2021)	
Year	2021
Basic Algorithm	PSO
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Material: The study applies its method to an atmospheric entry vehicle (spacecraft). Sensors: Not applicable. The optimization is based on a mathematical model of the vehicle's dynamics rather than real-time sensor inputs. Environment: A numerical simulation of Earth's atmospheric entry, including models for gravity and atmospheric density.
Test Tools	Framework: Numerical simulations were performed on a PC with an Intel Core i7-4790 CPU. Algorithm: A biased multi-objective particle swarm optimization (MOPSO) algorithm is proposed. It includes novel features like an E-bias selection strategy and an evolution restart strategy. This method was compared to a penalty function-based PSO (PFPSO), an artificial bee colony algorithm (PFABC), and the numerical solver CASADI.
Performance	- The proposed algorithm successfully found feasible trajectories that satisfied all complex mission constraints. - It achieved more optimal objective function values than the other heuristic algorithms (PFPSO, PFABC) and the CASADI solver in most of the tested scenarios.
Study: (Phung and Ha 2021)	
Year	2021
Basic Algorithm	PSO
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: Yes
Sensors and Material	Material: The study focuses on Unmanned Aerial Vehicles (UAVs). A 3DR Solo drone was used for real-world experimental verification. Sensors: The planning scenarios are based on real Digital Elevation Model (DEM) maps derived from LiDAR sensors. The drone uses GPS for navigation during the real-world flight tests. Environment: Eight benchmarking scenarios were created from real DEM maps of Christmas Island, Australia. Real-world experiments were conducted in a park in Sydney, Australia.
Test Tools	Framework: The primary evaluation is conducted in a simulation based on the DEM maps. Real-world flight paths are managed using Mission Planner ground control station software. Algorithm: A new algorithm, Spherical Vector-based Particle Swarm Optimization (SPSO), is proposed. It is compared against other PSO variants (classic PSO, O-PSO, QPSO) and other common metaheuristic algorithms like Genetic Algorithm (GA), Artificial Bee Colony (ABC), and Differential Evolution (DE).
Performance	- The proposed SPSO algorithm statistically outperformed other PSO variants and other state-of-the-art metaheuristic algorithms (GA, ABC, DE) in most of the eight complex scenarios tested. - SPSO was able to generate smooth, feasible, and safe paths while properly maintaining flight height relative to the terrain. - The SPSO approach also showed strong scalability, continuing to perform well when the path complexity was increased by doubling the number of waypoints.
Study: (Qiuyun et al. 2021)	
Year	2021
Basic Algorithm	PSO + GA
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: No
Sensors and Material	Material: The study focuses on an Automated Guided Vehicle (AGV) in a smart manufacturing context. Sensors: Not applicable. The path planning problem is solved in a known, structured environment (a one-line production workshop) where machine tool locations are predefined.
Test Tools	Framework: The algorithm was implemented in C++ and results were visualized using MATLAB. Algorithm: An Improved Particle Swarm Optimization (IPSO) algorithm is proposed, featuring a new coding method, a crossover operation to update particle positions, and a mutation mechanism. It was compared against standard PSO, Genetic Algorithm (GA), and Ant Colony Optimization (ACO). Metrics: The primary performance metric was the shortest transportation time. This was also evaluated using Relative Percentage Increase (RPI) and the stability of the results (standard deviation) over 25 runs.
Performance	- The proposed IPSO algorithm consistently found the shortest transportation time when compared to standard PSO, GA, and ACO in both small-scale (10 machines) and large-scale (25 machines) scenarios. - IPSO achieved the lowest average Relative Percentage Increase (RPI) of 3.30%, indicating its solutions were closest to optimal compared to GA (6.04%) and ACO (6.98%). - The improvements, particularly the mutation mechanism, were shown to be effective in helping the algorithm avoid premature convergence and find better solutions than the standard PSO.

Table 11 continued

Study Details	
Study: (Research on Multi-Robot 2025)	
Year	2019
Basic Algorithm	PSO
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: No
Sensors and Material	Material: The study involves a multi-robot system of mobile robots. Environment: The experiments are conducted in unknown environments, tested through both simulation and with real mobile robots.
Test Tools	Algorithm: The paper proposes and analyzes several intelligent optimization algorithms for multi-robot target search, including: - A hybrid algorithm combining Particle Swarm Optimization (PSO) and Fruit Fly Optimization Algorithm. - An Adaptive Robotic Bat Algorithm (ARBA). - A Robotic Grey Wolf Optimization (RGWO) algorithm. - A Dynamic Self-Organizing Map (DSOM) neural network for task allocation.
Performance	- The proposed hybrid algorithm (based on PSO and Fruit Fly) was shown to be superior to the comparison algorithms mentioned in the abstract. - The method effectively avoids premature convergence and getting stuck in local optima by using a multi-swarm strategy that improves search diversity.

tion to the problem. However, limitations exist in PSO for path planning situations; that include its potential for premature convergence in environments with complexity, its higher computational costs for dynamic problems and problems that exist in high dimensional spaces, scalability issues, sensitivity to parameters and search space definition, and typically PSO has constraint-handling techniques (obstacle avoidance techniques) included as part of the viable solution set. Several researchers have attempted to improve PSO algorithms by resolving their limitations; for example, this study (Zheng et al. [2023](#)) presents a novel approach to mobile robot path planning by enhancing the particle swarm optimization algorithm with adjusted inertia weight parameters and integrating it with the artificial potential field method, significantly improving global optimization capabilities and addressing limitations of traditional methods. Another study (Yang et al. [2023](#)) introduces an improved particle swarm optimization (IPSO) algorithm that addresses the limitations of traditional PSO, such as premature convergence and local optima, by enhancing velocity updates and inertia weight and incorporating strategies for initialization and local optimum avoidance. Another study (Li and Chen [2005](#)) introduces a novel path planning method for mobile robots using particle swarm optimization combined with adaptive neural network control, enabling smooth trajectory tracking with low computational cost while effectively avoiding obstacles. Also, in this study (Tao and Kim [2024](#)), a bi-population PSO algorithm with adaptive speed adjustment and random perturbation strategies was proposed to enhance the efficiency of path planning. Finally, this research (Dewang et al. [2018](#)) presents a novel approach using adaptive particle swarm optimization (APSO) by presenting A new objective function for path planning optimized using adaptive PSO for mobile robot path planning in static environments. The Table 11 represents other studies that have utilized the PSO method.

5.4 Ant colony optimization (ACO)

Ant colony optimization (ACO), first introduced by Marco Dorigo (Maniezzo [1991](#)), is inspired by the natural behavior of ant colonies during foraging. Ants rely on pheromone trails for communication, a biochemical marker left on the ground as they move. When searching for food, ants initially explore various paths at random, but over time, they converge on the shortest path between their nest and the food source. This is due to a feedback mechanism: paths with higher pheromone concentrations attract more ants, who, in turn, reinforce these trails by depositing additional pheromones. This self-organizing behavior, rooted in swarm intelligence, is effectively adapted to solve optimization problems in robotics and computing. By simulating how ants discover optimal paths, ACO algorithms utilize virtual pheromone maps to guide decision-making. Ants in the algorithmic context are agents exploring potential solutions. Paths with higher "pheromone" values represent better solutions and are iteratively improved through reinforcement, while less effective paths diminish over time. In robotic path planning, ACO has been employed to navigate efficiently through dynamic environments. Modifications to the original ACO, such as combining pheromone-based searches with heuristic evaluations, have enhanced its ability to overcome challenges like local optima and environmental complexity. The result is a robust method for finding globally optimized solutions, analogous to how ant colonies achieve remarkable coordination without centralized control (Figs. [15](#)).

The flowchart depicts the ant colony optimization (ACO) process. When ACO is initialized, a colony of ants is also initialized and positioned at the entry state. Ants will successively select the next state based on pheromonal information and heuristic information. Once an ant reaches a final state, it deposits pheromone along its path while losing some to

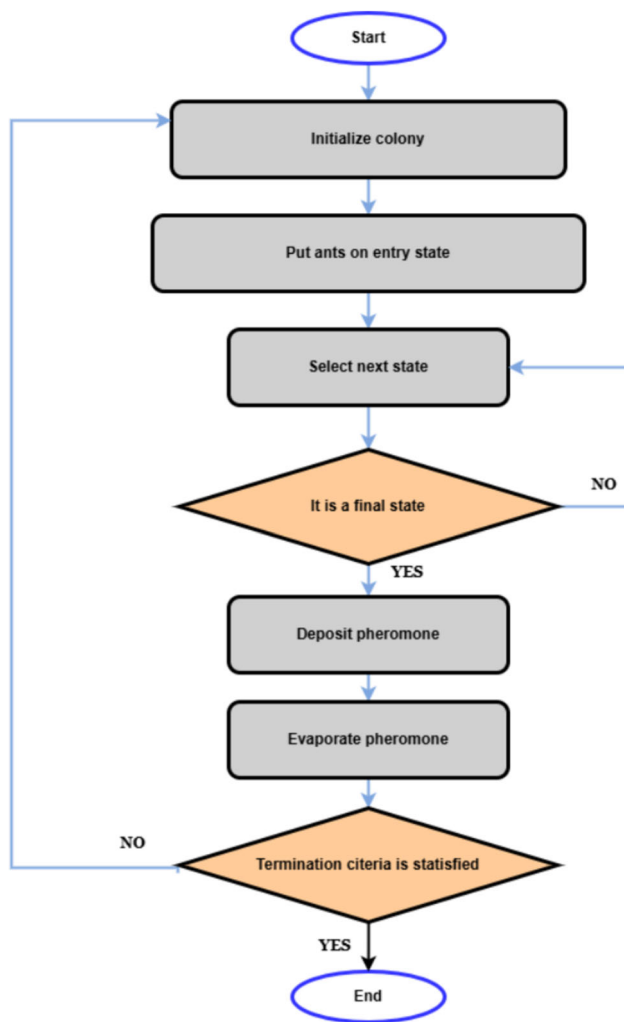


Fig. 15 The ACO process

evaporation - purposed to eliminate the possibility of getting stuck in suboptimal local solutions. This cycle of building a solution and depositing pheromone continues until a termination criterion is reached, such as using an upper limit for the number of iterations to converge. ACO has great potential for use in path planning; however, ACO algorithms have limitations related to the high computational cost in environments that are large, or dynamic, tuning the parameters is critical, and there is a risk of premature convergence if pheromone updates overfavor specific paths. Many researchers have made attempts to improve ASO algorithms because of their limitations and challenges, for example the study [108] developed a new method of robot path planning based on an improved ant colony and enhanced convergence speed and efficiency in complex environments by implementing grid modeling, obstacle avoidance factors, and dynamic pheromone modulation techniques. Another study (Dai et al. 2019) presents a novel improvement to the ant colony algorithm for mobile robot path planning, demonstrat-

ing enhanced convergence speed, reduced path bending, and effective performance in complex map environments through a retraction mechanism and an improved MAX-MIN Ant System method. This study also Garcia et al. (2009) proposes a new path planning technique for mobile robots through a simple ant colony optimization meta-heuristic (SACO-MH) called SACOdm, which accounts for distance and memory to produce better-researched routes. The improvement is especially evident in scenarios without obstacles. Lastly, this study (Zhang et al. 2021) provides major improvements in ant colony optimization, with hybrid algorithms and adaptive approaches. These improvements increase global search ability and optimization efficiency, and address issues that include local optima and a lack of cooperation between ants. The Table 12 represents other studies that have utilized the ACO method.

6 Learning based

Although genetic strategies enable adaptive exploration of the configuration space, the rise of machine learning paves the way for models that learn navigation policies through trial and error; this is the focus of learning-based methods.

6.1 Reinforcement learning

Reinforcement learning (RL) is a machine learning approach that allows an agent to learn the correct interaction patterns with the environment found in its context, based on feedback of rewards (or penalties). RL does not rely on labeled data and does not require an explicit model of the environment, but rather enables learning through exploratory interaction with the environment. The exploratory learning style of RL is ideal for sequential decision-making in uncertain or dynamic environments (Lamini et al. 2017) (Fig. 16).

RL has greatly impacted mobile robot navigation, effectively addressing various navigation issues including path finding, tracking, and obstacle avoidance. Novel algorithms leveraging the efficiency and generality of RL algorithms are being presented. The paper (Dynamic Path 2025), for example, presented the integration of MDP with improved A* path

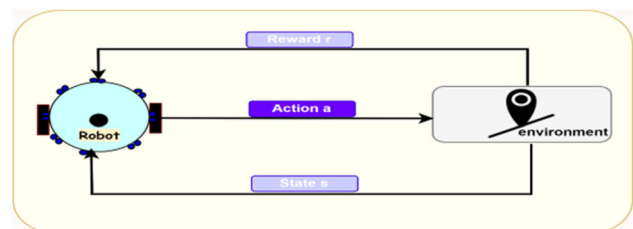


Fig. 16 The reinforcement learning process

Table 12 Other studies using ACO techniques

Study Details	
Study: (Chen and Zhou 2021)	
Year	2021
Basic Algorithm	ACO
Flags	Optimization: Yes, Fusion: No, Simulation: No, Real: No
Sensors and Material	Material: The study uses a generic mobile robot model. Sensors: The paper does not specify the use of any particular sensors, as the path planning is performed on a known grid map. Environment: A simulated grid-based environment with obstacles.
Test Tools	Framework: The paper does not specify the simulation software used for the experiments. Algorithm: An Improved Ant Colony Algorithm is proposed, which modifies the traditional version with features like an adaptive volatilization coefficient and adjustments to the initial pheromone distribution. Metrics: Performance is described qualitatively in terms of convergence speed and the ability to find a stable, optimal path while avoiding local optima.
Performance	- The paper states that the improved algorithm converges faster and more efficiently than the traditional ant colony algorithm. - The method successfully finds better and more stable paths and can avoid getting stuck in local optimal solutions to find the global best route.
Study: (Ji and Liu 2022)	
Year	2022
Basic Algorithm	ACO
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Material: The study uses a generic mobile robot model. Sensors: Not applicable, as the experiments involve planning a path through a known set of points without environmental perception. Environment: A simulated map where points are randomly placed for the path planning task.
Test Tools	Framework: The simulation was developed and run using MATLAB. Algorithm: The study implements a standard Ant Colony Algorithm to solve a Traveling Salesman-like problem for robot path planning. Metrics: Performance is evaluated by the final shortest path length and the path length change curve over iterations.
Performance	- The algorithm successfully solved the path planning problem, finding a shortest path of 14465.495 units for the given set of points. - The algorithm showed fast convergence, with the path length stabilizing to the optimal value after approximately 20 iterations.
Study: (Jiao et al. 2018)	
Year	2018
Basic Algorithm	ACO
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Material: The method is designed for smart wheelchairs. Sensors: The paper does not specify the sensors used, as the experiments are based on a simulated environment with predefined obstacle and target locations. Environment: The experiments are conducted in a simulated grid-based environment.
Test Tools	Framework: An unspecified simulation environment was used to conduct the experiments. Algorithm: The study proposes an Adaptive Polymorphic Ant Colony Algorithm (APACA) and compares it against other variants, including the original ant colony algorithm (OACA), an improved version (IACA), and a general polymorphic version (GPACA). Metrics: The algorithms are evaluated based on the Optimal Path Length and the Number of Iterations needed to find that path.
Performance	- The proposed APACA found the shortest optimal path length in all tested scenarios, for example, achieving a path of 36.9428 m compared to 37.2991 m for other algorithms in one test. - APACA demonstrated significantly faster convergence, finding the optimal solution in as few as 5-6 iterations, while other methods required 10 to 60 iterations. - In obstacle avoidance tests, the optimal path found by APACA was consistently shorter than those found by the IACA and GPACA baselines.

planning to traverse within a dynamic environment which outperforms the traditional Dijkstra's algorithm. By using the hippocampal forward prediction method observed in rats as inspiration, the authors of Huang et al. (2024) propose an advanced Dyna-Q algorithm that improves decision making using biological-inspired processes including vicarious trial and error (VTE). They show their model is also proficient in spatial cognition and dynamic obstacle avoidance ability. In contrast, Munaf and Almusawi (2024) contributed with a hybrid Q-Learning and proportional-integral-derivative (PID) control framework that combines the advantages of RL adaptability and the accountability of PID to give experimental vehicle better trajectory tracking in a random environment. These paper examples show the changing landscape of RL use - the shift is away from the model-free learning paradigm (Q-learning and SARSA) which have slow convergence, towards a model-based paradigm which includes use of a virtual environment (using Dyna-Q for example) before implementing in real world tasks. This Lamini et al. (2017) presents a novel collaborative Q-Learning approach utilizing a Holonic Multi-Agent System (H-MAS). Unlike traditional methods that rely on a single Q-table, the proposed method incorporates two distinct types of Q-tables: the Q-Master Table (QMT) and the Q-Embedded Table (QET). The QMT is updated through a collaborative policy among head holons, facilitating knowledge sharing among agents. In contrast, the QET is updated based on the individual experiences of each agent, enabling them to benefit from both their own experiences and those of other agents within the system. The approach is further supported by a H-MAS architecture, which combines holonic and internal agents, along with a Fuzzy Logic System, to address the challenge of path planning for mobile robots operating in partially or completely unknown environments. This method enhances the robots' ability to make efficient decisions by leveraging both individual and collective learning, thus improving overall performance in uncertain and dynamic settings. The Table 13 represents other studies that have utilized the RL method.

6.2 Imitation learning

Imitation learning (Billard and Grollman 2012) is a promising approach to autonomous navigation for mobile robots. It enables them to learn complex tasks by observing and replicating expert demonstrations (Waga et al. 2025). Unlike traditional programming methods, which require designing explicit rules and reward functions for navigation, imitation learning allows robots to acquire behaviors through examples provided by a human operator or other expert systems (Fig. 17).

In the context of navigation, several researchers have attempted to avoid programming from scratch by utilizing imitation learning techniques. (Karnan et al. 2021)

introduces VOILA (visual-observation-only imitation learning for autonomous navigation), an algorithm enabling autonomous navigation through video-only demonstrations collected from a different platform. It addresses the challenge of viewpoint mismatch between the demonstrator's and the robot's perspectives using a novel reward function based on keypoint detection. VOILA leverages reinforcement learning to train navigation policies that generalize to new environments. Its effectiveness is demonstrated in both simulated and real-world settings. The authors (Nozari et al. 2024) combine hierarchical cognitive representations with active inference and imitation learning to enhance AVs' adaptability to new observations. This integration allows the vehicle to make human-like decisions, particularly in unobserved or dynamic environments, by learning from expert demonstrations and updating its behavior based on new information. In Giusti et al. (2016), researchers gathered 8 hours of video recorded at 30fps using three GoPro cameras mounted on a hiker's head, each pointing in a different direction. This data was then used to train a deep neural network to guide the drone's navigation, relying solely on the angles captured by its onboard cameras. The study in Lu et al. (2016) presents a convolutional neural network (CNN) approach for training an NAO robot to perform path planning and collision avoidance, leveraging a global rolling prediction algorithm. The proposed method is evaluated against support vector machine (SVM) and particle swarm optimization (PSO) techniques within a 2D grid environment featuring dynamic obstacles. The study in Wang et al. (2021) employs artificial potential fields combined with a fuzzy inference system to create datasets for training a neural network. The target generates an attractive force in open spaces that guides the agent. In contrast, in hazardous regions, the repulsive force from obstacles and their orientations are used to determine movement via fuzzy logic. The trained neural network was evaluated against the dynamic window algorithm and ant colony optimization in a dynamic indoor environment, demonstrating superior performance. In Yuan et al. (2019), researchers developed an enhanced ant colony algorithm to address distant obstacles and utilized modified artificial potential fields for nearby obstacles. These methods were employed to train a gated recurrent unit (GRU)-based recurrent neural network. Despite the necessity of establishing kinematic models for both the robot and its environment, the trained network achieved smoother and faster robot motion compared to the teacher network. In Tai et al. (2016), researchers employed a CNN to extract relevant features from raw depth images. These features were then passed through a fully connected decision network, enabling the system to map the input depth images directly to control commands based on the actions of a human operator driving the robot. The two studies (Kim and Chen 2015; Wu et al. 2022) developed CNN to learn from the expert; the first study used a small quadcopter equipped with

Table 13 Other studies using RL techniques

Study Details	
Study: (Dobriborsci et al. 2024a)	
Year	2023
Basic Algorithm	Q-learning
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: Yes
Sensors and Material	Robot: A mobile robot with a differential drive (Robotis Turtlebot3) was used for the experiments. Sensors: The robot used laser scan measurements to perceive obstacles in the environment. Environment: The paper mentions that experiments are conducted in real-world physical environments.
Test Tools	Algorithm: The study proposes and tests a novel algorithm called stacked Q-learning. This is compared against a classical model predictive control (MPC) and a rollout Q-learning method.
Performance	- Both of the reinforcement learning methods (rollout and stacked Q-learning) outperformed the baseline model-predictive controller in terms of accumulated cost. - The stacked Q-learning variant demonstrated the best performance among the tested methods. - The results are described as "promising" and suggest that the proposed agent design can be a foundation for predictive reinforcement learning methods.
Study: (Dobriborsci et al. 2024b)	
Year	2024
Basic Algorithm	Q-learning
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Robot: A mobile robot was used for the experiments. The specific model is not mentioned. Environment: A simulation environment was used, the code for which is available on GitHub.
Test Tools	Framework: A custom simulation environment. Algorithm: The study compares two predictive controllers: a conventional Model-Predictive Control (MPC) as a baseline, and a Q-learning predictive controller, which integrates reinforcement learning (Q-function estimates) into the MPC framework.
Performance	- The proposed learning predictive controller (Q-learning based) outperformed the conventional model-predictive control (MPC) baseline in the experiments. - A key finding is that the learning approach improved performance while maintaining the same computational complexity for the action calculation step as the MPC baseline.
Study: (Maoudj and Christensen 2021)	
Year	2021
Basic Algorithm	Q-learning
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Environment: The robot is trained in complex, continuous, and dynamic environments using a high-fidelity simulator.
Test Tools	Framework: A high-fidelity simulator is used for training and experiments. Algorithm: The study proposes an improved Q-learning approach. This includes a novel state space definition with binary variables and two heuristic algorithms to give the robot prior knowledge.
Performance	- The proposed method significantly shortens training time by using heuristics to reduce the initial "blind exploration" phase. - The experiments demonstrate the effectiveness of the approach in terms of both training time and the quality of the learned navigation policy.
Study: (Yun et al. 2013)	
Year	2013
Basic Algorithm	ANN + Q-learning
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: No
Sensors and Material	Robot: Team AmigoBot™ robot was used for the real-world experiments. Environment: An "unknown environment" was used for both simulation and real-time implementation.
Test Tools	Framework: A simulation was developed in MATLAB to validate the algorithm before real-world deployment. Algorithm: A two-stage Neural Q-Learning method. First, the robot uses Q-Learning to explore and map the environment. Then, this data is used to train a feed-forward back-propagation Artificial Neural Network (ANN). Metrics: The paper mentions that results from both simulation and real-world experiments are discussed, but does not specify the quantitative metrics used for evaluation.
Performance	- This paper primarily presents a methodology for combining Q-Learning with an ANN. - The approach is designed to overcome the challenges of pure Q-Learning, such as the large amount of time and computation required to explore every state-action pair in an environment. - The trained ANN is used to approximate the correct action when the robot encounters a state that was not visited during the initial Q-Learning exploration phase.

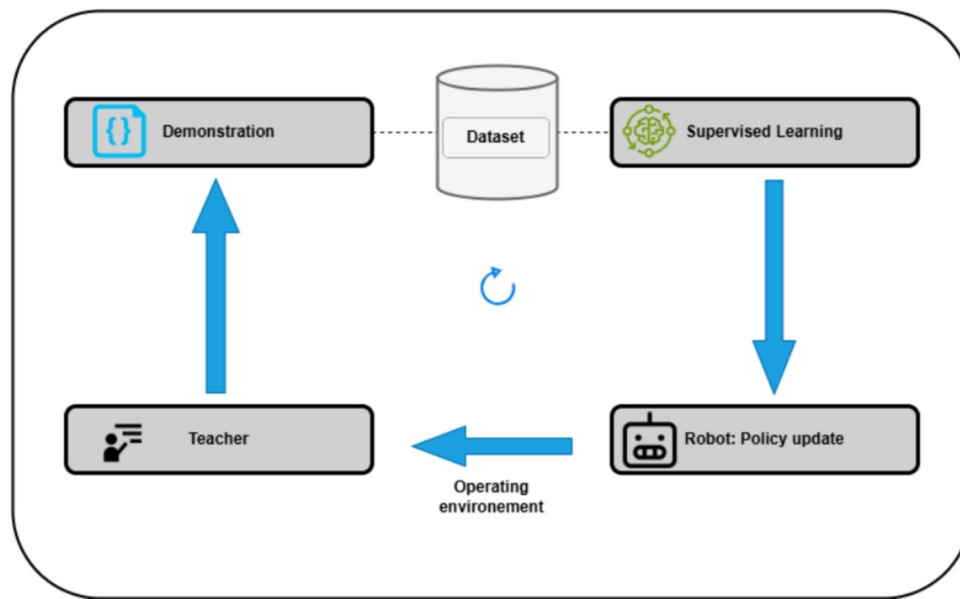


Fig. 17 The imitation learning process

a monocular camera to learn a control strategy using a convolutional neural network trained on an expert pilot's action choices during target object searches. The second study proposed an online three-dimensional path planning (OTDPP) approach based on imitation learning, where actions were labeled using the Dijkstra algorithm. The Table 14 represents other studies that have utilized the IL method.

6.3 Deep reinforcement learning

Recent years have witnessed remarkable progress in artificial intelligence through the development of deep reinforcement learning (DRL) (Botvinick et al. 2019) techniques. These methods have demonstrated superhuman performance across domains, such as Atari games (Mnih et al. 2013), strategic board games like Go, and complex environments like no-limit poker. This success has garnered significant interest from cognitive scientists aiming to explore the parallels between artificial and human learning processes. However, critics often argue that DRL exhibits substantial inefficiencies in sample usage, rendering it an implausible framework for understanding rapid human learning. In this review, we challenge this notion by highlighting innovative approaches that have dramatically enhanced the efficiency and adaptability of DRL systems. Although conceived within AI, these advancements have profound implications for psychology and neuroscience, particularly in bridging fast adaptive learning and slower, incremental learning mechanisms. To address challenges like the curse of dimensionality in high-dimensional state spaces, Q-learning has evolved through parameterizing the optimal action-value function Q . By transforming the traditional Q -table into a function approx-

imation problem, modern implementations leverage neural networks to approximate $Q(s, a; \theta)$, where θ represents the network parameters. DRL has since expanded its utility into autonomous navigation and dynamic obstacle avoidance domains. Agents are typically trained in a simulated environment, and the agent's task is to learn an optimal policy based on reward signals that optimize behavior. Agent's inputs are typically sensor data, including images, lidar point clouds, and state information related to the target position, which are processed by a neural network (NN) to suggest an action that maximizes cumulative reward. Strategies can be improved through enhancing the Q-values, or just improving the policy. While standard algorithms are reasonable for all global motion planning problems in static scenarios, DRL allows agents to develop strategies when dealing with dynamic and finite states of uncertainty, and particularly with respect to collision avoidance through continuous or adaptive decision making. This diagram represents a deep reinforcement learning (DRL) framework for mobile robot control in which the Policy takes actions based on the robot state, and is being improved through the RL update process, with the data generated from Experience. The robot interacts with the environment, receiving states and rewards. Simultaneously, a model trained using collected experience simulates the environment, generating more data for training within the Simulator block of the framework. The framework provides a mechanism to optimize learning and use of the mobile robot by using a mix of real and simulated experience while leveraging the simulation environment for safe policy optimization and control mechanisms to guide actions (Fig. 18).

The work Lee and Yusuf (2022) proposed an integrated framework employing deep reinforcement learning (DRL)

Table 14 Other studies using IL techniques

Study Details	
Study: (Hamandi et al. 2019)	
Year	2019
Basic Algorithm	CNN + LSTM
Flags	Optimization: No, Fusion: Yes, Simulation: Yes, Real: Yes
Sensors and Material	Sensors: The model uses raw data from a 360° LiDAR as its primary input. Hardware: The algorithm was tested in real-time on a 3.3 GHz CPU, with a GPU used for the training phase. Environment: The system was trained and tested in a simulator using the ETH BIWI pedestrian dataset, which provides real-world human trajectories on two different maps.
Test Tools	Framework: A custom simulator was used to generate LiDAR scans based on the pedestrian dataset. Algorithm: DeepMoTion, a deep imitation learning model featuring convolutional layers and two LSTM branches to separately predict robot speed and direction.
Performance	- The model reached the target in 100% of test cases, outperforming other learning-based benchmarks. - It achieved the best human imitation results, with a 24% reduction in path deviation (DTW metric) compared to the next-best algorithm.
Study: (Waga et al. 2024)	
Year	2024
Basic Algorithm	Ensemble Learning
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: No
Sensors and Material	Sensors: The simulated robot uses eight ultrasonic distance sensors distributed around its chassis. Hardware: Experiments were simulated on a PC with an Intel Core i3 CPU and 8 GB of RAM. Environment: The system was trained and validated in 40 different simulated 2D environments (53x53 pixels) with varying levels of obstacle complexity.
Test Tools	Framework: A custom simulation environment was developed in Python. Algorithm: The study uses Learning from Demonstration, where an A* algorithm acts as the expert. Several machine learning classifiers (Random Forest, Xgboost, Adaboost, Bagging) are trained on the expert's data and then combined using ensemble voting techniques. Metrics: The models were evaluated on classification accuracy during training and on Success Rate, Average Elapsed Time, and Average Path Traveled during validation.
Performance	- The best individual classifier (Random Forest) achieved a success rate of 77.5% in new environments. - The ensemble models improved performance significantly, with the best combinations achieving a success rate of 87.5%.
Study: (Waga et al. 2023)	
Year	2023
Basic Algorithm	CNN
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Sensors: A simulated mobile robot equipped with eight ultrasonic sensors. Hardware: Simulations were performed on a PC with an Intel Core i3 CPU and 8GB of RAM. Environment: The model was trained and validated in 40 simulated 2D environments (53x53 pixels) of varying difficulty.
Test Tools	Framework: A custom simulation environment was created in Python. Algorithm: The study uses Deep Imitation Learning. A Convolutional Neural Network (CNN) is trained on demonstrations generated by the Dijkstra algorithm, which serves as the navigation expert.
Performance	- The trained model achieved a high success rate of 89.8% when tested on 98 new, unseen environments. - During training, the model learned the navigation policy efficiently, reaching over 90% accuracy after 50 epochs without overfitting.
Study: (Cèsar-Tondreau et al. 2021)	
Year	2021
Basic Algorithm	CNN
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: No

Table 14 continued

Study Details

Sensors and Material	Robot: A simulated Clearpath Husky ground robot. Sensors: A 360° planar LiDAR and four cameras providing panoramic visual data that is converted into a semantic map. Environment: Experiments were conducted in both Gazebo and Unity simulators, each with distinct training and testing environments.
Test Tools	Framework: The model was integrated into the ROS navigation stack, acting as an intermediary between the global and local planners. Algorithm: An Imitation Learning model trained via Behavioral Cloning. It learns from both a traditional path planner and a small number of human demonstrations. Metrics: Successful Navigation Rate and Trajectory Similarity, measured using the Modified Hausdorff Distance (where higher values indicate greater similarity).
Performance	- The proposed method achieved a very high navigation success rate (over 90%) in both familiar and novel testing environments, significantly outperforming the baseline methods. - It produced trajectories that were statistically more similar to the human demonstrator's paths compared to the baselines.

at reducing the uncertainty associated with decision-making by mobile robots in dynamic environments. The authors proposed a method that amalgamated real-world interactions with simulated environments, and learned an environment model, simulated experiences were used to fine-tune the policy of the robot. This approach allows the robot to learn in a safe environment, with a known situation, thereby enhancing the robots ability to operationalize knowledge gained from simulated environments when making real-world decisions in the future, this also allows for a safe manner of learning in complicated environments where there may be complexity or danger, since it allows for learning without the possibility of physical danger to either the robot, the humans around it or any physical objects in its work environment. This Quan et al. (2020) work develops an end-to-end navigation method for mobile robots using deep reinforcement learning. The authors describe the dueling double deep Q-network (D3QN) algorithm that allows mobile robots to autonomously navigate in unknown indoor environments while using only RGB-D cameras to take depth images. By leveraging the D3QN approach, the robot learns its surroundings and navigation strategies through interaction without requiring pre-mapped environments or supervised labels. The model

was validated in simulation environments using Gazebo. The paper (Ruan et al. 2019) introduces a navigation method for mobile robots that integrates recurrent neural networks (RNN), specifically GRU (gated recurrent unit), with Double deep Q-learning (DDQN). This approach focuses on improving path planning in dynamic environments by designing an enhanced reward system and discount factor, enabling robots to navigate more efficiently. Experiments conducted in both 2D and 3D simulated environments and on physical robots demonstrate that the proposed method enhances global pathfinding efficiency, reduces computational time, and minimizes path lengths compared to traditional algorithms. The Table 15 represents other studies that have utilized the DRL method:

6.3.1 State-of-the-Art DRL methodologies and platforms

To overcome the limitations of early DRL methods, such as sample inefficiency, modern research has converged on a set of standardized algorithms and simulation platforms. These tools enable rigorous, reproducible, and fair comparisons of new contributions. We will detail the most prominent of these below.

Dominant reinforcement learning algorithms

While foundational algorithms like DQN were revolutionary, the field has since evolved. The following policy-gradient methods are now central to state-of-the-art research in robotic navigation.

Proximal policy optimization (PPO)

Proximal Policy Optimization (PPO) (Schulman et al. 2017) is a highly effective reinforcement learning algorithm used to teach a robot how to move intelligently from a starting point to a goal while avoiding obstacles. The core strength of PPO lies in its "trust region" approach, which ensures that

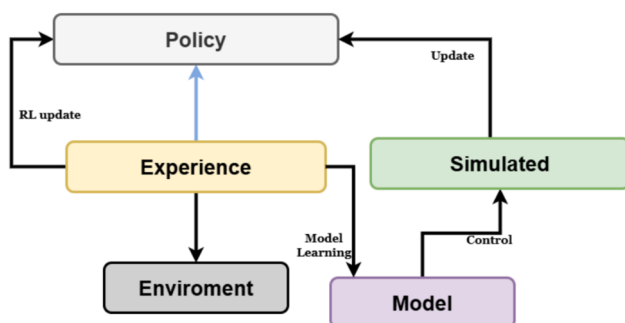
**Fig. 18** The deep reinforcement learning process

Table 15 Other studies using DRL techniques

Study Details	
Study: (Mirowski et al. 2017)	
Year	2017
Basic Algorithm	A3C
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Sensors: RGB images (84x84), depth maps (used as an auxiliary learning task), and agent-relative velocity data. Environment: Five different types of 3D mazes (static goals, random goals, I-maze) within the DeepMind Lab simulation environment.
Test Tools	Framework: DeepMind Lab simulator. Algorithm: Asynchronous Advantage Actor-Critic (A3C) and its variations (e.g., Nav A3C) augmented with auxiliary learning tasks for depth and loop-closure prediction. Metrics: Cumulative reward, position decoding accuracy, and latency to goal, benchmarked against human expert performance.
Performance	- The best models achieved human-level performance on mazes with static goals. - On more difficult mazes with random goals, the agent achieved 59% to 91% of human performance. - The agent learned to localize itself, achieving up to 85.5% position decoding accuracy without being explicitly trained for it.
Study: (Zhang et al. 2018)	
Year	2018
Basic Algorithm	LSTM + A3C
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: No
Sensors and Material	Robot: A 3D physics model of a Jaguar 4x4 wheeled mobile platform. Sensors: The robot model was equipped with a front-facing depth sensor and 3D odometry. Environment: The system was trained in 2,000 unique, randomly generated 100 m ² simulated environments containing hills, valleys, and irregular rubble piles.
Test Tools	Framework: Gazebo simulator within the Robot Operating System (ROS). Algorithm: Asynchronous Advantage Actor-Critic (A3C) with a custom network using convolutional layers (CNNs) and a Long Short-Term Memory (LSTM) layer. Metrics: The success rate of the robot reaching its goal and the average computation time per action.
Performance	- The model achieved high success rates, ranging from 71.3% to 85%, in new environments it had not seen during training, demonstrating robust generalization. - The navigation module was highly efficient, taking an average of only 0.074 seconds to compute and select a navigation action.
Study: (Zhu et al. 2017)	
Year	2017
Basic Algorithm	AC
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: Yes
Sensors and Material	Robot: SCITOS mobile robot for real-world testing. Sensors: The model uses only visual input from an RGB camera. Environment: A custom simulator, AI2-THOR, with 32 photorealistic 3D indoor scenes (kitchens, living rooms) and a real-world office/lab setting.
Test Tools	Framework: AI2-THOR simulator, which integrates the Unity 3D engine with Tensorflow. Algorithm: A novel deep siamese actor-critic network based on the A3C training protocol, which takes the current state and a target image as inputs. Metrics: Average trajectory length (number of steps to reach the goal) and success rate for generalization tasks.
Performance	- Significantly outperformed standard DRL methods, achieving an average trajectory length of 210.7 steps versus 723.5 for a comparable A3C model. - The policy trained in simulation was successfully transferred to a real robot, with fine-tuning converging 44% faster than training from scratch on real images.
Study: (Surmann et al. 2020)	
Year	2020
Basic Algorithm	GA3C
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: Yes

Table 15 continued

Study Details

Sensors and Material	Robot: Turtlebot 2 (Kobuki) platform. Sensors: A 2D laser scanner (Hokuyo UST-20LX) and an Intel Realsense D435 RGB-D camera. Data is fused into a single 2D scan format. Hardware: An Nvidia Jetson TX1 on the robot and a PC with a GeForce 1070ti for training. Environment: A custom-built, high-speed 2D simulator using real and hand-drawn maps, along with real-world tests in a university lab.
Test Tools	Framework: A custom C++ parallel simulator and the Robot Operating System (ROS) for real-world deployment. Algorithm: A GPU-accelerated Asynchronous Advantage Actor-Critic (GA3C) algorithm. Metrics: Average score (per step and per episode) and the average number of steps per episode.
Performance	- The custom simulator was about 1000 times faster than alternatives like Gazebo, reducing training time from months to hours. - After training, the agent achieved collision-free navigation in 95% of test runs in the simulated environment. - The policy was successfully deployed from the simulator to the real Turtlebot 2, demonstrating good performance similar to a human in real-world navigation without a map.
Study: (Everett et al. 2018)	
Year	2018
Basic Algorithm	GA3C + LSTM
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: Yes
Sensors and Material	Robot: A small ground robot chassis, likely a Turtlebot derivative, was used for hardware experiments. Sensors: A single 2D RPLidar and three Intel Realsense R200 cameras. Notably, a 3D Lidar was not used. Environment: A custom simulation environment for multi-agent scenarios and a real-world setting with human pedestrians.
Test Tools	Framework: Tensorflow and a custom Python simulation; ROS for hardware deployment. Algorithm: GA3C-CADRL, an actor-critic (GA3C) approach that uses an LSTM at the input layer to handle a variable number of observed agents. Metrics: Percentage of failures (collisions or getting stuck) and average extra time to goal compared to a straight path.
Performance	- The network is highly efficient, with a single query taking only 0.4-0.5ms on a CPU (20x faster than the previous approach). - The use of an LSTM successfully allowed the network to process an arbitrary number of agents as input, a key limitation of prior work. - The algorithm was demonstrated on a real robot navigating safely among pedestrians at human walking speed.
Study: (Yin et al. 2023)	
Year	2023
Basic Algorithm	DQN
Flags	Optimization: Yes, Fusion: No, Simulation: Yes, Real: Yes
Sensors and Material	Robot: Turtlebot3 Waffle Pi. Sensors: A 2D 360° LiDAR is used to provide environmental data as input to the network. Hardware: An NVIDIA Jetson TX2 board on the robot and an Intel i5 CPU for simulations. Environment: Three simulated stages in Gazebo with static and dynamic obstacles, and a corresponding real-world lab environment.
Test Tools	Framework: Gazebo simulator and the Robot Operating System (ROS). Algorithm: A novel method called RND3QN, based on n-step Dueling Double DQN with a reward-based e-greedy strategy. Metrics: Total reward per episode and success rate, measured as the average number of successful goal arrivals per episode.
Performance	- The RND3QN algorithm increased path planning success rates by 174%, 65%, and 61% over the D3QN baseline across the three different simulation stages. - The model achieved a significantly higher average total reward compared to other tested DQN-based algorithms.

the robot's decision-making policy is updated in small, incremental steps. This prevents drastic, unpredictable changes in behavior during training, leading to more stable and reliable learning. By efficiently learning from its experiences of rewards (for getting closer to the goal) and penalties (for collisions), the robot develops a sophisticated navigation strategy. This makes PPO particularly well-suited for handling the continuous action spaces of robotics, enabling the smooth control of velocity and steering required for fluid movement in complex, real-world environments. Recent studies investigating the application of the PPO algorithm to a range of mobile robotics tasks can also be consulted (Taheri et al. 2024; Cheng et al. 2024; Kabas 2022).

Soft actor-critic (SAC)

Soft Actor-Critic (SAC) (Haarnoja et al. 2018) is another advanced reinforcement learning algorithm that stands out for its remarkable efficiency and exploratory capabilities. Unlike methods that solely focus on maximizing rewards, SAC incorporates the concept of entropy, meaning it not only tries to find the best path but also actively encourages the robot to explore as many different, viable paths as possible. This intrinsic drive for exploration helps the robot to discover non-obvious or more robust solutions and prevents it from getting stuck in a single, suboptimal strategy. As an "off-policy" algorithm, SAC can learn from a replay buffer of past experiences, making it highly sample-efficient, which is crucial for reducing the wear and tear and training time on physical robots. This combination of maximizing rewards while also encouraging exploration results in a robot that learns a stable and effective navigation policy very quickly, often demonstrating smoother and more intelligent-looking behavior in complex and cluttered environments. Recent studies investigating the application of the SAC algorithm to a range of mobile robotics tasks can also be consulted (Elallid et al. 2024; Mohammad and Bezzo 2025; Lun et al. 2024; Regragui et al. 2025).

Leading simulation and testing platforms Validating and training DRL agents for navigation requires simulators that are both fast and realistic. The following platforms are the current standards in the field.

The AI habitat platform AI Habitat is a high-performance, open-source simulation platform specifically designed for research in Embodied AI (agents that act within an environment) (Savva et al. 2019). Its primary advantage is speed, enabling the training of agents across thousands of photo-realistic 3D environments in parallel at speeds of thousands of frames per second. For instance, a researcher could train a robot to navigate from a starting point to a specified object (e.g., 'go to the television') in thousands of different simulated apartments within hours—a task that would be

prohibitively slow, costly, and dangerous to perform with a physical robot.

The AI2-THOR and RoboTHOR simulators While Habitat excels at navigation, the AI2-THOR simulator focuses on rich physical interaction between an agent and objects in an environment (Kolve et al. 2022). It provides a virtual laboratory where an agent can learn to open doors, pick up objects, and manipulate appliances. A key feature of this ecosystem is RoboTHOR, which provides a physical robot counterpart to the simulated agent. This is critical for addressing the "sim-to-real" gap; a policy that works perfectly in a clean simulation might fail on a real robot due to unmodeled physics like wheel slippage, sensor noise, or unexpected lighting changes. RoboTHOR provides a direct testbed to measure and mitigate this gap, making it an indispensable tool for developing truly deployable robotic systems.

6.4 Large language model

In recent years, the development of large language models such as ChatGPT, Gemini, and DeepSeek (Nazir and Wang 2023; Kalyan 2024; Ray 2023) has marked a significant advancement in artificial intelligence. These models have been applied across various domains, including mobile robotics, demonstrating their versatility and transformative potential (Fig. 19).

Several research studies have been published recently, addressing various issues related to robotics in general and large language models (LLMs). However, in this section, we focus specifically on using LLMs in the context of autonomous navigation. For example, trajectory planning algorithms such as A* and Dijkstra perform well in static environments. However, their performance significantly decreases when semantic command interpretation is required. To address this limitation, a study Doma et al. (2024) proposes integrating large language models (LLMs) into mobile robot navigation by introducing the dynamic chain of instruction-based planning (DCIP) framework., which leverages large language models (LLMs) to bridge the gap between static path planners and context-aware navigation in dynamic, resource-limited settings. Its main contributions are threefold:

- (1) Enhanced Environmental Understanding, achieved by fusing 2D occupancy grids with visual and language inputs so the robot can semantically recognize and classify obstacles (e.g., potholes or repair zones) rather than just geometric occupancy.
- (2) Real-Time Instruction-Based Planning, whereby natural language commands are parsed into structured actions

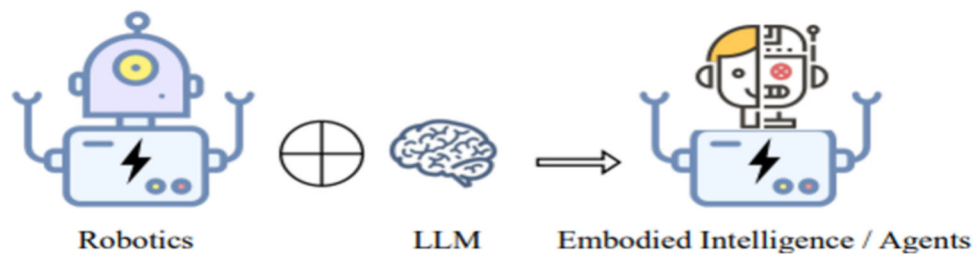


Fig. 19 Robotics based on LLM

that adaptively update goals and constraints on the fly, enabling flexible replanning in response to changing surroundings ;

- (3) Dynamic Occupancy Grid Manipulation, which uses semantic landmarks extracted via NLU to modify traversal costs in designated regions and thus steer path planners toward safe, efficient routes without heavy sensor requirements . These elements allow DCIP to decompose high-level instructions into sub-tasks, continuously recalculating paths that respect safety norms and user preferences, demonstrating significant improvements in adaptability and task reliability over traditional A*-based methods. Another study Latif (2024) introduces 3P-LLM, a probabilistic path planning framework that uses the generative power of GPT-3.5-turbo to guide a robot through complex environments in real-time . Rather than relying solely on geometric planners, 3P-LLM converts the robot's current map (built via GMapping from lidar scans), pose, and high-level instruction into a natural-language prompt. An intermediate "translator" service publishes scan, map, and pose information as text to GPT-3.5-turbo, then parses its response back into velocity commands for execution on ROS. In gazebo simulations, 3P-LLM was pitted against classic planners A* and RRT. It achieved a remarkable 10 ms average planning time while producing paths only slightly longer (6.34 m on average) and maintaining an 81% correctness rate . These results highlight 3P-LLM's ability to leverage LLM reasoning for fast, adaptive navigation with minimal additional sensing or computation, laying the groundwork for more intelligent, language-driven robotic control. This study Tariq et al. (2025) introduces a novel LLM-based dynamic waypoint generation framework for robust mobile robot path planning in complex environments. By leveraging large language models to interpret high-level natural language commands, the system translates user inputs into actionable sequences of waypoints. It dynamically adjusts its path in response to unexpected obstacles via a real-time replanning mechanism. Evaluated across three simulated corridor layouts of increasing complexity, the framework, particularly when using the llama3.1 model,

demonstrated faster planning times, higher waypoint generation success rates, and superior collision avoidance compared to other LLMs, thereby showcasing the potential of integrating LLMs for safer, more adaptable autonomous navigation. Another study Mahmud et al. (2025) introduces a knowledge-driven framework that leverages large language models (LLMs) to enhance robotic odor source localization by combining real-time sensory inputs with a memory module of past experiences. The proposed system encodes environmental data into natural language descriptions, retrieves the most relevant few-shot examples via embedding-based similarity, and constructs Chain-of-Thought prompts for the LLM to reason step-by-step about whether to surge upwind or cast crosswind. By integrating contextual understanding and scenario-specific memories, the robot makes more informed navigation decisions, achieving higher accuracy, efficiency, and generalization in both simulated and real-world plume fields compared to traditional Deep Q-Network and bio-inspired methods The Table 16 represents other studies that have utilized the LLM method:

6.4.1 Advances in vision-language navigation (VLN)

The most impactful application of LLMs to robot navigation is within the specialized research area of Vision-Language Navigation (VLN).

Technically, VLN models must solve the critical challenge of language grounding—connecting abstract linguistic concepts to concrete visual data and physical actions. State-of-the-art architectures typically employ a transformer-based structure with cross-modal attention mechanisms. These models learn to associate phrases in the instruction (e.g., "the red chair") with specific object features extracted from visual inputs (e.g., from a CNN backbone). The fused linguistic and visual representation is then used by a policy network, often recurrent to maintain memory of the environment and instructions, to output a sequence of low-level actions (e.g., move-

Table 16 Other studies using LLM techniques

Study Details	
Study: (Liu et al. 2024)	
Year	2024
Basic Algorithm	Neural SDF + LLM
Flags	Optimization: No, Fusion: Yes, Simulation: No, Real: Yes
Sensors and Material	Robot: Robotnik Summit-XL Gen with Kinova arm. Sensors: 2x RealSense D435 RGB-D cameras, Lidar scanner. Materials: Engine block components (valve cover, ignition coils).
Test Tools	Software: ROS, CUDA. AI Models: GPT-4, Lidar-based SLAM, Neural SDF. The system fuses Neural object-field 3D reconstruction and 6D pose with visual-servoing and LLM-driven logic reasoning.
Performance	- Reconstruction: Precise 3D model and 6D pose from a 708-frame video. - Interaction: Interpreted natural language voice commands to execute multi-step tasks. - Robustness: Tracked dynamic objects at 30 Hz and handled unexpected events (e.g., dropped tool).
Study: (Sun et al. 2025)	
Year	2025
Basic Algorithm	Swin-U-Net + LLM
Flags	Optimization: No, Fusion: Yes, Simulation: Yes, Real: Yes
Sensors and Material	Robot: Kobuki mobile robot. Sensors: Astra RGB-D camera, 2D LiDAR (RPLIDAR A1). Hardware: PC (Intel i9, RTX 3080Ti). Environment: 50.36 m ² house.
Test Tools	Simulator: Habitat. Datasets: Gibson, Matterport3D (MP3D). Software: ROS, PyTorch, Hector SLAM, GPT-4. AI Models: Fuses Multi-channel Swin-U-Net for map-based navigation with LLM-inferred object-to-room priors. Metrics: Success Rate (SR), Success-weighted by Path Length (SPL).
Performance	- Simulation (Gibson): SR 78.3%, SPL 44.6%. - Simulation (MP3D): SR 50.1%, SPL 19.1%. - Real-World: SR 82.8%, SPL 74.9%. - Improvement: 10.6% average SPL improvement over other methods.
Study: (Ye et al. 2025)	
Year	2025
Basic Algorithm	VLM + RRT
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: No
Sensors and Material	Robot: Unmanned Aerial Vehicle (UAV). Sensors: Assumes environmental snapshots from a system with satellite imagery and meteorological data. Hardware: Not specified (simulation-based). Environment: Bounded 2D environment (500m x 500m) simulating a fire-affected forest.
Test Tools	Simulator: Custom simulation framework using the Gurobi solver for optimization. Datasets: No standard datasets; uses Monte Carlo simulations with 100-250 randomly generated environments. Software: Gurobi. AI Models: OpenAI GPT-4o, Meta Llama 3.2 90B Vision Instruct. Metrics: Success Rate, Average Number of Iterations (N), Average Path Length (m).
Performance	- VLM-RRT (GPT-4o, Few-shot) achieved a 94% success rate, compared to 82% for traditional RRT. - Required significantly fewer iterations for convergence (avg. 89) than RRT (avg. 343). - Produced shorter paths (avg. 46m) compared to RRT (avg. 58m). - Achieved path quality comparable to RRT* but with faster convergence. - Showed 92% detection rate for dynamic goal locations.
Study: (Meng et al. 2025)	
Year	2024
Basic Algorithm	LLM + A*
Flags	Optimization: Yes, Fusion: Yes, Simulation: Yes, Real: No
Sensors and Material	Robot: Not applicable (algorithmic focus). Sensors: Not applicable (input is map data). Hardware: Not specified (simulation-based). Environment: 100 manually selected 50x30 maps with horizontal and vertical barriers.
Test Tools	Simulator: Custom Python-based system for pathfinding in a continuous environment. Datasets: Custom dataset of 1000 samples (100 maps with 10 start/goal pairs each). Software: Python, shapely, matplotlib. AI Models: GPT-3.5-TURBO, LLAMA3-8B-16bit. Metrics: Operation Ratio, Storage Ratio, Relative Path Length, Valid Path Ratio.
Performance	- LLM-A* (LLAMA3, Few-Shot) reduced operations to 44.59% and storage to 64.02% compared to A*. - Path length was only slightly increased (102.47%) compared to the optimal path from A*. - Maintained a 100% valid path ratio across all tests. - Exhibited near-linear scalability with increasing environment size, whereas A* showed exponential growth.

forward, turn-left). This process allows the agent to translate a high-level command into a series of executable steps grounded in its perception of the world.

We introduce this subfield and its standard evaluation platforms below.

The emergence of vision-language navigation (VLN)

VLN is a subfield of Embodied AI where an agent must follow detailed, natural language instructions to navigate within a visually-rendered environment (Tsai et al. 2023). The core challenge is "grounding" abstract language into concrete perception and action. For example, an instruction like, "Find the book I left on the kitchen counter, then bring it to the upstairs office," requires the agent to perform a complex chain of reasoning: identify multiple objects ("book," "kitchen counter"), understand spatial prepositions ("on"), navigate between different rooms ("kitchen," "office"), and recall hierarchical goals ("upstairs"). Multiple studies have attempted to advance Vision-and-Language Navigation (VLN) by addressing key challenges in realistic environments. The Yue et al. (2024) study, Safe-VLN, focuses on the critical but often overlooked problem of collisions in continuous environments. Its main idea is to improve safety by classifying collisions into three categories—waypoint, navigation, and dynamic—and introducing an algorithm to mitigate them. The Safe-VLN framework uses two key components: a waypoint predictor that incorporates a simulated 2D LIDAR occupancy mask to prevent planning routes through obstacles, and a "re-selection navigator" that allows the agent to choose an alternative sub-goal if it becomes trapped or encounters a collision. A primary limitation of this research is that its evaluations are conducted entirely within a simulator, with plans for future work to deploy it on a real-world robot and address the sim-to-real transfer challenge. Furthermore, its concept of "dynamic collision" is simulated by probabilistically making waypoints non-navigable rather than by interacting with actual moving objects. A different approach, DynamicVLN (Sun et al. 2025), was proposed to bridge the gap between static VLN tasks and the complexities of the real world by introducing dynamic elements. The main idea of this work is the creation of a new task and dataset that incorporates scenarios with moving vehicles, pedestrians, changing traffic signals, and variable weather conditions based on the CARLA simulator. To handle these situations, the framework introduces a "temporal stop" action, allowing the agent to pause in response to dynamic events before continuing on its path. The study's primary limitation is that the agent's response to dynamic events is confined to this temporary pause rather than more complex behaviors like re-routing. Furthermore, like the previous study, its dataset and baseline model evaluation are purely simulation-based. Shifting the focus from ground robots to aerial vehicles, the

Saxena et al. (2025) study introduces UAV-VLN, an end-to-end framework for Unmanned Aerial Vehicles (UAVs) that interprets free-form natural language commands. Its core idea is to use a fine-tuned Large Language Model (LLM) to parse a high-level user instruction into a sequence of simpler, actionable sub-goals. These sub-goals are then passed to an automated task planner that generates a flight plan, while an open-vocabulary vision model, Grounding DINO, locates the relevant objects from the instruction in the UAV's camera feed. The main limitation of this approach, as noted in its future work, is the lack of navigation history or semantic mapping, which prevents the UAV from reasoning globally to avoid redundant exploration or plan more efficient long-horizon paths. This framework was also evaluated exclusively in the Gazebo simulation environment, without deployment on a physical drone.

Key benchmarks for instruction-guided agents To drive progress and ensure fair comparisons, the research community has developed standardized benchmarks for VLN.

The VLN-CE benchmark

The Vision-and-Language Navigation in Continuous Environments (VLN-CE) benchmark (Krantz et al. 2020) represents a significant increase in realism and difficulty. It operates within the Habitat simulator and requires agents to navigate in photorealistic, continuous 3D spaces. This means the agent is not restricted to a predefined graph of nodes but can move freely, making decisions at a much finer granularity. This setup more accurately reflects the real world, where a robot must navigate through open spaces and around obstacles with arbitrary shapes and positions, making VLN-CE a more challenging and realistic test of an agent's true navigation abilities.

The ALFRED benchmark The Action-Learning-From-Video-and-Text-At-Home-Depot (ALFRED) (Shridhar et al. 2020) benchmark pushes the boundaries even further by focusing on long-horizon, multi-step tasks that combine navigation with object interaction. For the task "Rinse a mug and place it in the coffee maker," the agent must execute a long sequence of actions: locate a mug, navigate to it, pick it up, find a sink, turn on the faucet, rinse the mug, turn it off, locate the coffee maker, navigate to it, and finally place the mug inside. The ALFRED benchmark measures success on these long and complex sequences of interactions, making it a key test for high-level, hierarchical reasoning in household environments.

Advanced simulators for VLN research The aforementioned benchmarks are enabled by powerful and realistic simulation platforms. Habitat and AI2-THOR are the primary environments used for VLN research, as they provide the

photorealistic scenes and rich interaction capabilities needed to train and evaluate these complex models. For example, both VLN-CE and ALFRED are implemented within the Habitat and AI2-THOR simulators, respectively. These platforms provide the necessary APIs for perception (RGB-D cameras, semantic segmentation) and action (movement, object interaction), creating a standardized environment where different models can be fairly compared on these complex, language-driven tasks.

7 Discussion

This review has surveyed a broad spectrum of autonomous navigation techniques for mobile robots, ranging from established graph-search algorithms to contemporary learning-based methods and the emerging integration of Large Language Models. The analysis of cited studies allows for a discussion centered around the key questions regarding the strengths, weaknesses, computational demands, sensor dependencies, applicability, and inherent challenges of these diverse approaches.

7.1 Strengths and weaknesses of navigation methods

The reviewed literature highlights a clear trade-off across different navigation paradigms.

- Graph-based search (Dijkstra, A*, D*): These methods, particularly A* and Dijkstra, excel in finding optimal paths in static, known environments (Dijkstra 1959; Foad et al. 2021). Their strength lies in systematic exploration and guaranteed completeness where solutions exist. However, their primary weakness is computational cost, especially in large or complex environments, and the inherent difficulty in handling dynamic obstacles without significant modifications (Guruji et al. 2016). D* (Optimal and Efficient 2025) was specifically developed to address dynamic environments by replanning efficiently but still relies on graph discretization. Improvements often focus on heuristic refinement (Jingcun et al. 2007; Erke et al. 2020; Fransen and Van Eekelen 2023) or hybrid approaches (Zhenyang and Wei 2022; Zagradjanin et al. 2021) to mitigate weaknesses like path smoothness (Xin et al. 2019) or computational load (Guruji et al. 2016). Neural A* (Yonetani et al. 2021) represents a significant shift, making the search process data-driven and differentiable.
- Sampling-based (RRT, PRM): These methods are particularly effective in high-dimensional configuration spaces and complex environments where graph-based methods become computationally intractable. Their strengths are probabilistic completeness and relative ease of implementation. Weaknesses include non-optimal paths (though variants like RRT* address this) and potential difficulties in narrow passages, which require specific optimizations (Qiao et al. 2022). Improvements focus on efficiency (bidirectional search), path quality (cubic splines), and handling dynamic elements (Ravankar et al. 2020).
- Gradient-based (APF): The artificial potential field method is simple and reactive, suitable for real-time obstacle avoidance. Its major drawback is its susceptibility to local minima, which could potentially trap the robot (Research on Automatic 2025; Wang et al. 2021). Hybridization with Fuzzy Logic (Zheyi and Bing 2021), Neural Networks (Wang et al. 2021), or other planners (Shin and Kim 2021) is a common strategy to overcome this limitation.
- Bio-inspired algorithms (GA, PSO, ACO): Inspired by nature, these metaheuristics demonstrate strength in solving complex optimization problems, potentially finding near-optimal solutions in large search spaces where other methods fail (Mahmud et al. 2019; Chai et al. 2021; Chen and Zhou 2021). However, they often suffer from high computational costs, sensitivity to parameter tuning, and potential premature convergence. Research focuses on improving convergence speed and hybridization (Qiuyun et al. 2021) and adapting them for dynamic environments (Ab Wahab et al. 2024).
- Learning-based (RL, DRL, IL): These methods offer significant advantages in adapting optimal policies through interaction with dynamic and uncertain environments without explicit modeling (Botvinick et al. 2019; Mnih et al. 2013). DRL, in particular, can handle high-dimensional inputs like camera feeds (Lee and Yusuf 2022; Quan et al. 2020). Key weaknesses include sample inefficiency, the need for careful reward shaping (Ruan et al. 2019), and the complexity of training and deployment. Hybrid approaches (Q-learning + PID (Munaf and Almusawi 2024)) and biologically inspired models (Huang et al. 2024) aim to enhance efficiency and performance. Imitation learning (Hamandi et al. 2019; Waga et al. 2024) offers an alternative by learning from demonstrations.
- Large language models (LLMs): Representing the newest frontier, LLMs bring the potential to leverage common-sense reasoning, interpret natural language commands, and process unstructured environmental data. Their strength lies in high-level understanding and human-robot interaction. Weaknesses currently include computational expense, the challenge of grounding language in physical actions, and ensuring real-time performance and reliability in safety-critical navigation tasks. Their appli-

cation is still emerging, often fused with other sensors and mapping techniques Table 16.

7.2 Computational resource requirements

Computational demand varies significantly.

- High demand: Traditional Dijkstra and A* can be demanding in large grid maps. Bio-inspired algorithms like GA, PSO, and ACO often require numerous iterations, making them computationally intensive. DRL training is notoriously resource-heavy, and inference with large DRL models or LLMs can also be demanding.
- Moderate/Variable demand: Sampling-based methods (RRT, PRM) vary depending on the required path quality and the complexity of the environment. D* involves replanning, adding overhead compared to the A* in static scenes but potentially faster than full replanning in dynamic ones. DWA is designed for local real-time planning, but its evaluation window can become complex.

7.3 Sensor usage

While the review broadly mentions cameras, lidar, and ultrasonic sensors (Katona et al. 2024), specific studies provide examples: ultrasonic sensors are used for real-time detection with Dijkstra (Alshammrei et al. 2022), lidar with fuzzy logic controllers (Tuan et al. 2024), and RGB-D cameras are standard inputs for DRL/LLM approaches (Lee and Yusuf 2022) generally, graph-based and sampling-based methods often rely on pre-built maps (derived from sensors like lidar) or assume global knowledge, while reactive (APF, DWA) and some learning-based methods depend heavily on real-time local sensor data (ultrasonic, infrared, short-range lidar, cameras).

7.4 Applicability to critical areas (autonomous driving)

Applying these techniques to fully autonomous driving presents significant challenges. Although components are relevant, A* for route planning (Erke et al. 2020), DWA/APF for local avoidance, and DRL/IL for learning driving policies, no single method reviewed is sufficient. Autonomous driving demands extreme reliability, safety, and real-time performance in highly complex and dynamic environments. Many methods show limitations: A* struggles with dynamism (Guruji et al. 2016), DWA with complex scenarios, APF with local minima, and learning-based methods with sample effi-

ciency and interpretability. Achieving full autonomy requires a sophisticated fusion of multiple techniques, advanced sensor integration (lidar, radar, cameras), robust perception, and decision-making systems far exceeding the scope of individual algorithms discussed here. The current state suggests component applicability rather than complete solutions.

7.5 Comparative analysis

The extensive review of navigation techniques allows a direct comparison of the different paradigms. Each approach presents a distinct set of trade-offs regarding optimality, computational requirements, robustness, and adaptability. This section provides a consolidated qualitative and quantitative comparison based on the findings reported in the surveyed literature.

7.5.1 Qualitative comparison

The Table 17 offers a qualitative synthesis of the key strengths, weaknesses, computational costs, and adaptability of each major navigation method category discussed in this review.

7.5.2 Quantitative comparison and evaluation metrics

The following Table 18 synthesizes quantitative results from a range of studies discussed in this review. It highlights the specific evaluation metrics used for each method and reports the notable performance improvements achieved, offering a comparative snapshot of advancements in algorithm efficiency, path quality, and task success.

7.6 Challenges in autonomous navigation

Several recurring challenges emerge from the reviewed literature:

- Dynamic environments: Handling unpredictable changes (moving obstacles, changing terrain) remains a primary hurdle for many classical methods (Guruji et al. 2016).
- Computational efficiency: Balancing path optimality/completeness with real-time performance constraints is critical (Guruji et al. 2016).
- Scalability: Ensuring methods work efficiently in large, complex, or high-dimensional spaces.
- Optimality vs. Reactivity: Finding the right balance between globally optimal paths and fast, reactive maneuvers.
- Local minima: A specific issue for potential field methods.

Table 17 Comparison of Pathfinding Methodologies

Method Category	Key Strengths	Key Weaknesses	Cost	Adaptability to Dynamic Environments	Examples
Graph Search	- Excels in finding optimal paths in static, known environments. - Guarantees completeness and systematic exploration. - Highly effective in high-dimensional configuration spaces. - Features probabilistic completeness.	- High computational cost, especially in large or complex environments. - Difficulty handling dynamic obstacles. - Paths are often non-optimal (RRT* addresses this).	High	Low for A* and Dijkstra; D* is designed for efficient replanning.	Dijkstra, A*, D*
Sampling-Based	- Simple and reactive, suitable for real-time, local obstacle avoidance. - Powerful for solving complex optimization problems.	- Difficulties navigating narrow passages. - Highly susceptible to local minima, which can trap the robot. - Often suffers from high computational costs.	Moderate / Variable	Can be adapted to handle dynamic elements through optimizations.	RRT, PRM, DWA
Gradient-Based			Low	Reactivity is a strength, but local minima limit reliability in complex scenes.	APF
Bio-Inspired			High	Adaptability is an active area of research, focusing on hybridization.	GA, PSO, ACO
Learning-Based	- Adapts optimal policies through direct interaction with dynamic environments. - Can process high-dimensional sensor inputs. - Leverages common-sense reasoning for high-level commands. - Enables new level of human-robot interaction.	- Suffers from sample inefficiency, requiring extensive training. - Dependent on careful reward shaping. - High computational expense for inference. - Grounding language into real-time actions is a challenge.	High (Training)	Very High. This is its primary advantage for unpredictable environments.	RL, DRL, IL
Large Language Models (LLM)			High	Very High, for semantic adaptability and dynamic replanning from verbal instructions.	DCIP, 3P-LLM

Table 18 Quantitative Performance of Modern Navigation Methods

Method / Study	Evaluation Metric	Reported Quantitative Result
Improved Multi-objective A* (Martins et al. 2022)	Algorithm Processing Time	Reduced processing time by an average of 99.98% compared to the traditional A*.
Improved Dijkstra (Guo et al. 2019)	Search Area / Operation Time	Reduced the search area by over 67% and improved pathfinding operation time by 25%.
RDJ Algorithm (Huang 2018)	Execution Speed	Reported to be at least 10 times faster than both the standard Dijkstra (DJ) and A* algorithms.
A-RRT (Abdouni et al. 2022)	Convergence Time / Path Length	Reduced convergence time by 32.92% and shortened path length by 8.77% compared to RRT.
Energy-Efficient DWA (EDWA) (Henkel et al. 2016)	Energy Consumption	Achieved a 9.79% reduction in energy consumption compared to the standard Dynamic Window Approach.
FGDACO (Sangeetha et al. 2021)	Path Length	Outperformed competing algorithms, achieving path lengths shorter by up to 11% (vs. Fuzzy-GA) and 6% (vs. COA).
Imitation Learning (CNN) (Waga et al. 2023)	Success Rate	The trained model achieved a high success rate of 89.8% when tested on 98 new, unseen environments.
3P-LLM (Latif 2024)	Average Planning Time	Achieved a remarkable 10 ms average planning time, leveraging the generative power of an LLM.

- Sensor noise and uncertainty: Dealing with imperfect sensor data and environmental ambiguity (Wang et al. 2022).
- Safety and reliability: Guaranteeing collision-free and dependable operation, especially in human-populated or critical environments.
- Learning efficiency: Overcoming the sample inefficiency and training requirements of learning-based approaches.
- Integration and grounding: Effectively combining different methods (hybrid approaches) and grounding abstract knowledge (like from LLMs) into physical actions.

The field of mobile robot navigation offers a diverse toolkit, yet no single method achieves universal superiority. Traditional algorithms provide foundational guarantees but often struggle with adaptability. In contrast, modern learning-based and bio-inspired approaches offer greater flexibility, albeit sometimes at the cost of computational resources or sample efficiency. The evident trend towards hybrid systems (Yu et al. 2018; Zhenyang and Wei 2022; Qiuyun et al. 2021; Zheyi and Bing 2021) and the integration of advanced AI, including DRL and LLMs, points towards future progress likely stemming from synergistic combinations tailored to address the persistent difficulties of navigating complex, dynamic, and uncertain real-world environments.

8 Conclusions

This review has charted the evolution of autonomous navigation for mobile robots, from foundational graph-based (e.g., A*, Dijkstra) and sampling-based (e.g., RRT, PRM) algorithms to contemporary optimization (e.g., GA, PSO, ACO) and learning-based methods, including deep reinforcement learning (DRL) and the integration of large language models (LLMs). Our analysis highlights a fundamental trade-off: while traditional methods provide reliable baseline solutions, learning-based approaches such as DRL show significant promise for data-driven adaptation in non-trivial and uncertain environments. Despite substantial progress, significant challenges persist across the field. Key obstacles include ensuring robustness in dynamic settings, managing high computational complexity, overcoming scalability limitations, and mitigating sensitivity to hyperparameter tuning. Furthermore, synthesizing complex constraints and addressing multi-objective planning remain critical areas for improvement. To advance the capabilities of autonomous systems, future research must be directed toward overcoming these specific challenges. Focused efforts on enhancing the robustness, efficiency, and real-world applicability of navigation systems are essential for enabling the reliable operation of autonomous mobile robots in the complex, dynamic environments of the future.

Funding No funding was received for this work. The publication and research costs were covered personally by the corresponding author.

Data Availability All data used in this survey are obtained from publicly available sources and are properly cited within the manuscript.

Declarations

Competing Interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Conflict of Interest The authors declare that they have no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

- A Stars - an overview | ScienceDirect Topics. <https://www.sciencedirect.com/topics/physics-and-astronomy/a-stars>. Accessed 2025-01-08
- Ab Wahab MN, Nazir A, Khalil A, Ho WJ, Akbar MF, Noor MHM, Mohamed ASA (2024) Improved genetic algorithm for mobile robot path planning in static environments. *Expert Syst Appl* 249:123762. <https://doi.org/10.1016/j.eswa.2024.123762>. Accessed 2025-01-08
- Abdouni J, Jarou T, Mzili T, Waga A, Bensassi K (2025) Challenges and constraints in trajectory planning for autonomous robots. *Iraqi J Comput Sci Math* 6(3). <https://doi.org/10.52866/2788-7421.1274>
- Abdouni J, Jarou T, Waga A, El Idrissi S, El mahri M, Sefrioui I (2022) A new sampling strategy to improve the performance of mobile robot path planning algorithms. In: 2022 International Conference on Intelligent Systems and Computer Vision (ISCV), pp 1–7. <https://doi.org/10.1109/ISCV54655.2022.9806128>. ISSN: 2768-0754. <https://ieeexplore.ieee.org/abstract/document/9806128>. Accessed 2025-05-03
- Ahmed A, Abdalla T, Abed A (2015) Path Planning of Mobile Robot Using Fuzzy-Potential Field Method. *Iraqi J Electrical Electron Eng* 11(1):32–41. <https://doi.org/10.37917/ijeee.11.1.4>. Accessed 2025-05-03
- Alferov G, Korolev V, Fedorov V, Khokhriakova A (2024) RETRACTED: Correction of the movement of the mobile robot using the modified algorithm. *E3S Web of Conferences* 549:08005. <https://doi.org/10.1051/e3sconf/202454908005>. Accessed 2025-01-09
- Alshammrei S, Boubaker S, Kolsi L (2022) Improved Dijkstra Algorithm for Mobile Robot Path Planning and Obstacle Avoidance. *Comput, Mater Continua* 72(3):5939–5954. <https://doi.org/10.32604/cmc.2022.028165>. Accessed 2025-01-08
- Alyasin A, Abbas EI, Hasan SD (2019) An Efficient Optimal Path Finding for Mobile Robot Based on Dijkstra Method. In: 2019 4th Scientific International Conference Najaf (SICN), pp 11–14. <https://doi.org/10.1109/SICN47020.2019.9019345>. <https://ieeexplore.ieee.org/document/9019345>. Accessed 2025-01-07
- Arambula Cosío F, Padilla Castañeda MA (2004) Autonomous robot navigation using adaptive potential fields. *Math Comput Model* 40(9):1141–1156. <https://doi.org/10.1016/j.mcm.2004.05.001>. Accessed 2025-01-08
- Balado J, Díaz-Vilariño L, Arias P, Lorenzo H (2019) Point clouds for direct pedestrian pathfinding in urban environments. *ISPRS J Photogrammetry Remote Sens* 148:184–196. <https://doi.org/10.1016/j.isprsjprs.2019.01.004>. Accessed 2025-01-07
- Bhattacharyya S, Karmakar M (2023) Optimal Path Planning with Smart Energy Management Techniques Using Dijkstra's Algorithm. In: Bhattacharyya S, Banerjee JS, Köppen M (eds.) *Human-Centric Smart Computing*, pp 283–291. Springer, Singapore. https://doi.org/10.1007/978-981-19-5403-0_24
- Billard A, Grollman D (2012) Imitation Learning in Robots. In: Seel NM (ed.) *Encyclopedia of the Sciences of Learning*, pp 1494–1496. Springer, Boston, MA. https://doi.org/10.1007/978-1-4419-1428-6_758. Accessed 2025-01-09
- Botvinick M, Ritter S, Wang JX, Kurth-Nelson Z, Blundell C, Hassabis D (2019) Reinforcement Learning, Fast and Slow. *Trends in Cognitive Sci* 23(5):408–422. <https://doi.org/10.1016/j.tics.2019.02.006>. Accessed 2025-05-03
- Bremermann HJ (1958) *The Evolution of Intelligence: The Nervous System as a Model of Its Environment*. University of Washington, Department of Mathematics, ???
- Cao Y, Mohamad Nor N (2024) An improved dynamic window approach algorithm for dynamic obstacle avoidance in mobile robot formation. *Decision Analytics J* 11:100471. <https://doi.org/10.1016/j.dajour.2024.100471>. Accessed 2025-01-08
- Çelik OM, Köseoğlu M (2023) A Modified Dijkstra Algorithm for ROS Based Autonomous Mobile Robots. *J Adv Res Natural Appl Sci* 9(1):205–217. <https://doi.org/10.28979/jarnas.1119957>. Accessed 2025-01-08
- Cèsar-Tondreau B, Warnell G, Stump E, Kochersberger K, Waytowich NR (2021) Improving Autonomous Robotic Navigation Using Imitation Learning. *Front Robot AI* 8. <https://doi.org/10.3389/frobt.2021.627730>. Accessed 2025-05-03
- Chai R, Tsourdos A, Savvaris A, Chai S, Xia Y (2021) Solving constrained trajectory planning problems using biased particle swarm optimization. *IEEE Trans Aerospace Electron Syst* 57(3):1685–1701. <https://doi.org/10.1109/TAES.2021.3050645>. Accessed 2025-01-09
- Cheng W-C, Ni Z, Zhong X, Wei M (2024) Autonomous robot goal seeking and collision avoidance in the physical world: An automated learning and evaluation framework based on the ppo method. *Appl Sci* 14(23). <https://doi.org/10.3390/app142311020>
- Chen G, Luo N, Liu D, Zhao Z, Liang C (2021) Path planning for manipulators based on an improved probabilistic roadmap method. *Robot Comput-Integrated Manufact* 72:102196. <https://doi.org/10.1016/j.rcim.2021.102196>. Accessed 2025-01-08
- Chentoufi A, Fatmi AE, Bekri A, Benhlila S, Sabbane M, (2017) Genetic algorithms and dynamic weighted sum method for RNA alignment. In: (2017) *Intelligent Systems and Computer Vision (ISCV)*, pp 1–5. IEEE, Fez, Morocco. <https://doi.org/10.1109/ISACV.2017.8054965>. <http://ieeexplore.ieee.org/document/8054965/>. Accessed 2025-05-03
- Chen Y, Zhou X (2021) Path planning of robot based on improved ant colony algorithm in computer technology. *J Phys: Conference Series* 1744(4):042092. <https://doi.org/10.1088/1742-6596/1744/4/042092>. Accessed 2025-01-09

- Dai X, Long S, Zhang Z, Gong D (2019) Mobile robot path planning based on ant colony algorithm with a* heuristic method. *Front Neurobot* 13. <https://doi.org/10.3389/fnbot.2019.00015>. Accessed 2025-01-09
- Deshpande S, Kashyap AK, Patle BK (2023) A review on path planning ai techniques for mobile robots. *Robotic Systems and Applications* 3(1):27–46. <https://doi.org/10.21595/rsa.2023.23090>. Accessed 2025-05-03
- Dewang HS, Mohanty PK, Kundu S (2018) A robust path planning for mobile robot using smart particle swarm optimization. *Procedia Computer Science* 133:290–297. <https://doi.org/10.1016/j.procs.2018.07.036>. Accessed 2025-01-09
- Dijkstra Algorithms - an overview | ScienceDirect Topics. <https://www.sciencedirect.com/topics/computer-science/dijkstra-algorithms>. Accessed 2025-01-08
- Dijkstra EW (1959) A note on two problems in connexion with graphs. *Numerische Mathematik* 1(1):269–271. <https://doi.org/10.1007/BF01386390>. Accessed 2025-01-07
- Dobriborsci D, Chichkanov I, Zashchitin R, Osinenko P (2024b) Model-based reinforcement learning experimental study for mobile robot navigation. In: 2024 10th International conference on control, decision and information technologies (CoDIT), pp 1825–1830. <https://doi.org/10.1109/CoDIT62066.2024.10708080>. ISSN: 2576-3555. <https://ieeexplore.ieee.org/document/10708080/>. Accessed 2025-01-09
- Dobriborsci D, Zashchitin R, Kakanov M, Aumer W, Osinenko P (2024a) Predictive reinforcement learning: map-less navigation method for mobile robot. *J Intell Manufac* 35(8):4217–4232. <https://doi.org/10.1007/s10845-023-02197-y>. Accessed 2025-01-09
- Doma P, Arab A, Xiao X (2024) LLM-enhanced path planning: safe and efficient autonomous navigation with instructional inputs <https://doi.org/10.48550/arXiv.2412.02655>. [arXiv:2412.02655](https://arxiv.org/abs/2412.02655) Accessed 2025-05-03
- Dong X, Wang Y, Fang C, Ran K, Liu G (2025) Fhq-rrt*: An improved path planning algorithm for mobile robots to acquire high-quality paths faster. *Sensors* 25(7). <https://doi.org/10.3390/s25072189>
- Dynamic Path Planning Algorithm for Mobile Robots: Leveraging Reinforcement Learning for Efficient Navigation – Journal of Internet Services and Information Security. <https://jis.is.org/article/2024.12.014/71132/>. Accessed 2025-05-03
- Elallid BB, Benamar N, Bagaa M, Hadjadj-Aoul Y (2024) Enhancing autonomous driving navigation using soft actor-critic. *Future Int* 16(7). <https://doi.org/10.3390/fi16070238>
- Erke S, Bin D, Yiming N, Qi Z, Liang X, Dawei Z (2020) An improved A-Star based path planning algorithm for autonomous land vehicles. *Int J Adv Robotic Syst* 17(5):1729881420962263. <https://doi.org/10.1177/1729881420962263>. Accessed 2025-01-07
- Esposito JM, Wright JN (2019) Matrix completion as a post-processing technique for probabilistic roadmaps. *Int J Robot Res* 38(2–3):388–400. <https://doi.org/10.1177/0278364919830554>. Accessed 2025-01-08
- Everett M, Chen YF, How JP (2018) Motion Planning Among Dynamic, Decision-Making Agents with Deep Reinforcement Learning. <https://doi.org/10.48550/arXiv.1805.01956>. [arXiv:1805.01956](https://arxiv.org/abs/1805.01956) Accessed 2025-05-03
- Faisal M, Hedjar R, Al Sulaiman M, Al-Mutib K (2013) Fuzzy Logic Navigation and Obstacle Avoidance by a Mobile Robot in an Unknown Dynamic Environment. *Int J Adv Robot Syst* 10(1):37. <https://doi.org/10.5772/54427>. Accessed 2025-01-08
- Fan H, Huang J, Huang X, Zhu H, Su H (2024) BI-RRT*: An improved path planning algorithm for secure and trustworthy mobile robots systems. *Heliyon* 10(5):26403. <https://doi.org/10.1016/j.heliyon.2024.e26403>. Accessed 2025-01-08
- Fatmi AE, Chentoufi A, Bekri MA, Benhlila S, Sabbane M, (2017) A heuristic algorithm for RNA secondary structure based on genetic algorithm. In: (2017) *Intelligent Systems and Computer Vision (ISCV)*, pp 1–7. IEEE, Fez, Morocco. <https://doi.org/10.1109/ISACV.2017.8054964>. <https://ieeexplore.ieee.org/document/8054964/> Accessed 2025-05-03
- Feng S, Qian Y, Wang Y (2021) Collision avoidance method of autonomous vehicle based on improved artificial potential field algorithm. *Proceed Institution Mech Eng, Part D: J Automobile Eng* 235(14):3416–3430. <https://doi.org/10.1177/09544070211014319>. Accessed 2025-01-08
- Ferguson D, Stentz A (2006) Using interpolation to improve path planning: The Field D* algorithm. *J Field Robot* 23(2):79–101. <https://doi.org/10.1002/rob.20109>. Accessed 2025-01-08
- Fink W, Baker VR, Brooks AJ-W, Flammia M, Dohm JM, Tarbell MA (2019) Globally optimal rover traverse planning in 3D using Dijkstra's algorithm for multi-objective deployment scenarios. *Planetary Space Sci* 179:104707. <https://doi.org/10.1016/j.pss.2019.104707>. Accessed 2025-01-07
- Foad D, Ghifari A, Kusuma MB, Hanafiah N, Gunawan E (2021) A systematic literature review of a* pathfinding. *Procedia Computer Science* 179:507–514. <https://doi.org/10.1016/j.procs.2021.01.034>. Accessed 2025-05-02
- Fox D, Burgard W, Thrun S (1997) The dynamic window approach to collision avoidance. *IEEE Robot Autom Mag* 4(1):23–33. <https://doi.org/10.1109/100.580977>. Accessed 2025-01-08
- Fransen K, Van Eekelen J (2023) Efficient path planning for automated guided vehicles using A* (Astar) algorithm incorporating turning costs in search heuristic. *Int J Product Res* 61(3):707–725. <https://doi.org/10.1080/00207543.2021.2015806>. Accessed 2025-01-07
- Garcia MAP, Montiel O, Castillo O, Sepúlveda R, Melin P (2009) Path planning for autonomous mobile robot navigation with ant colony optimization and fuzzy cost function evaluation. *Appl Soft Comput* 9(3):1102–1110. <https://doi.org/10.1016/j.asoc.2009.02.014>. Accessed 2025-01-09
- Garg S, Devi B (2023) Shortest Path Finding using Modified Dijkstra's algorithm with Adaptive Penalty Function. In: 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT), pp 1–9. <https://doi.org/10.1109/ICCCNT56998.2023.10308130>. ISSN: 2473-7674. <https://ieeexplore.ieee.org/document/10308130?denied=> Accessed 2025-01-07
- Giusti A, Guzzi J, Cireşan DC, He F-L, Rodríguez JP, Fontana F, Faessler M, Forster C, Schmidhuber J, Caro GD, Scaramuzza D, Gambardella LM (2016) A Machine Learning Approach to Visual Perception of Forest Trails for Mobile Robots. *IEEE Robot Autom Lett* 1(2):661–667. <https://doi.org/10.1109/LRA.2015.2509024>. Accessed 2025-05-03
- Guo D, Wang J, Zhao JB, Sun F, Gao S, Li CD, Li MH, Li CC (2019) A vehicle path planning method based on a dynamic traffic network that considers fuel consumption and emissions. *Sci Total Environ* 663:935–943. <https://doi.org/10.1016/j.scitotenv.2019.01.222>. Accessed 2025-01-07
- Guruji AK, Agarwal H, Parsediya DK (2016) Time-efficient a* algorithm for robot path planning. *Procedia Technol* 23:144–149. <https://doi.org/10.1016/j.protcy.2016.03.010>. Accessed 2025-01-07
- Haarhoja T, Zhou A, Abbeel P, Levine S (2018) Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. [arXiv:1801.01290](https://arxiv.org/abs/1801.01290)
- Hamandi M, D'Arcy M, Fazli P (2019) DeepMoTion: Learning to Navigate Like Humans. <https://doi.org/10.48550/arXiv.1803.03719>. [arXiv:1803.03719](https://arxiv.org/abs/1803.03719) Accessed 2025-05-03
- He Z, Liu C, Chu X, Negenborn RR, Wu Q (2022) Dynamic anti-collision A-star algorithm for multi-ship encounter situations. *Appl Ocean Res* 118:102995. <https://doi.org/10.1016/j.apor.2021.102995>. Accessed 2025-01-08

- Henkel C, Bubeck A, Xu W (2016) Energy efficient dynamic window approach for local path planning in mobile service robotics*. IFAC-PapersOnLine 49(15):32–37. <https://doi.org/10.1016/j.ifacol.2016.07.610>. Accessed 2025-01-08
- Holland JH (1992) Genetic Algorithms. <https://www.scientificamerican.com/article/genetic-algorithms/> Accessed 2025-05-03
- Hong Z, Chun-Long S, Zi-Jun Z, Wei A, De-Qiang Z, Jing-Jing W (2015) A modified dynamic window approach to obstacle avoidance combined with fuzzy logic. In: 2015 14th International symposium on distributed computing and applications for business engineering and science (DCABES), pp 523–526. <https://doi.org/10.1109/DCABES.2015.136>. <https://ieeexplore.ieee.org/abstract/document/7429670> Accessed 2025-01-08
- Hou J, Jiang W, Luo Z, Yang L, Hu X, Guo B (2024) Dynamic path planning for mobile robots by integrating improved sparrow search algorithm and dynamic window approach. *Actuators* 13(1):24. <https://doi.org/10.3390/act13010024>. Accessed 2025-01-08
- Huang C (2018) Study on path planning and location of mobile robot based on intelligent optimization algorithm. Dalian Jiaotong University, Dalian, China
- Huang J, Zhang Z, Ruan X (2024) An improved Dyna-Q algorithm inspired by the forward prediction mechanism in the rat brain for mobile robot path planning. *Biomimetics* 9(6):315. <https://doi.org/10.3390/biomimetics9060315>. Accessed 2025-05-03
- Hu Y, Yang SX (2022) A Novel Knowledge-Based Genetic Algorithm for Robot Path Planning in Complex Environments. <https://doi.org/10.48550/arXiv.2209.01482>. [arXiv:2209.01482](https://arxiv.org/abs/2209.01482) Accessed 2025-01-08
- Jiao Z, Ma K, Rong Y, Wang P, Zhang H, Wang S (2018) A path planning method using adaptive polymorphic ant colony algorithm for smart wheelchairs. *J Computat Sci* 25:50–57. <https://doi.org/10.1016/j.jocs.2018.02.004>. Accessed 2025-01-09
- Ji Y, Liu B (2022) Research and implementation of robot path planning based on ant colony algorithm. *J Phys: Conference Series* 2171(1):012074. <https://doi.org/10.1088/1742-6596/2171/1/012074>. Accessed 2025-01-09
- Jingcun W, Xiaotong Z, Bin C, Heping C (2007) A heuristic optimization path-finding algorithm based on Dijkstra algorithm. *Chinese J Eng* 29(3):346–350. <https://doi.org/10.13374/j.issn1001-053x.2007.03.022>. Accessed 2025-01-07
- Kabas B (2022) Autonomous uav navigation via deep reinforcement learning using ppo. In: 2022 30th Signal processing and communications applications conference (SIU), pp 1–4. <https://doi.org/10.1109/SIU55565.2022.9864769>
- Kadry S, Alferov G, Fedorov V (2020) D-Star Algorithm Modification. *Int J Online Biomed Eng (iJOE)* 16(08):108–113. <https://doi.org/10.3991/ijoe.v16i08.14243>. Accessed 2025-05-03
- Kalyan KS (2024) A survey of GPT-3 family large language models including ChatGPT and GPT-4. *Natural Lang Process J* 6:100048. <https://doi.org/10.1016/j.nlp.2023.100048>. Accessed 2025-05-03
- Karaman S, Frazzoli E (2011) Sampling-based algorithms for optimal motion planning. *Int J Robot Res* 30(7):846–894. <https://doi.org/10.1177/0278364911406761>. Accessed 2025-05-03
- Karnan H, Warnell G, Xiao X, Stone P (2021) VOILA: Visual Observation-Only Imitation Learning for Autonomous Navigation. <https://doi.org/10.48550/arXiv.2105.09371>. [arXiv:2105.09371](https://arxiv.org/abs/2105.09371) Accessed 2025-05-03
- Katona K, Neamah HA, Korondi P (2024) Obstacle avoidance and path planning methods for autonomous navigation of mobile robot. *Sensors* 24(11):3573. <https://doi.org/10.3390/s24113573>. Accessed 2025-01-07
- Kennedy J, Eberhart R (1995) Particle swarm optimization. In: Proceedings of ICNN'95 - International Conference on Neural Networks, vol. 4, pp 1942–19484. <https://doi.org/10.1109/ICNN.1995.488968>. <https://ieeexplore.ieee.org/document/488968> Accessed 2025-01-09
- Khatib O (1985) Real-time obstacle avoidance for manipulators and mobile robots. In: 1985 IEEE International conference on robotics and automation proceedings, vol. 2, pp 500–505. <https://doi.org/10.1109/ROBOT.1985.1087247>. <https://ieeexplore.ieee.org/document/1087247> Accessed 2025-01-08
- Kim DK, Chen T (2015) Deep Neural Network for Real-Time Autonomous Indoor Navigation. <https://doi.org/10.48550/ARXIV.1511.04668>. [arXiv:1511.04668](https://arxiv.org/abs/1511.04668) Accessed 2025-05-03
- Kolve E, Mottaghi R, Han W, VanderBilt E, Weihs L, Herrasti A, Deitke M, Ehsani K, Gordon D, Zhu Y, Kembhavi A, Gupta A, Farhadi A (2022) AI2-THOR: An Interactive 3D Environment for Visual AI. [arXiv:1712.05474](https://arxiv.org/abs/1712.05474)
- Krantz J, Wijmans E, Majumdar A, Batra D, Lee S (2020) Beyond the Nav-Graph: Vision-and-Language Navigation in Continuous Environments. [arXiv:2004.02857](https://arxiv.org/abs/2004.02857)
- Lamini C, Benhlila S, Elbekri A (2018) Genetic Algorithm Based Approach for Autonomous Mobile Robot Path Planning. *Procedia Comput Sci* 127:180–189. <https://doi.org/10.1016/j.procs.2018.01.113>. Accessed 2025-01-08
- Lamini C, Fathi Y, Benhlila S (2017) A fuzzy path planning system based on a collaborative reinforcement learning. *Int Rev Autom Control (IREACO)* 10(2):126. <https://doi.org/10.15866/ireaco.v10i2.10623>. Accessed 2025-05-03
- Lamini C, Fathi Y, Benhlila S (2017) H-mas architecture and reinforcement learning method for autonomous robot path planning. In: 2017 Intelligent systems and computer vision (ISCV), pp 1–7. <https://doi.org/10.1109/ISACV.2017.8054978>
- Latif E (2024) 3P-LLM: probabilistic path planning using large language model for autonomous robot navigation. <https://doi.org/10.48550/arXiv.2403.18778>. [arXiv:2403.18778](https://arxiv.org/abs/2403.18778) Accessed 2025-05-03
- Lazarowska A (2019) Discrete artificial potential field approach to mobile robot path planning. IFAC-PapersOnLine 52(8):277–282. <https://doi.org/10.1016/j.ifacol.2019.08.083>. Accessed 2025-01-08
- Lee M-FR, Yusuf SH (2022) Mobile robot navigation using deep reinforcement learning. *Processes* 10(12):2748. <https://doi.org/10.3390/pr10122748>. Accessed 2025-05-03
- Li X (2021) Path planning of intelligent mobile robot based on Dijkstra algorithm. *J Phys: Conference Series* 2083(4):042034. <https://doi.org/10.1088/1742-6596/2083/4/042034>. Accessed 2025-01-08
- Li Y, Chen X (2005) Mobile robot navigation using particle swarm optimization and adaptive NN. In: Wang L, Chen K, Ong YS (eds.) *Advances in Natural Computation*, pp 628–631. Springer, Berlin, Heidelberg. https://doi.org/10.1007/11539902_76
- Likhachev M, Ferguson D, Gordon G, Stentz A, Thrun S (2005) Anytime dynamic a*: an anytime, replanning algorithm. In: Proceedings of the fifteenth international conference on international conference on automated planning and scheduling. ICAPS'05, pp 262–271. AAAI Press, ???
- Li Y, Li J, Zhou W, Yao Q, Nie J, Qi X (2022) Robot path planning navigation for dense planting red Jujube orchards based on the joint improved a* and DWA algorithms under laser SLAM. *Agriculture* 12(9):1445. <https://doi.org/10.3390/agriculture12091445>. Accessed 2025-01-08
- Liu Y, Li X (2022) A hybrid mobile robot path planning scheme based on modified gray wolf optimization and situation assessment. *J Robot* 2022:1–9. <https://doi.org/10.1155/2022/4167170>. Accessed 2025-01-08
- Liu F, Liang S, Xian DX (2014) Optimal Path Planning for Mobile Robot Using Tailored Genetic Algorithm. *TELKOMNIKA Indonesian J Electrical Eng* 12(1):1–9. Accessed 2025-01-08
- Liu B, Liu C (2022) Path planning of mobile robots based on improved RRT algorithm. *J Phys: Conference Series* 2216(1):012020.

- <https://doi.org/10.1088/1742-6596/2216/1/012020>. Accessed 2025-01-08
- Liu L, Wang X, Yang X, Liu H, Li J, Wang P (2023) Path planning techniques for mobile robots: Review and prospect. *Expert Systems with Applications* 227:120254. <https://doi.org/10.1016/j.eswa.2023.120254>. Accessed 2025-05-03
- Liu C, Xie S, Sui X, Huang Y, Ma X, Guo N, Yang F (2023) PRM-D* Method for Mobile Robot Path Planning. *Sensors* 23(7):3512. <https://doi.org/10.3390/s23073512>. Accessed 2025-01-08
- Liu C, Yan X, Liu C, Li G (2010) Dynamic Path Planning for Mobile Robot Based on Improved Genetic Algorithm. *Chinese J Electron* 19(2):245–248. <https://doi.org/10.23919/CJE.2010.10151834>. Accessed 2025-01-08
- Liu S, Zhang J, Wang L, Gao RX (2024) Vision AI-based human-robot collaborative assembly driven by autonomous robots. *CIRP Ann* 73(1):13–16. <https://doi.org/10.1016/j.cirp.2024.03.004>. Accessed 2025-05-03
- Lun G, Xiaoguang Z, Yongcong W (2024) Learning to drive as humans do: Reinforcement learning for autonomous navigation. *Int J Adv Robot Syst* 21(5):17298806241278910. <https://doi.org/10.1177/17298806241278910>, <https://arxiv.org/abs/doi.org/10.1177/17298806241278910>
- Luo J, Wang Z-X, Pan K-L (2022) Reliable Path Planning Algorithm Based on Improved Artificial Potential Field Method. *IEEE Access* 10:108276–108284. <https://doi.org/10.1109/ACCESS.2022.3212741>. Accessed 2025-01-08
- Lu Y, Yi S, Liu Y, Ji Y (2016) A novel path planning method for biomimetic robot based on deep learning. *Assembly Autom* 36(2):186–191. <https://doi.org/10.1108/AA-11-2015-108>. Accessed 2025-05-03
- Ma X, Gong R, Tan Y, Mei H, Li C (2022) Path planning of mobile robot based on improved PRM based on cubic spline. *Wireless Commun Mobile Comput* 2022:1–12. <https://doi.org/10.1155/2022/1632698>. Accessed 2025-01-08
- Mahmud KR, Wang L, Hassan S, Zhang Z (2025) A knowledge-driven framework for Robotic Odor Source Localization using large language models. *Robot Autonomous Syst* 186:104915. <https://doi.org/10.1016/j.robot.2025.104915>. Accessed 2025-05-03
- Mahmud MSA, Abidin MSZ, Mohamed Z, Rahman MKIA, Iida M (2019) Multi-objective path planner for an agricultural mobile robot in a virtual greenhouse environment. *Comput Electron Agric* 157:488–499. <https://doi.org/10.1016/j.compag.2019.01.016>. Accessed 2025-01-08
- Maniezzo V (1991) The ant system: An autocatalytic optimizing process. Italy: Dipartimento di Elettronica, Politecnico di Milano. Accessed 2025-05-03
- Maoudj A, Christensen AL (2021) Q-learning-based navigation for mobile robots in continuous and dynamic environments. In: 2021 IEEE 17th International conference on automation science and engineering (CASE), pp 1338–1345. <https://doi.org/10.1109/CASE49439.2021.9551641>. ISSN: 2161-8089. <https://ieeexplore.ieee.org/document/9551641> Accessed 2025-01-09
- Martins OO, Adekunle AA, Olaniyan OM, Bolaji BO (2022) An Improved multi-objective a-star algorithm for path planning in a large workspace: Design, Implementation, and Evaluation. *Sci African* 15:01068. <https://doi.org/10.1016/j.sciaf.2021.e01068>. Accessed 2025-01-08
- Melchiorre M, Salamina L, Scimmi LS, Mauro S, Pastorelli S (2023) Experiments on the Artificial Potential Field with Local Attractors for Mobile Robot Navigation. *Robotics* 12(3):81. <https://doi.org/10.3390/robotics12030081>. Accessed 2025-01-08
- Meng S, Wang Y, Yang C-F, Peng N, Chang K-W (2025) LLM-A*: large language model enhanced incremental heuristic search on path planning. [arXiv:2407.02511](https://arxiv.org/abs/2407.02511)
- Mirowski P, Pascanu R, Viola F, Soyer H, Ballard AJ, Banino A, Denil M, Goroshin R, Sifre L, Kavukcuoglu K, Kumaran D, Hadsell R (2017) Learning to Navigate in Complex Environments. <https://doi.org/10.48550/arXiv.1611.03673>. [arXiv:1611.03673](https://arxiv.org/abs/1611.03673) Accessed 2025-05-03
- Mnih V, Kavukcuoglu K, Silver D, Graves A, Antonoglou I, Wierstra D, Riedmiller M (2013) Playing Atari with Deep Reinforcement Learning. <https://doi.org/10.48550/arXiv.1312.5602>. [arXiv:1312.5602](https://arxiv.org/abs/1312.5602) Accessed 2025-05-03
- Mohammad N, Bezzi N (2025) Soft Actor-Critic-based Control Barrier Adaptation for Robust Autonomous Navigation in Unknown Environments. [arXiv:2503.08479](https://arxiv.org/abs/2503.08479)
- Mohanty PK, Singh AK, Kumar A, Mahto MK, Kundu S (2022) Path planning techniques for mobile robots: a review. In: Abraham A, Engelbrecht A, Scotti F, Gandhi N, Manghirmalani Mishra P, Fortino G, Sakalauskas V, Pllana S (eds.) *Proceedings of the 13th International Conference on Soft Computing and Pattern Recognition (SoCPaR 2021)*, pp 657–667. Springer, Cham. https://doi.org/10.1007/978-3-030-96302-6_62
- Munaf A, Almusawi ARJ (2024) Integration of Q-Learning and PID Controller for Mobile Robots Trajectory Tracking in Unknown Environments. *J Européen des Systèmes Automatisés* 57(4):1023–1033. <https://doi.org/10.18280/jesa.570410>. Accessed 2025-05-03
- Nasir J, Islam F, Malik U, Ayaz Y, Hasan O, Khan M, Muhammad MS (2013) RRT*-SMART: A Rapid Convergence Implementation of RRT*. *Int J Adv Robot Syst* 10(7):299. <https://doi.org/10.5772/56718>. Accessed 2025-01-08
- Nazir A, Wang Z (2023) A comprehensive survey of ChatGPT: Advancements, applications, prospects, and challenges. *Meta-Radiol* 1(2):100022. <https://doi.org/10.1016/j.metrad.2023.100022>. Accessed 2025-05-03
- Nozari S, Krayani A, Marin P, Marcenaro L, Gomez DM, Regazzoni C (2024) Modeling autonomous vehicle responses to novel observations using hierarchical cognitive representations inspired active inference. *Computers* 13(7):161. <https://doi.org/10.3390/computers13070161>. Accessed 2025-05-03
- Optimal and Efficient Path Planning for Partially-Known Environments. <https://www.ri.cmu.edu/publications/optimal-and-efficient-path-planning-for-partially-known-environments/> Accessed 2025-01-08
- Ouach MK, Eren T (2022) PRM path smoothing by circular arc fillet method for mobile robot navigation. <https://doi.org/10.48550/arXiv.2112.03604>. [arXiv:2112.03604](https://arxiv.org/abs/2112.03604) Accessed 2025-01-08
- Phung MD, Ha QP (2021) Safety-enhanced UAV path planning with spherical vector-based particle swarm optimization. *Appl Soft Comput* 107:107376. <https://doi.org/10.1016/j.asoc.2021.107376>. Accessed 2025-01-09
- Qiao L, Luo X, Luo Q (2022) An optimized probabilistic roadmap algorithm for path planning of mobile robots in complex environments with narrow channels. *Sensors* 22(22):8983. <https://doi.org/10.3390/s22228983>. Accessed 2025-01-08
- Qin H, Wu Z, Pan P, Wang M, Qin J, Ye Z (2022) A new multi-modal planning algorithm based on PRM. *ITM Web of Conferences* 47:02005. <https://doi.org/10.1051/itmconf/20224702005>. Accessed 2025-05-03
- Qiuyun T, Hongyan S, Hengwei G, Ping W (2021) Improved particle swarm optimization algorithm for AGV path planning. *IEEE Access* 9:33522–33531. <https://doi.org/10.1109/ACCESS.2021.3061288>. Accessed 2025-01-09
- Quan H, Li Y, Zhang Y (2020) A novel mobile robot navigation method based on deep reinforcement learning. *Int J Adv Robot Syst* 17(3):1729881420921672. <https://doi.org/10.1177/1729881420921672>. Accessed 2025-05-03
- Rafai ANA, Adzhar N, Jaini NI (2022) A Review on Path Planning and Obstacle Avoidance Algorithms for Autonomous Mobile Robots. *J Robot* 2022:1–14. <https://doi.org/10.1155/2022/2538220>. Accessed 2025-05-03

- Raheem FA, Hameed UI (2018) Path Planning Algorithm using D* Heuristic Method Based on PSO in Dynamic Environment. *Am Sci Res J Eng, Technol, Sci* 49(1):257–271. Accessed 2025-01-08
- Ran K, Wang Y, Fang C, Chai Q, Dong X, Liu G (2024) Improved rrt* path-planning algorithm based on the clothoid curve for a mobile robot under kinematic constraints. *Sensors* 24(23). <https://doi.org/10.3390/s24237812>
- Ravankar AA, Ravankar A, Emaru T, Kobayashi Y (2020) HPPRM: hybrid potential based probabilistic roadmap algorithm for improved dynamic path planning of mobile robots. *IEEE Access* 8:221743–221766. <https://doi.org/10.1109/ACCESS.2020.3043333>. Accessed 2025-01-08
- Ray PP (2023) ChatGPT: A comprehensive review on background, applications, key challenges, bias, ethics, limitations and future scope. *Int Things Cyber-Phys Syst* 3:121–154. <https://doi.org/10.1016/j.iotcps.2023.04.003>. Accessed 2025-05-03
- Regragui A, Benhlila S, Bekri A (2025) Vis-to-nav: Visual autonomous navigation for mobile robots with a limited field of view. In: Farhaoui Y, Herawan T, Lucky Imoize A, Allaoui AE (eds) *Intersection of Artificial Intelligence, Data Science, and Cutting-Edge Technologies: From Concepts to Applications in Smart Environment*. Springer, Cham, pp 196–201
- Research on Automatic Overtaking Control Method Based on Improved Artificial Potential Field Method. <https://chn.oversea.cnki.net/kcms/detail/detail.aspx?filename=1021698938.nh&dbcode=CMFD&dbname=CMFD202201&uniplatform=NZKPT>. Accessed 2025-05-03
- Research on Multi-Robot Target Search Based on Intelligent Optimization Algorithms in Unknown Environments. <https://chn.oversea.cnki.net/kcms/detail/detail.aspx?filename=1020813785.nh&dbcode=CFD&dbname=CFDLAST2021&uniplatform=NZKPT>. Accessed 2025-01-09
- Ruan X, Ren D, Zhu X, Huang J, (2019) Mobile robot navigation based on deep reinforcement learning. In, (2019) Chinese control and decision conference (CCDC), pp 6174–6178. IEEE, Nanchang, China. <https://doi.org/10.1109/CCDC.2019.8832393>. <https://ieeexplore.ieee.org/document/8832393/>. Accessed 2025-05-03
- Samadi Gharajeh M, Jond HB (2022) An intelligent approach for autonomous mobile robots path planning based on adaptive neuro-fuzzy inference system. *Ain Shams Eng J* 13(1):101491. <https://doi.org/10.1016/j.asej.2021.05.005>. Accessed 2025-01-08
- Sangeetha V, Krishankumar R, Ravichandran KS, Cavallaro F, Kar S, Pamucar D, Mardani A (2021) A Fuzzy Gain-Based Dynamic Ant Colony Optimization for Path Planning in Dynamic Environments. *Symmetry* 13(2):280. <https://doi.org/10.3390/sym13020280>. Accessed 2025-01-08
- Savva M, Kadian A, Maksymets O, Zhao Y, Wijmans E, Jain B, Straub J, Liu J, Koltun V, Malik J, Parikh D, Batra D (2019) Habitat: A Platform for Embodied AI Research. [arXiv:1904.01201](https://arxiv.org/abs/1904.01201)
- Saxena P, Raghuvanshi N, Goveas N (2025) UAV-VLN: End-to-End Vision Language guided Navigation for UAVs. [arXiv:2504.21432](https://arxiv.org/abs/2504.21432)
- Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O (2017) Proximal Policy Optimization Algorithms. [arXiv:1707.06347](https://arxiv.org/abs/1707.06347)
- Sfeir J, Saad M, Saliha-Hassane H (2011) An improved artificial potential field approach to real-time mobile robot path planning in an unknown environment. In: 2011 IEEE International symposium on robotic and sensors environments (ROSE), pp 208–213. <https://doi.org/10.1109/ROSE.2011.6058518>. <https://ieeexplore.ieee.org/document/6058518>. Accessed 2025-01-08
- Shahabi C, Sharifzadeh M (2018) Voronoi diagrams. In: Liu L, Özsu MT (eds.) *Encyclopedia of Database Systems*, pp 4580–4582. Springer, New York, NY. https://doi.org/10.1007/978-1-4614-8265-9_451. Accessed 2025-01-08
- Shin Y, Kim E (2021) Hybrid path planning using positioning risk and artificial potential fields. *Aerospace Sci Technol* 112:106640. <https://doi.org/10.1016/j.ast.2021.106640>. Accessed 2025-01-08
- Shivgan R, Dong Z (2020) Energy-Efficient Drone Coverage Path Planning using Genetic Algorithm. In: 2020 IEEE 21st International conference on high performance switching and routing (HPSR), pp 1–6. <https://doi.org/10.1109/HPSR48589.2020.9098989>. ISSN: 2325-5609. <https://ieeexplore.ieee.org/document/9098989>. Accessed 2025-01-08
- Shridhar M, Thomason J, Gordon D, Bisk Y, Han W, Mottaghi R, Zettlemoyer L, Fox D (2020) ALFRED: A Benchmark for Interpreting Grounded Instructions for Everyday Tasks. [arXiv:1912.01734](https://arxiv.org/abs/1912.01734)
- Singh R, Ren J, Lin X (2023) A review of deep reinforcement learning algorithms for mobile robot path planning. *Vehicles* 5(4):1423–1451. <https://doi.org/10.3390/vehicles5040078>. Accessed 2025-05-03
- Steven M. La Valle. <https://lavalle.pl/rtrpubs.html>. Accessed 2025-05-03
- Sun L, Kanezaki A, Caron G, Yoshiyasu Y (2025) Enhancing multimodal-input object goal navigation by leveraging large language models for inferring room-object relationship knowledge. *Adv Eng Inf* 65:103135. <https://doi.org/10.1016/j.aei.2025.103135>. Accessed 2025-05-03
- Sun Y, Qiu Y, Aoki Y (2025) Dynamicvln: Incorporating dynamics into vision-and-language navigation scenarios. *Sensors* 25(2). <https://doi.org/10.3390/s25020364>
- Sun Y, Wang W, Xu M, Huang L, Shi K, Zou C, Chen B (2023) Local path planning for mobile robots based on fuzzy dynamic window algorithm. *Sensors* 23(19):8260. <https://doi.org/10.3390/s23198260>. Accessed 2025-01-08
- Surmann H, Jestel C, Marchel R, Musberg F, Elhadj H, Ardani M (2020) Deep Reinforcement learning for real autonomous mobile robot navigation in indoor environments. <https://doi.org/10.48550/arXiv.2005.13857>. [arXiv:2005.13857](https://arxiv.org/abs/2005.13857). Accessed 2025-05-03
- Taheri H, Hosseini SR, Nekoui MA (2024) Deep Reinforcement Learning with Enhanced PPO for Safe Mobile Robot Navigation. [arXiv:2405.16266](https://arxiv.org/abs/2405.16266)
- Tai L, Li S, Liu M (2016) A deep-network solution towards model-less obstacle avoidance. In: 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp 2759–2764. IEEE, Daejeon, South Korea. <https://doi.org/10.1109/IROS.2016.7759428>. <https://ieeexplore.ieee.org/document/7759428/>. Accessed 2025-05-03
- Tang H, Yuan C, Li Z, Tang J (2022) Learning attention-guided pyramidal features for few-shot fine-grained recognition. *Pattern Recognition* 130:108792. <https://doi.org/10.1016/j.patcog.2022.108792>
- Tang H, Li Z, Zhang D, He S, Tang J (2025) Divide-and-conquer: confluent triple-flow network for rgb-t salient object detection. *IEEE Trans Pattern Anal Mach Intell* 47(3):1958–1974. <https://doi.org/10.1109/TPAMI.2024.3511621>
- Tang H, Li Z, Peng Z, Tang J (2020) Blockmix: Meta regularization and self-calibrated inference for metric-based meta-learning. In: *Proceedings of the 28th ACM international conference on multimedia*. MM '20, pp 610–618. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3394171.3413884>
- Tang H, Liu J, Yan S, Yan R, Li Z, Tang J (2023) M3net: Multi-view encoding, matching, and fusion for few-shot fine-grained action recognition. In: *Proceedings of the 31st ACM international conference on multimedia*. MM '23, pp 1719–1728. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3581783.3612221>
- Tao B, Kim J-H (2024) Mobile robot path planning based on bi-population particle swarm optimization with random perturbation strategy. *J King Saud University - Comput Inf Sc* 36(2):101974. <https://doi.org/10.1016/j.jksuci.2024.101974>. Accessed 2025-01-09

- Tao Y, Wen Y, Gao H, Wang T, Wan J, Lan J (2022) A Path-Planning Method for Wall Surface Inspection Robot Based on Improved Genetic Algorithm. *Electronics* 11(8):1192. <https://doi.org/10.3390/electronics11081192>. Accessed 2025-01-08
- Tariq MT, Hussain Y, Wang C (2025) Robust mobile robot path planning via LLM-based dynamic waypoint generation. *Expert Syst Appl* 282:127600. <https://doi.org/10.1016/j.eswa.2025.127600>. Accessed 2025-05-03
- Tsai Y-HH, Dhar V, Li J, Zhang B, Zhang J (2023) Multimodal large language model for visual navigation. *arXiv:2310.08669*
- Tuan PM, Tai ND, Huy TQ, Thanh NT (2024) Flexible path planning of mobile robot for avoiding the dynamic obstacles using fuzzy controllers. *Int J Mech Eng Robot Res* 13(1):126–132. <https://doi.org/10.18178/ijmerr.13.1.126-132>. Accessed 2025-01-08
- Waga A, Ba-ichou A, Benhlima S, Bekri A, Abdouni J (2024) Efficient autonomous navigation for mobile robots using machine learning. *IAES Int J Artif Intell (IJ-AI)* 13(3):3061–3071. <https://doi.org/10.11591/ijai.v13.i3.pp3061-3071>. Accessed 2025-05-03
- Waga A, Benhlima S, Bekri A, Abdouni J (2025) A novel approach for end-to-end navigation for real mobile robots using a deep hybrid model. *Intell Serv Robot* 18(1):75–95. <https://doi.org/10.1007/s11370-024-00569-8>. Accessed 2025-05-03
- Waga A, Benhouria Y, Ba-ichou A, Benhlima S, Bekri A, Abdouni J (2023) A New Method for Mobile Robots to Learn an Optimal Policy from an Expert Using Deep Imitation Learning. In: Motahhir S, Bossoufi B (eds.) *Digital Technologies and Applications*, pp 873–882. Springer, Cham. https://doi.org/10.1007/978-3-031-29857-8_87
- Waga A, Lamini C, Benhlima S, Bekri A (2021) Fuzzy logic obstacle avoidance by a nao robot in unknown environment. In: 2021 Fifth international conference on intelligent computing in data sciences (ICDS), pp 1–7. <https://doi.org/10.1109/ICDS53782.2021.9626718>
- Wang J (2023) Intelligent Path Planning of Mobile Robot Based on Genetic Algorithm. *J Phys: Conference Series* 2547(1):012001. <https://doi.org/10.1088/1742-6596/2547/1/012001>. Accessed 2025-01-08
- Wang C, Cheng C, Yang D, Pan G, Zhang F (2022) Path Planning in Localization Uncertain Environment Based on Dijkstra Method. *Front Neurorobot* 16:821991. <https://doi.org/10.3389/fnbot.2022.821991>. Accessed 2025-01-07
- Wang D, Chen S, Zhang Y, Liu L (2021) Path planning of mobile robot in dynamic environment: fuzzy artificial potential field and extensible neural network. *Artif Life Robot* 26(1):129–139. <https://doi.org/10.1007/s10015-020-00630-6>. Accessed 2025-01-08
- Wang H, Li G, Hou J, Chen L, Hu N (2022) A path planning method for underground intelligent vehicles based on an improved RRT* Algorithm. *Electronics* 11(3):294. <https://doi.org/10.3390/electronics11030294>. Accessed 2025-01-08
- Wang J, Zheng E (2024) Path planning of a mobile robot based on the improved rapidly exploring random trees star algorithm. *Electronics* 13(12):2340. <https://doi.org/10.3390/electronics13122340>. Accessed 2025-01-08
- Wondosen A, Shiferaw D (2024) Fuzzy Logic Controller Design for Mobile Robot Outdoor Navigation. <https://doi.org/10.48550/arXiv.2401.01756>. *arXiv:2401.01756* Accessed 2025-01-09
- Wu B, Chi X, Zhao C, Zhang W, Lu Y, Jiang D (2022) Dynamic path planning for forklift AGV based on smoothing a* and improved DWA hybrid algorithm. *Sensors* 22(18):7079. <https://doi.org/10.3390/s22187079>. Accessed 2025-01-08
- Wu K, Wang H, Esfahani MA, Yuan S (2022) Achieving real-time path planning in unknown environments through deep neural networks. *IEEE Trans Intell Transport Syst* 23(3):2093–2102. <https://doi.org/10.1109/TITS.2020.3031962>. Accessed 2025-05-03
- Xin W, Wanlin L, Chao F, Likai H (2019) Path planning research based on an improved a* algorithm for mobile robot. *IOP Conference Series: Mater Sci Eng* 569(5):052044. <https://doi.org/10.1088/1757-899X/569/5/052044>. Accessed 2025-01-07
- Yang Z, Li N, Zhang Y, Li J (2023) Mobile robot path planning based on improved particle swarm optimization and improved dynamic window approach. *J Robot* 2023:1–16. <https://doi.org/10.1155/2023/6619841>. Accessed 2025-01-09
- Yang H, Teng X (2022) Mobile robot path planning based on enhanced dynamic window approach and improved a* algorithm. *J Robot* 2022:1–9. <https://doi.org/10.1155/2022/2183229>. Accessed 2025-01-08
- Ye L, Chen J, Zhou Y (2022) Real-time path planning for robot using OP-PRM in complex dynamic environment. *Front Neurorobotics* 16:910859. <https://doi.org/10.3389/fnbot.2022.910859>. Accessed 2025-01-08
- Ye J, Papaioannou S, Kolios P (2025) Vlm-rrt: Vision language model guided rrt search for autonomous uav navigation. In: 2025 International conference on unmanned aircraft systems (ICUAS), pp 633–640. IEEE, ??? <https://doi.org/10.1109/icuas65942.2025.11007837>
- Yin Y, Chen Z, Liu G, Guo J (2023) A mapless local path planning approach using deep reinforcement learning framework. *Sensors* 23(4):2036. <https://doi.org/10.3390/s23042036>. Accessed 2025-05-03
- Yonetani R, Taniat T, Barekatin M, Nishimura M, Kanazaki A (2021) Path planning using neural a* search. <https://doi.org/10.48550/arXiv.2009.07476>. *arXiv:2009.07476* Accessed 2025-01-07
- Yuan J, Wang H, Lin C, Liu D, Yu D (2019) A novel GRU-RNN network model for dynamic path planning of mobile robot. *IEEE Access* 7:15140–15151. <https://doi.org/10.1109/ACCESS.2019.2894626>. Accessed 2025-05-03
- Yue L, Zhou D, Xie L, Zhang F, Yan Y, Yin E (2024) Safe-rln: collision avoidance for vision-and-language navigation of autonomous robots operating in continuous environments. *IEEE Robot Automation Lett* 9(6):4918–4925. <https://doi.org/10.1109/lra.2024.3387171>
- Yu L, Kong D, Shao X, Yan X (2018) A Path Planning and Navigation Control System Design for Driverless Electric Bus. *IEEE Access* 6:53960–53975. <https://doi.org/10.1109/ACCESS.2018.2868339>. Accessed 2025-01-07
- Yun SC, Parasuraman S, Ganapathy V (2013) Mobile Robot Navigation: Neural Q-Learning. In: Meghanathan N, Nagamalai D, Chaki N (eds.) *Advances in Computing and Information Technology*, pp 259–268. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-31600-5_26
- Zadeh LA (1965) Fuzzy sets. *Inf. Control* 8(3):338–353. [https://doi.org/10.1016/S0019-9958\(65\)90241-X](https://doi.org/10.1016/S0019-9958(65)90241-X). Accessed 2025-05-03
- Zagradjanin N, Rodic A, Pamucar D, Pavkovic B (2021) Cloud-based multi-robot path planning in complex and crowded environment using fuzzy logic and online learning. *Inf Technol Control* 50(2):357–374. <https://doi.org/10.5755/j01.itc.50.2.28234>. Accessed 2025-01-08
- Zhang H-y, Lin W-m, Chen A-x (2018) Path Planning for the Mobile Robot: A Review. *Symmetry* 10(10):450. <https://doi.org/10.3390/sym10100450>. Accessed 2025-05-03
- Zhang K, Niroui F, Ficocelli M, Nejat G (2018) Robot navigation of environments with unknown rough terrain using deep reinforcement learning. In: 2018 IEEE International symposium on safety, security, and rescue robotics (SSRR), pp 1–7. <https://doi.org/10.1109/SSRR.2018.8468643>. ISSN: 2475-8426. <https://ieeexplore.ieee.org/document/8468643> Accessed 2025-05-03
- Zhang S, Pu J, Si Y, Sun L (2021) Path planning for mobile robot using an enhanced ant colony optimization and path geometric optimization. *Int J Adv Robot Syst* 18(3):17298814211019222. <https://doi.org/10.1177/17298814211019222>. Accessed 2025-01-09

- Zhang Z, Yang H, Bai X, Zhang S, Xu C (2025) The path planning of mobile robots based on an improved genetic algorithm. *Appl Sc* 15(7). <https://doi.org/10.3390/app15073700>
- Zheng L, Yu W, Li G, Qin G, Luo Y (2023) Particle swarm algorithm path-planning method for mobile robots based on artificial potential fields. *Sensors* 23(13):6082. <https://doi.org/10.3390/s23136082>. Accessed 2025-01-09
- Zhenyang X, Wei Y (2022) Mobile robot path planning based on fusion of improved A* algorithm and adaptive DWA algorithm. *J Phys: Conference Series* 2330(1):012003. <https://doi.org/10.1088/1742-6596/2330/1/012003>. Accessed 2025-01-08
- Zheyi C, Bing X (2021) AGV Path Planning Based on Improved Artificial Potential Field Method. In: 2021 IEEE International conference on power electronics, computer applications (ICPECA), pp 32–37. <https://doi.org/10.1109/ICPECA51329.2021.9362519>. <https://ieeexplore.ieee.org/document/9362519> Accessed 2025-01-08
- Zhu Y, Mottaghi R, Kolve E, Lim JJ, Gupta A, Fei-Fei L, Farhadi A (2017) Target-driven visual navigation in indoor scenes using deep reinforcement learning. In: 2017 IEEE International conference on robotics and automation (ICRA), pp 3357–3364. <https://doi.org/10.1109/ICRA.2017.7989381>. <https://ieeexplore.ieee.org/document/7989381> Accessed 2025-05-03
- Ziegler J, Werling M, Schroder J (2008) Navigating car-like robots in unstructured environments using an obstacle sensitive cost function. In: 2008 IEEE Intelligent Vehicles Symposium, pp 787–791. <https://doi.org/10.1109/IVS.2008.4621302>. ISSN: 1931-0587. <https://ieeexplore.ieee.org/document/4621302/> Accessed 2025-01-08

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.